

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



BÁO CÁO
ĐỒ ÁN TỐT NGHIỆP
HỌC KỲ: 252 - NĂM HỌC: 2025-2026

HỆ THỐNG THƯƠNG MẠI ĐIỆN TỬ CHO BÁN LẺ
ĐIỆN THOẠI TÍCH HỢP TRỢ LÝ ẢO AI AGENT

NGÀNH: KHOA HỌC MÁY TÍNH

HỘI ĐỒNG: 1L Đồ án chuyên ngành

GVHD: PGS. TS. Quản Thành Thơ

TKHD: ThS. Huỳnh Phúc Nghị

—o0o—

SVTH1: Phạm Quốc Toàn - 2213539

SVTH2: Nguyễn Ngọc Quế Chi - 2210371

SVTH3: Trần Nguyễn Anh Khoa - 2211651

TP. Hồ Chí Minh, Tháng 05/2026

Chữ ký giảng viên hướng dẫn

Ngày:

Phó giáo sư, Tiến sĩ Quản Thành Thơ (giảng viên hướng dẫn)
Khoa Khoa học và Kỹ thuật Máy tính

Lời cảm ơn

Nhóm chúng em xin bày tỏ lòng biết ơn sâu sắc đến Phó Giáo sư, Tiến sĩ Quản Thành Thơ – người đã tận tình hướng dẫn, chỉ bảo và đồng hành cùng chúng em trong suốt quá trình thực hiện đề án. Những định hướng quý báu, những góp ý xác đáng cùng sự hỗ trợ tận tâm của Thầy đã giúp chúng em hoàn thiện hơn về kiến thức, tư duy nghiên cứu và kỹ năng thực hành.

Bên cạnh đó, nhóm cũng xin chân thành cảm ơn các thầy cô Trường Đại học Bách khoa, Đại học Quốc gia Thành phố Hồ Chí Minh đã truyền đạt cho chúng em nền tảng kiến thức vững chắc trong suốt quá trình học tập, tạo điều kiện thuận lợi để chúng em có thể vận dụng vào việc thực hiện đề tài.

Nhóm cũng xin gửi lời cảm ơn đến gia đình và bạn bè, những người luôn ở bên động viên, khích lệ và tiếp thêm động lực để chúng em vượt qua những khó khăn trong quá trình học tập và nghiên cứu. Sự tin tưởng và ủng hộ của mọi người chính là nguồn động lực to lớn giúp chúng em hoàn thành đề án.

Cuối cùng, nhóm xin gửi lời cảm ơn chân thành đến tất cả những ai đã đóng góp, hỗ trợ dù trực tiếp hay gián tiếp cho sự thành công của đề án này.

Nhóm thực hiện,

Phạm Quốc Toàn – Nguyễn Ngọc Quế Chi – Trần Nguyễn Anh Khoa

Lời cam đoan

Nhóm chúng em xin cam đoan rằng đề tài “Hệ thống Thương mại Điện tử cho Bán lẻ Điện thoại tích hợp Trợ lý Ảo AI Agent” là kết quả tìm hiểu, nghiên cứu và thực hiện của chính nhóm chúng em, dưới sự hướng dẫn của Phó Giáo sư, Tiến sĩ Quản Thành Thơ, thuộc Khoa Khoa học và Kỹ thuật Máy tính – Trường Đại học Bách khoa, Đại học Quốc gia Thành phố Hồ Chí Minh.

Toàn bộ nội dung, số liệu, hình ảnh và kết quả trình bày trong đồ án đều do nhóm tự thực hiện, tổng hợp từ quá trình nghiên cứu, thiết kế và triển khai hệ thống. Những tài liệu, công trình, hay kết quả nghiên cứu của người khác được sử dụng hoặc tham khảo trong đồ án đều đã được trích dẫn và ghi nguồn đầy đủ, đúng quy định.

Nhóm hoàn toàn chịu trách nhiệm về tính trung thực và tính hợp pháp của toàn bộ nội dung trong đồ án này. Nếu có bất kỳ sai phạm nào liên quan đến đạo văn, vi phạm bản quyền hoặc gian lận học thuật, nhóm xin hoàn toàn chịu mọi hình thức xử lý theo quy định của Trường Đại học Bách khoa, Đại học Quốc gia Thành phố Hồ Chí Minh.

TP. Hồ Chí Minh, Tháng 05/2026

Nhóm thực hiện,

Phạm Quốc Toàn – Nguyễn Ngọc Quế Chi – Trần Nguyễn Anh Khoa

Tóm tắt

Đề tài “Hệ thống Thương mại Điện tử cho Bán lẻ Điện thoại tích hợp Trợ lý Ảo AI Agent” hướng đến việc phát triển một nền tảng mua sắm trực tuyến thông minh, ứng dụng công nghệ trí tuệ nhân tạo nhằm nâng cao trải nghiệm người dùng trong lĩnh vực bán lẻ điện thoại và các thiết bị công nghệ. Hệ thống được xây dựng như một nền tảng thương mại điện tử tích hợp, nơi người dùng có thể tra cứu thông tin sản phẩm, so sánh các lựa chọn và tương tác trực tiếp với Trợ lý Ảo AI Agent bằng ngôn ngữ tự nhiên.

Trong quá trình triển khai, nhóm tập trung hiện thực hóa các chức năng cốt lõi của hệ thống, bao gồm xây dựng nền tảng thương mại điện tử và phát triển Trợ lý Ảo AI Agent dựa trên công nghệ xử lý ngôn ngữ tự nhiên (Natural Language Processing – NLP). AI Agent được thiết kế với khả năng hiểu ngữ cảnh và hỗ trợ người dùng thông qua các chức năng chính như tư vấn sản phẩm, so sánh sản phẩm và giải đáp các câu hỏi liên quan. Các chức năng này nhằm hỗ trợ hiệu quả quá trình tìm hiểu thông tin và ra quyết định mua sắm của người dùng.

Trong phạm vi báo cáo này, nhóm trình bày quá trình xây dựng và tích hợp các thành phần của hệ thống, bao gồm phát triển backend, frontend, cơ sở dữ liệu và module AI Agent; đồng thời mô tả cách thức các thành phần này phối hợp để đảm bảo hệ thống hoạt động ổn định và nhất quán. Bên cạnh đó, nhóm cũng tiến hành kiểm thử các chức năng chính và đánh giá bước đầu về hiệu năng, độ chính xác trong phản hồi của AI Agent cũng như trải nghiệm người dùng.

Kết quả đạt được là một hệ thống thương mại điện tử hoàn chỉnh với Trợ lý Ảo AI Agent có khả năng tương tác tự nhiên, cung cấp thông tin chính xác và hỗ trợ người dùng hiệu quả trong quá trình tìm kiếm và lựa chọn sản phẩm. Hệ thống là minh chứng cho khả năng ứng dụng thực tiễn của AI trong việc nâng cao chất lượng dịch vụ và trải nghiệm người dùng trong lĩnh vực thương mại điện tử.

Mục lục

1	Giới thiệu đề tài	1
1.1	Lý do chọn đề tài	1
1.2	Mục tiêu đề tài	1
1.3	Phạm vi đề tài	2
2	Phân tích yêu cầu	3
2.1	Khảo sát thị trường	3
2.1.1	CellphoneS	3
2.1.2	FPT Shop	4
2.1.3	Thế Giới Di Động	5
2.1.4	So sánh tính năng giữa các nền tảng hiện có và hệ thống đề xuất	6
2.2	Phân tích các bên liên quan	7
2.2.1	Khách hàng	7
2.2.2	Đội ngũ vận hành và quản trị viên	7
2.2.3	Chủ sở hữu doanh nghiệp	7
2.3	Yêu cầu chức năng	8
2.3.1	Đối với khách hàng	8
2.3.2	Đối với quản trị viên	9
2.4	Yêu cầu phi chức năng	9
2.4.1	Tính ổn định và trải nghiệm người dùng	9
2.4.2	Bảo mật, tin cậy và khả năng bảo trì	10
2.5	Sơ đồ và đặc tả các trường hợp sử dụng	10
2.5.1	Sơ đồ các trường hợp sử dụng	10
2.5.2	Bảng mô tả chi tiết các trường hợp sử dụng	11
2.6	Sơ đồ hoạt động	17
2.7	Sơ đồ tuần tự	23
3	Cơ sở lý thuyết	30

3.1	Lý thuyết về kiến trúc hệ thống	30
3.1.1	Kiến trúc Monolithic	30
3.1.2	Kiến trúc Microservices	30
3.1.3	Kiến trúc EDA + Cloud-Native	31
3.1.4	So sánh giữa các kiến trúc hệ thống	32
3.2	Lý thuyết về AI Agent	33
3.2.1	Trí tuệ nhân tạo	33
3.2.2	AI Agent	33
3.2.3	ReAct	33
3.2.4	RAG	34
3.2.5	Đồ thị tri thức	34
3.2.6	Hệ thống đề xuất	35
4	Công nghệ	36
4.1	Backend	36
4.1.1	Framework	36
4.1.2	Message Broker	38
4.1.3	Reactive Framework	39
4.1.4	Tổng hợp các công nghệ sử dụng tại Backend	39
4.2	AI Agent	40
4.2.1	Cơ sở dữ liệu quan hệ: PostgreSQL / Supabase	40
4.2.2	Cơ sở dữ liệu đồ thị: Neo4j / AuraDB	41
4.2.3	Cơ sở dữ liệu vector: ChromaDB	41
4.2.4	Công nghệ xây dựng AI Agent: LangGraph	41
4.2.5	Công nghệ xây dựng API: FastAPI	42
4.2.6	Công nghệ triển khai: Railway	42
4.3	Hạ tầng và triển khai	42
4.3.1	Cơ sở dữ liệu	43
4.3.2	Công cụ đóng gói	44

4.3.3	Công cụ quản lí phiên bản	44
4.3.4	Tóm tắt công nghệ	45
5	Thiết kế hệ thống	46
5.1	Thiết kế kiến trúc	46
5.1.1	Lựa chọn kiến trúc	46
5.1.2	Các lựa chọn thiết kế trong kiến trúc EDA + Cloud-Native . . .	47
5.1.3	Các tầng của hệ thống	51
5.2	Thiết kế cơ sở dữ liệu	54
5.2.1	ERD chi tiết cho user service	56
5.2.2	ERD chi tiết cho product service	56
5.2.3	ERD chi tiết cho cart service	57
5.2.4	ERD chi tiết cho order service	58
5.2.5	ERD chi tiết cho admin service	59
5.3	Thiết kế API	60
5.3.1	Mục tiêu của thiết kế API	60
5.3.2	Nguyên tắc thiết kế API	60
5.3.3	Cấu trúc mẫu của Request/Response trong API	61
5.3.4	User Service	62
5.3.5	Product Service	63
5.3.6	Cart Service	63
5.3.7	Orchestration Service	63
5.3.8	Order Service	64
5.3.9	Admin Service	64
5.4	Thiết kế các topic Kafka cho hệ thống	65
5.4.1	Nhóm Topic đồng bộ hóa dữ liệu	65
5.4.2	Nhóm Topic điều phối giao dịch Saga	66
5.5	Thiết kế AI Agent	67
5.5.1	Thiết kế cơ sở dữ liệu	67

5.5.2	Thiết kế AI Agent	69
5.5.3	Thiết kế API	72
6	Hiện thực và triển khai hệ thống	75
6.1	Backend	75
6.1.1	Hiện thực Backend	75
6.1.2	Triển khai Backend	76
6.2	AI Agent	78
6.2.1	Hiện thực AI Agent	78
6.2.2	Triển khai AI Agent	80
7	Kiểm thử và đánh giá hệ thống	81
7.1	Kiểm thử luồng nghiệp vụ cuối	81
7.1.1	Tính năng tìm kiếm và lọc sản phẩm	81
7.1.2	Tính năng thêm sản phẩm vào giỏ hàng và đặt hàng	83
7.1.3	Tính năng thanh toán đa phương thức	86
7.1.4	Tài liệu quản lý kiểm thử chi tiết	88
7.2	Kiểm thử yêu cầu phi chức năng	88
7.2.1	Kiểm thử tính khả dụng và trải nghiệm người dùng	88
7.2.2	Kiểm thử hiệu năng hệ thống và AI Agent	88
7.2.3	Kiểm thử bảo mật và toàn vẹn dữ liệu phân tán	88
7.3	Unit testing cho các thành phần logic backend	88
7.3.1	Chiến lược và mô hình kiểm thử	90
7.3.2	Công cụ thực hiện chuyên sâu	90
7.3.3	Phân loại và các kịch bản kiểm thử tiêu biểu	90
7.3.4	Tổng hợp và thống kê kết quả kiểm thử đơn vị	94
7.4	Kiểm thử API với SwaggerUI	94
7.4.1	Thiết lập môi trường kiểm thử	94
7.4.2	Phạm vi và kịch bản tiêu biểu	95
7.5	Kiểm thử AI Agent	98

7.5.1	Kiểm thử node Parse	98
7.5.2	Kiểm thử node Resolve	99
7.5.3	Kiểm thử tìm kiếm ngữ nghĩa	99
7.5.4	Kiểm thử tích hợp API	100
7.6	Tổng kết kiểm thử	100
7.6.1	Hạn chế và hướng cải thiện	100
8	Kết luận và hướng phát triển	101
	Tài liệu tham khảo	102

Danh sách hình vẽ

2.1	Trang chủ của CellphoneS	3
2.2	Trang chủ của FPT Shop	4
2.3	Trang chủ của Thế Giới Di Động	5
2.4	Trợ lý ảo của Thế Giới Di Động	6
2.5	Use case diagram tổng thể của hệ thống	11
2.6	Sơ đồ hoạt động Tìm kiếm sản phẩm	18
2.7	Sơ đồ hoạt động Thêm vào giỏ hàng và Đặt hàng	19
2.8	Sơ đồ hoạt động Lựa chọn phương thức thanh toán	20
2.9	Sơ đồ hoạt động Phản hồi ý kiến khách hàng	21
2.10	Sơ đồ hoạt động Xem báo cáo thống kê	22
2.11	Sơ đồ hoạt động Tư vấn mua sắm	22
2.12	Sơ đồ hoạt động Hỗ trợ sau bán hàng	23
2.13	Sơ đồ tuần tự Tìm kiếm sản phẩm	24
2.14	Sơ đồ tuần tự Thêm vào giỏ và Đặt hàng	25
2.15	Sơ đồ tuần tự Thanh toán đa phương thức	26
2.16	Sơ đồ tuần tự Gửi phản hồi khách hàng	27
2.17	Sơ đồ tuần tự Xem báo cáo thống kê	28
2.18	Sơ đồ tuần tự Tư vấn mua sắm	28
2.19	Sơ đồ tuần tự Hỗ trợ sau mua hàng	29
5.1	Sơ đồ kiến trúc hệ thống	51
5.2	Cấu trúc nội bộ Microservice của các dịch vụ Java	53
5.3	Sơ đồ ERD	55
5.4	ERD của orchestrator service	55
5.5	ERD chi tiết cho user service	56
5.6	ERD chi tiết của product service	57
5.7	ERD chi tiết của cart service	58
5.8	ERD chi tiết của order service	59
5.9	ERD chi tiết của admin service	60

5.10	Sơ đồ kiến trúc AI Agent	70
6.1	Giao diện quản lý Virtual Machine (myVm) trên Microsoft Azure Portal	76
6.2	Quá trình thực thi lệnh docker build để đóng gói dịch vụ trên máy ảo Azure	77
7.1	Sơ đồ luồng nghiệp vụ tính năng tìm kiếm và lọc sản phẩm	82
7.2	Sơ đồ luồng nghiệp vụ Thêm sản phẩm và Quản lý giỏ hàng	84
7.3	Sơ đồ luồng nghiệp vụ Đặt hàng	85
7.4	Sơ đồ luồng nghiệp vụ tính năng Thanh toán đa phương thức	87
7.5	Các kịch bản kiểm thử thuộc nhóm thành công trong ProductReadOn-lyRepositoryTest	91
7.6	Kiểm tra điều kiện ngưỡng giá trị đơn hàng tối thiểu trong CountPrice-ServiceTest	92
7.7	Kịch bản kiểm thử giới hạn mức giảm giá tối đa	93
7.8	Các kịch bản kiểm thử xử lý ngoại lệ trong UpdateOrderStatusServiceTest	93
7.9	Thực thi thành công API lấy thông tin đặt hàng trên Swagger UI	96
7.10	Minh họa phản hồi kết quả tính toán giá trị giỏ hàng trên Cart Service .	97

Danh sách bảng

2.1	So sánh tính năng giữa các nền tảng hiện có và hệ thống đề xuất	6
2.2	Đặc tả trường hợp sử dụng: Tìm kiếm sản phẩm	12
2.3	Đặc tả trường hợp sử dụng: [UC7] Thêm sản phẩm vào giỏ hàng và đặt hàng	12
2.4	Đặc tả trường hợp sử dụng: Thanh toán đa phương thức	13
2.5	Đặc tả trường hợp sử dụng: Đánh giá và phản hồi của khách hàng	14
2.6	Đặc tả trường hợp sử dụng: Xem báo cáo cơ bản	15
2.7	Đặc tả trường hợp sử dụng: Gợi ý sản phẩm	16
2.8	Đặc tả trường hợp sử dụng: Hỗ trợ trong và sau mua hàng	17
3.1	So sánh giữa các kiến trúc hệ thống	32
4.1	So sánh các framework phát triển Backend	36
4.2	So sánh các message broker	38
4.3	Tổng hợp các công nghệ sử dụng tại Backend	40
4.4	So sánh các hệ quản trị cơ sở dữ liệu	43
4.5	Tổng hợp các công nghệ sử dụng tại Hạ tầng	45
5.1	Các API quan trọng của User Service	62
5.2	Các API quan trọng của Product Service	63
5.3	Danh mục API của Cart Service	63
5.4	Các API quan trọng của Orchestration Service	64
5.5	Các API quan trọng của Order Service	64
5.6	Danh mục API của Admin Service	65
7.1	Các ca kiểm thử chi tiết tính năng Tìm kiếm và lọc sản phẩm	83
7.2	Các ca kiểm thử quy trình Quản lý giỏ hàng và Đặt hàng	86
7.3	Các ca kiểm thử tính năng Thanh toán đa phương thức	88
7.4	Kết quả kiểm thử Usability và Learnability	89
7.5	Kết quả kiểm thử hiệu năng và độ nhạy AI	89
7.6	Kết quả kiểm thử bảo mật và đồng bộ dữ liệu	89
7.7	Thông kê chi tiết kết quả Unit Testing toàn bộ hệ thống Backend	94

7.8	Tổng quan các cấp độ kiểm thử	98
7.9	Thiết kế test case node Parse	98
7.10	Thiết kế test case node Resolve	99
7.11	Kết quả semantic search	100
7.12	Kết quả kiểm thử API	100
7.13	Tổng kết kết quả kiểm thử	100

1 Giới thiệu đề tài

Trong chương này, nhóm trình bày tổng quan về đề tài, bao gồm bối cảnh và lý do lựa chọn đề tài, mục tiêu cần đạt được cũng như phạm vi thực hiện của hệ thống. Thông qua đó, chương này nhằm xác định rõ ý nghĩa, hướng phát triển và giới hạn của đề tài.

1.1 Lý do chọn đề tài

Sự phát triển mạnh mẽ của thương mại điện tử trong những năm gần đây đã làm thay đổi toàn diện cách thức mua sắm và tiêu dùng của người dùng. Đặc biệt trong lĩnh vực bán lẻ điện thoại và thiết bị công nghệ, các doanh nghiệp lớn trong nước như CellphoneS, FPT Shop, Thế Giới Di Động đã xây dựng những nền tảng thương mại điện tử hiện đại, đáp ứng đa dạng nhu cầu từ việc tra cứu sản phẩm, so sánh giá, đến đặt hàng trực tuyến. Các hệ thống này được đầu tư bài bản cả về giao diện, trải nghiệm người dùng (UI/UX) lẫn hạ tầng kỹ thuật, mang lại sự tiện lợi và tin cậy cho khách hàng.

Tuy nhiên, khi khảo sát và trải nghiệm thực tế, nhóm nhận thấy rằng các nền tảng thương mại điện tử hiện nay vẫn chủ yếu hoạt động theo cơ chế tương tác truyền thống, nơi người dùng cần thao tác thủ công để tìm kiếm sản phẩm hoặc truy cập thông tin. Một số trang web đã tích hợp chatbot hoặc trợ lý ảo cơ bản, tuy nhiên các hệ thống này phần lớn chỉ dừng lại ở mức trả lời câu hỏi đơn giản, chưa khai thác hiệu quả khả năng hỗ trợ người dùng trong quá trình tìm hiểu và ra quyết định mua sắm, chẳng hạn như tư vấn sản phẩm phù hợp với nhu cầu hoặc so sánh các lựa chọn một cách trực quan và nhanh chóng.

Trong bối cảnh trí tuệ nhân tạo (AI) và xử lý ngôn ngữ tự nhiên (Natural Language Processing – NLP) ngày càng phát triển, việc tích hợp Trợ lý Ảo thông minh vào hệ thống thương mại điện tử mở ra hướng tiếp cận mới – nơi người dùng có thể tương tác trực tiếp với hệ thống bằng ngôn ngữ tự nhiên, thay vì qua các bước điều hướng phức tạp. Một trợ lý ảo có khả năng hiểu ngữ cảnh, tư vấn sản phẩm, so sánh các lựa chọn và giải đáp thắc mắc sẽ góp phần nâng cao trải nghiệm người dùng, hỗ trợ quá trình ra quyết định và tăng tính cá nhân hóa trong mua sắm.

Xuất phát từ thực tế đó, nhóm quyết định lựa chọn và thực hiện đề tài “Hệ thống Thương mại Điện tử cho Bán lẻ Điện thoại tích hợp Trợ lý Ảo AI Agent” với mục tiêu xây dựng một mô hình thương mại điện tử thông minh, có khả năng tương tác tự nhiên, hiểu ngữ cảnh và hỗ trợ người dùng thông qua các chức năng tư vấn, so sánh và hỏi đáp. Đề tài không chỉ mang ý nghĩa thực tiễn trong việc ứng dụng công nghệ AI vào thương mại điện tử, mà còn góp phần nghiên cứu và phát triển các mô hình tương tác người – máy thế hệ mới.

1.2 Mục tiêu đề tài

Mục tiêu của đề tài là thiết kế và phát triển một hệ thống thương mại điện tử hoàn chỉnh, có khả năng hoạt động ổn định, dễ sử dụng, và tích hợp Trợ lý Ảo AI Agent để hỗ trợ người dùng trong quá trình mua sắm trực tuyến. Cụ thể, các mục tiêu chính của đề tài

bao gồm:

- Xây dựng một hệ thống thương mại điện tử chuyên biệt cho lĩnh vực bán lẻ điện thoại và thiết bị công nghệ. Hệ thống cần đáp ứng đầy đủ các chức năng cơ bản như hiển thị sản phẩm, tìm kiếm, quản lý giỏ hàng, đặt hàng, hủy đơn hàng, thanh toán và theo dõi trạng thái đơn hàng. Giao diện người dùng được thiết kế trực quan và thân thiện.
- Phát triển Trợ lý Ảo AI Agent có khả năng giao tiếp tự nhiên với người dùng bằng tiếng Việt, tư vấn sản phẩm mà khách hàng quan tâm và thực hiện tra cứu các thông tin mà khách hàng yêu cầu.
- Tích hợp Trợ lý Ảo vào hệ thống web thương mại điện tử nhằm tạo ra trải nghiệm liền mạch cho người dùng.
- Phát triển giao diện và chức năng quản trị cho phép quản lý hệ thống hiệu quả. Quản trị viên có thể thêm, chỉnh sửa, hoặc xóa sản phẩm; theo dõi đơn hàng; quản lý người dùng; và giám sát hoạt động của Trợ lý Ảo. Điều này đảm bảo hệ thống hoạt động ổn định, dễ bảo trì và dễ mở rộng trong tương lai.

1.3 Phạm vi đề tài

Phạm vi thực hiện của đề tài được xác định như sau:

- **Website thương mại điện tử:**

Hệ thống được thiết kế dưới dạng ứng dụng web, hướng đến người dùng truy cập qua trình duyệt trên máy tính. Các chức năng chính của website bao gồm:

- Hiển thị danh sách và chi tiết sản phẩm.
- Quản lý giỏ hàng, đặt hàng, hủy đơn và theo dõi trạng thái đơn hàng.
- Quản lý tài khoản người dùng (đăng ký, đăng nhập, cập nhật thông tin).
- Trang quản trị dành cho admin để quản lý sản phẩm, người dùng và đơn hàng.

- **Trợ lý Ảo AI Agent:**

Trợ lý Ảo được phát triển nhằm hỗ trợ người dùng trong quá trình tương tác với hệ thống, đóng vai trò như một nhân viên bán hàng tự động. Các khả năng trong phạm vi đề tài bao gồm:

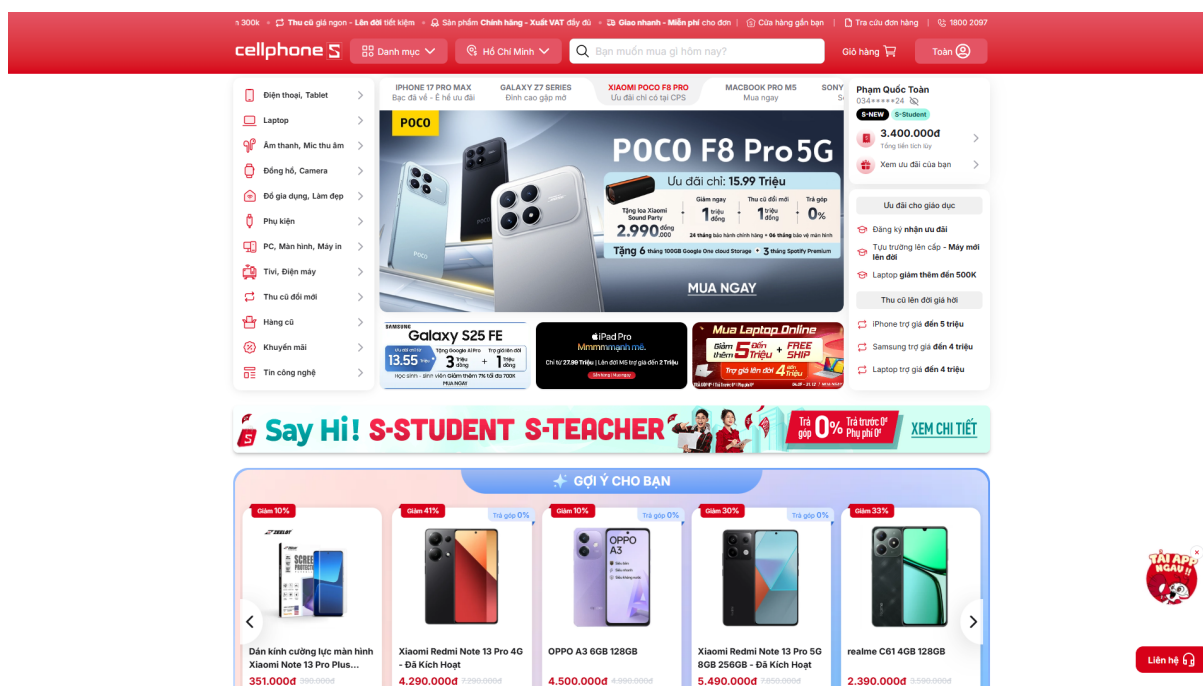
- Hiểu và phản hồi các truy vấn tiếng Việt bằng ngôn ngữ tự nhiên.
- Hỗ trợ tìm kiếm sản phẩm, gợi ý theo nhu cầu người dùng.
- Cung cấp thông tin về chính sách, chương trình khuyến mãi hoặc hướng dẫn sử dụng sản phẩm.

2 Phân tích yêu cầu

Trong chương này, nhóm đã làm rõ các yêu cầu và luồng nghiệp vụ thông qua đặc tả các trường hợp sử dụng cùng các biểu đồ UML. Đây là khung sườn quan trọng giúp xác định phạm vi chức năng và là cơ sở để triển khai thiết kế kỹ thuật ở chương tiếp theo.

2.1 Khảo sát thị trường

2.1.1. CellphoneS



Hình 2.1: Trang chủ của CellphoneS

CellphoneS là hệ thống bán lẻ điện thoại, laptop, máy tính bảng và nhiều phụ kiện công nghệ phổ biến tại Việt Nam, hoạt động song song giữa chuỗi cửa hàng và nền tảng thương mại điện tử. Website của CellphoneS hướng tới trải nghiệm mua sắm nhanh, tiện lợi, hỗ trợ trả góp, chương trình thu cũ đổi mới (trade-in) và hệ thống tích điểm khách hàng thân thiết Smember.

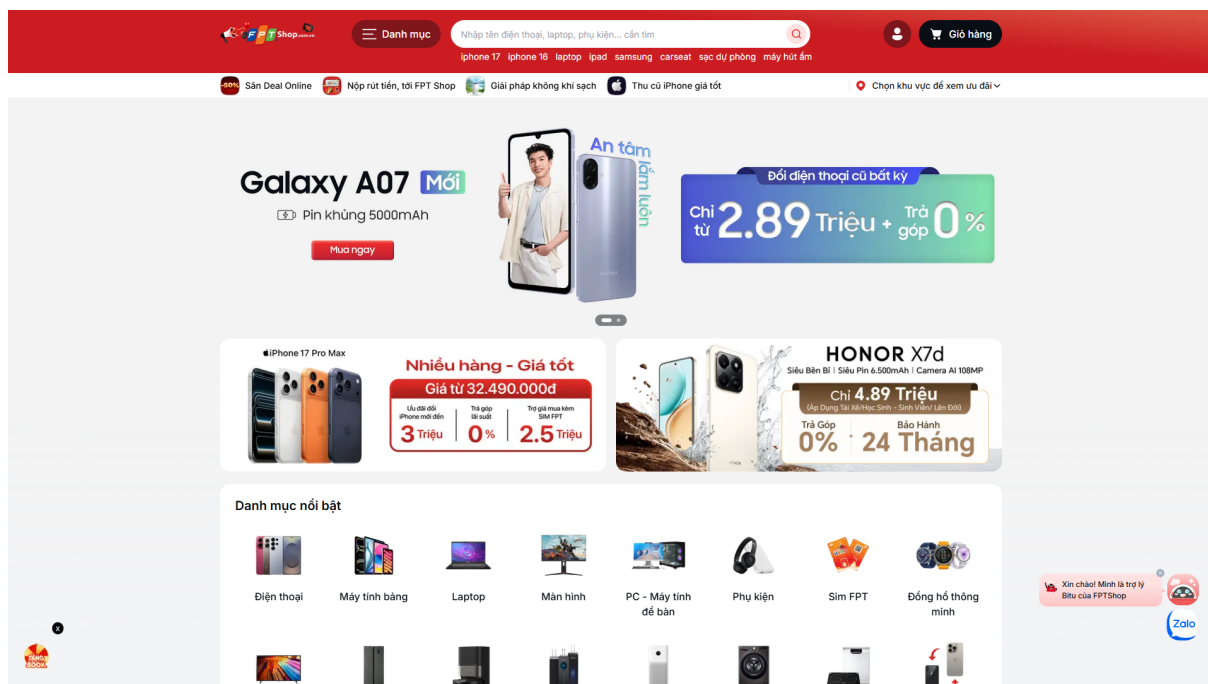
Tính năng nổi bật:

- Bộ lọc và tìm kiếm sản phẩm theo giá, thương hiệu, thông số kỹ thuật và chương trình khuyến mãi.
- Hỗ trợ trade-in với quy trình định giá trực tuyến và phản hồi nhanh.
- Trả góp linh hoạt qua nhiều hình thức như thẻ tín dụng, công ty tài chính hoặc ví điện tử.

- Ứng dụng Smember giúp người dùng quản lý điểm thưởng, theo dõi ưu đãi và thông báo khuyến mãi.

Website của CellphoneS thể hiện thể mạnh rõ rệt ở việc xây dựng quy trình trade-in minh bạch và dễ sử dụng, cùng với hệ thống trả góp linh hoạt, giúp người dùng có nhiều lựa chọn tài chính phù hợp. Nền tảng Smember tạo ra hệ sinh thái khách hàng thân thiết, góp phần giữ chân người dùng lâu dài. Tuy nhiên, website vẫn còn hạn chế ở khả năng cá nhân hóa trải nghiệm mua sắm. Các tính năng như đề xuất sản phẩm thông minh hay chatbot hỗ trợ tương tác tự nhiên chưa được đầu tư sâu, khiến hệ thống vẫn chủ yếu hoạt động theo cơ chế tìm kiếm và thao tác thủ công.

2.1.2. FPT Shop



Hình 2.2: Trang chủ của FPT Shop

FPT Shop là thương hiệu bán lẻ thiết bị công nghệ thuộc Tập đoàn FPT, với mạng lưới cửa hàng rộng khắp và nền tảng thương mại điện tử tích hợp mạnh mẽ. Website của FPT Shop chú trọng trải nghiệm người dùng, hỗ trợ trả góp, khuyến mãi và dịch vụ hậu mãi, đồng thời ứng dụng công nghệ AI trong tư vấn, hỗ trợ khách hàng.

Tính năng nổi bật:

- Hệ thống tìm kiếm và lọc sản phẩm mạnh, hiển thị nhiều biến thể sản phẩm và cho phép so sánh cơ bản.
- Hỗ trợ trả góp đa dạng với thông tin minh bạch về lãi suất và các tùy chọn thanh toán.
- Ứng dụng di động cung cấp thông tin khuyến mãi, voucher và tích điểm khách hàng thân thiết.

FPT Shop sở hữu nền tảng công nghệ hiện đại với khả năng đồng bộ tốt giữa kênh bán hàng trực tuyến và hệ thống cửa hàng vật lý, mang lại trải nghiệm mua sắm liền mạch cho người dùng. Website cung cấp đầy đủ các chức năng như tra cứu sản phẩm, xem thông tin chi tiết, so sánh và đặt hàng trực tuyến, đáp ứng hiệu quả nhu cầu mua sắm của khách hàng.

Tuy nhiên, hệ thống hiện chưa tích hợp trợ lý ảo hoặc chatbot thông minh trên nền tảng web để hỗ trợ người dùng trong quá trình tương tác. Việc hỗ trợ khách hàng chủ yếu vẫn dựa vào các kênh truyền thống như tổng đài hoặc tư vấn viên trực tuyến. Cách tiếp cận này giúp đảm bảo độ chính xác trong phản hồi, nhưng đồng thời làm hạn chế khả năng tự động hóa và phản hồi nhanh, đặc biệt khi số lượng người dùng tăng cao.

Bên cạnh đó, việc thiếu một hệ thống trợ lý ảo cũng khiến trải nghiệm người dùng vẫn mang tính thụ động, phụ thuộc vào thao tác tìm kiếm và điều hướng thủ công. Điều này cho thấy tiềm năng ứng dụng trí tuệ nhân tạo trong việc hỗ trợ tư vấn, so sánh sản phẩm và giải đáp thắc mắc vẫn chưa được khai thác hiệu quả trong hệ thống.

2.1.3. Thế Giới Di Động



Hình 2.3: Trang chủ của Thế Giới Di Động

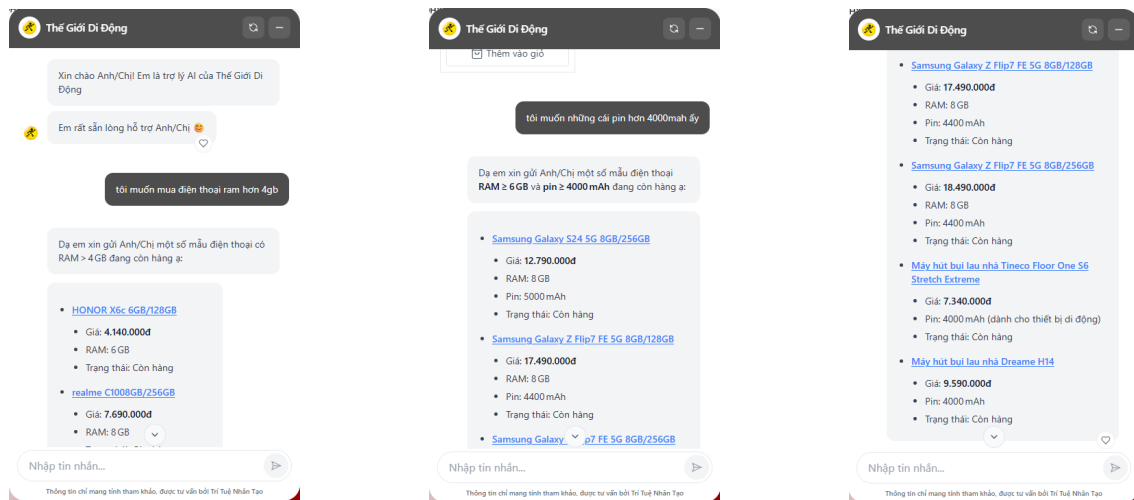
Thế Giới Di Động là hệ thống bán lẻ điện thoại và thiết bị điện tử tiêu dùng lớn nhất Việt Nam, với hàng nghìn cửa hàng trên toàn quốc. Website được tối ưu cho người dùng phổ thông, tập trung vào quy trình mua nhanh, khuyến mãi hấp dẫn và chính sách hậu mãi rõ ràng.

Tính năng nổi bật:

- Bộ lọc sản phẩm đa tiêu chí, gợi ý kết quả tức thì và hiển thị sản phẩm nổi bật.

- Chính sách giao hàng, đổi trả, bảo hành minh bạch.
- Hệ thống khuyến mãi, flash sale và combo sản phẩm trình bày nổi bật.
- Hỗ trợ khách hàng đa kênh: hotline, chat trực tuyến, mạng xã hội và cửa hàng.

Website của Thế Giới Di Động có thiết kế thân thiện, tốc độ truy cập cao và khuyến mãi cập nhật liên tục. Bên cạnh đó, Thế Giới Di Động đã tích hợp trợ lý ảo hỗ trợ khách hàng. Tuy nhiên, các trợ lý ảo này chỉ dừng lại ở mức trả lời câu hỏi hoặc hướng dẫn cơ bản. Các trợ lý ảo này chưa làm tốt trong việc duy trì một luồng hỗ trợ khách hàng nhiều lượt và phi tuyến tính.



Hình 2.4: Trợ lý ảo của Thế Giới Di Động

2.1.4. So sánh tính năng giữa các nền tảng hiện có và hệ thống đề xuất

Bảng 2.1: So sánh tính năng giữa các nền tảng hiện có và hệ thống đề xuất

Nhóm tính năng	CellphoneS	FPT Shop	Thế Giới Di Động	Hệ thống đề xuất
Tìm kiếm và lọc sản phẩm	Có	Có	Có	Có
Phân loại và hiển thị sản phẩm	Có	Có	Có	Có
Đánh giá và phản hồi sản phẩm	Có	Có	Có	Có
Quản lý giỏ hàng và đơn hàng	Có	Có	Có	Có
Hệ thống khuyến mãi và ưu đãi	Có	Có	Có	Có
Giao diện thân thiện, tối ưu UX/UI	Có	Có	Có	Có
Tư vấn sản phẩm	Không	Không	Hạn chế	Thông minh

Từ kết quả khảo sát ba nền tảng thương mại điện tử lớn tại Việt Nam, có thể nhận thấy rằng mỗi hệ thống đều có định hướng riêng trong việc phát triển và phục vụ người dùng.

Tuy nhiên, nhóm nhận thấy rằng mặc dù các doanh nghiệp trong ngành đã có bước tiến lớn trong việc số hóa hoạt động bán lẻ, các nền tảng thương mại điện tử trong mảng bán lẻ điện thoại di động và các đồ dùng công nghệ tại Việt Nam chưa thật sự tích hợp Trợ lý

Ảo thông minh có khả năng giao tiếp tự nhiên và duy trì luồng hỗ trợ nhiều lượt và phi tuyến tính.

Khoảng trống này mở ra cơ hội để nhóm nghiên cứu và phát triển một mô hình thương mại điện tử tích hợp Trợ lý Ảo AI Agent, hướng đến việc nâng cao trải nghiệm người dùng, tự động hóa quy trình giao dịch, và tăng tính cá nhân hóa trong tương tác mua sắm.

2.2 Phân tích các bên liên quan

Việc phát triển và triển khai hệ thống mang lại giá trị gia tăng rõ rệt cho các bên liên quan thông qua việc giải quyết trực tiếp các nhu cầu thực tế trong vận hành và kinh doanh. Những nhu cầu này chính là cơ sở để xác lập danh mục các yêu cầu chức năng và phi chức năng chi tiết ở các mục tiếp theo.

2.2.1. Khách hàng

Đối với người dùng cuối, hệ thống cung cấp một nền tảng mua sắm hiện đại, tập trung vào tính tiện dụng và cá nhân hóa. Nhu cầu cốt lõi bắt đầu từ việc thiết lập tài khoản an toàn [FR1, FR2, FR3] và quản lý thông tin cá nhân thuận tiện [FR4]. Trong quá trình mua sắm, khách hàng cần các công cụ tìm kiếm, lọc sản phẩm minh bạch [FR5, FR8] và quy trình giỏ hàng, thanh toán linh hoạt [FR6, FR7].

Điểm nhấn quan trọng là sự hỗ trợ thông minh từ trợ lý AI từ khâu tìm kiếm, gợi ý sản phẩm đến tư vấn các chính sách dịch vụ [FR10, FR11]. Để thỏa mãn nhóm người dùng này, hệ thống phải đáp ứng các tiêu chuẩn về tính ổn định của giao diện [NFR1], tính dễ học trong thao tác [NFR3] cùng khả năng phản hồi tự nhiên, đúng ngữ cảnh của AI Agent [NFR2].

2.2.2. Đội ngũ vận hành và quản trị viên

Đối với đội ngũ nội bộ, hệ thống đóng vai trò là công cụ tối ưu hóa hiệu suất làm việc hàng ngày. Nhu cầu chính là khả năng kiểm soát quyền hạn truy cập [FR14] và quản lý danh mục sản phẩm một cách tập trung [FR15]. Nhân viên vận hành cần được hỗ trợ cập nhật trạng thái đơn hàng chính xác [FR17] và theo dõi các đánh giá từ khách hàng sau khi hoàn tất mua sắm [FR13].

Về phía kỹ thuật, nhóm này yêu cầu hệ thống phải duy trì tính toàn vẹn dữ liệu cao giữa các dịch vụ [NFR6] và có khả năng bảo trì, nâng cấp linh hoạt nhờ vào kiến trúc phân tán [NFR5]. Việc theo dõi và phát hiện các dịch vụ đang hoạt động giúp đội ngũ vận hành duy trì độ tin cậy của toàn bộ hệ thống [NFR7].

2.2.3. Chủ sở hữu doanh nghiệp

Về mặt chiến lược kinh doanh, hệ thống được kỳ vọng sẽ nâng cao hiệu quả tiếp cận thị trường và tối ưu hóa lợi nhuận thông qua các báo cáo thống kê trực quan về doanh thu và sản phẩm [FR16]. Việc cho phép khách hàng chủ động theo dõi lịch sử đơn hàng [FR12] và thực hiện quyền hủy đơn theo ràng buộc [FR9] giúp xây dựng niềm tin thương hiệu

và sự chuyên nghiệp cho doanh nghiệp.

Ngoài ra, hệ thống phải đáp ứng đòi hỏi khắt khe về an toàn tài sản dữ liệu thông qua các cơ chế bảo mật nghiêm ngặt [NFR4]. Kiến trúc hệ thống cần đảm bảo tính mở và khả năng mở rộng linh hoạt [NFR5] để sẵn sàng tích hợp thêm các dịch vụ giá trị gia tăng trong tương lai mà không cần tái cấu trúc nền tảng, giúp doanh nghiệp tối ưu hóa chi phí đầu tư dài hạn.

2.3 Yêu cầu chức năng

2.3.1. Đối với khách hàng

- **[FR1] Đăng ký** (Ưu tiên: Cao)
Cho phép khách hàng tự đăng ký tài khoản mới và quản lý thông tin cá nhân. Tài khoản quản trị viên được hệ thống cấp riêng và không thể tự đăng ký.
- **[FR2] Đăng nhập/Đăng xuất** (Ưu tiên: Cao)
Cho phép khách hàng đăng nhập bằng tài khoản đã đăng ký hoặc tài khoản Google và đăng xuất khỏi hệ thống.
- **[FR3] Quên mật khẩu** (Ưu tiên: Trung bình)
Cung cấp cho khách hàng phương thức lấy lại mật khẩu trong trường hợp khách hàng quên mật khẩu.
- **[FR4] Quản lý hồ sơ** (Ưu tiên: Thấp)
Cho phép khách hàng thêm và sửa đổi hồ sơ thông tin cá nhân của bản thân.
- **[FR5] Tìm kiếm và lọc** (Ưu tiên: Cao)
Cung cấp chức năng tìm kiếm sản phẩm theo tên và lọc sản phẩm theo tiêu chí như giá, cấu hình và khuyến mãi.
- **[FR6] Giỏ hàng và đặt hàng** (Ưu tiên: Cao)
Cho phép khách hàng thêm sản phẩm vào giỏ hàng, chỉnh sửa và tiến hành đặt hàng.
- **[FR7] Đa phương thức thanh toán** (Ưu tiên: Trung bình)
Hỗ trợ đa phương thức thanh toán, bao gồm thẻ ngân hàng và ví điện tử.
- **[FR8] Thông tin sản phẩm minh bạch** (Ưu tiên: Cao)
Hiển thị thông tin chi tiết sản phẩm, giá cả, tình trạng tồn kho và phương thức thanh toán minh bạch. Hỗ trợ hiển thị sản phẩm theo phân trang để đảm bảo tốc độ tải và trải nghiệm người dùng.
- **[FR9] Hủy đơn hàng** (Ưu tiên: Trung bình)
Cho phép khách hàng hủy đơn hàng nếu đơn hàng đáp ứng được các điều kiện ràng buộc.
- **[FR10] Trợ lý ảo (Tìm kiếm/gợi ý)** (Ưu tiên: Cao)
Trợ lý ảo hỗ trợ khách hàng tìm kiếm sản phẩm theo ngôn ngữ tự nhiên và gợi ý sản phẩm phù hợp dựa trên nhu cầu, sở thích và ngân sách.

- **[FR11] Trợ lý ảo (Tư vấn FAQ)** *(Ưu tiên: Cao)*
Trợ lý ảo cung cấp tư vấn cơ bản và trả lời câu hỏi thường gặp, bao gồm chính sách bảo hành, đổi trả, khuyến mãi và phương thức thanh toán.
- **[FR12] Lịch sử và theo dõi đơn hàng** *(Ưu tiên: Cao)*
Cho phép khách hàng xem lịch sử đơn hàng và theo dõi trạng thái đơn hàng đang xử lý hoặc vận chuyển.
- **[FR13] Đánh giá** *(Ưu tiên: Trung bình)*
Cho phép khách hàng gửi đánh giá cho sản phẩm sau khi hoàn thành đơn hàng.

2.3.2. Đối với quản trị viên

- **[FR14] Phân quyền người dùng** *(Ưu tiên: Cao)*
Phân quyền giữa khách hàng và quản trị viên, đảm bảo mỗi nhóm có quyền truy cập và thao tác phù hợp với vai trò của mình trong hệ thống.
- **[FR15] Quản lý sản phẩm** *(Ưu tiên: Cao)*
Giao diện quản trị cho phép thêm, chỉnh sửa, xóa sản phẩm, cập nhật giá và quản lý các chương trình khuyến mãi.
- **[FR16] Báo cáo cơ bản** *(Ưu tiên: Trung bình)*
Hệ thống cung cấp báo cáo cơ bản cho quản trị viên, bao gồm tổng quan về số lượng đơn hàng, doanh thu mô phỏng và danh sách các sản phẩm bán chạy nhất.
- **[FR17] Cập nhật trạng thái đơn hàng** *(Ưu tiên: Cao)*
Tự động cập nhật thông tin trạng thái đơn hàng dựa trên dữ liệu tích hợp từ đơn vị vận chuyển (hoặc dữ liệu nhập tay của admin), đảm bảo thông tin chính xác đến khách hàng.

2.4 Yêu cầu phi chức năng

Các yêu cầu phi chức năng được thiết lập nhằm đảm bảo hệ thống vận hành đúng thiết kế, an toàn và có khả năng kiểm chứng trực tiếp trên kiến trúc Microservices.

2.4.1. Tính ổn định và trải nghiệm người dùng

- **[NFR1] Tính ổn định giao diện** *(Ưu tiên: Cao)*
Giao diện người dùng được thiết kế tối giản, tập trung vào việc hoàn thành quy trình đặt hàng thông qua các bước logic được điều hướng rõ ràng, tránh gây nhầm lẫn trong thao tác.
- **[NFR2] Hiệu năng phản hồi AI Agent** *(Ưu tiên: Cao)*
Trợ lý ảo được thiết kế để xử lý luồng suy luận tập trung, đảm bảo đưa ra phản hồi đúng ngữ cảnh nghiệp vụ bán lẻ điện thoại và hạn chế các thông báo lỗi kỹ thuật trực tiếp tới người dùng.
- **[NFR3] Tính dễ học** *(Ưu tiên: Trung bình)*
Người dùng có thể tự làm quen và sử dụng các chức năng tìm kiếm, xem chi tiết sản phẩm và thao tác giỏ hàng một cách tự nhiên mà không cần hướng dẫn phức tạp.

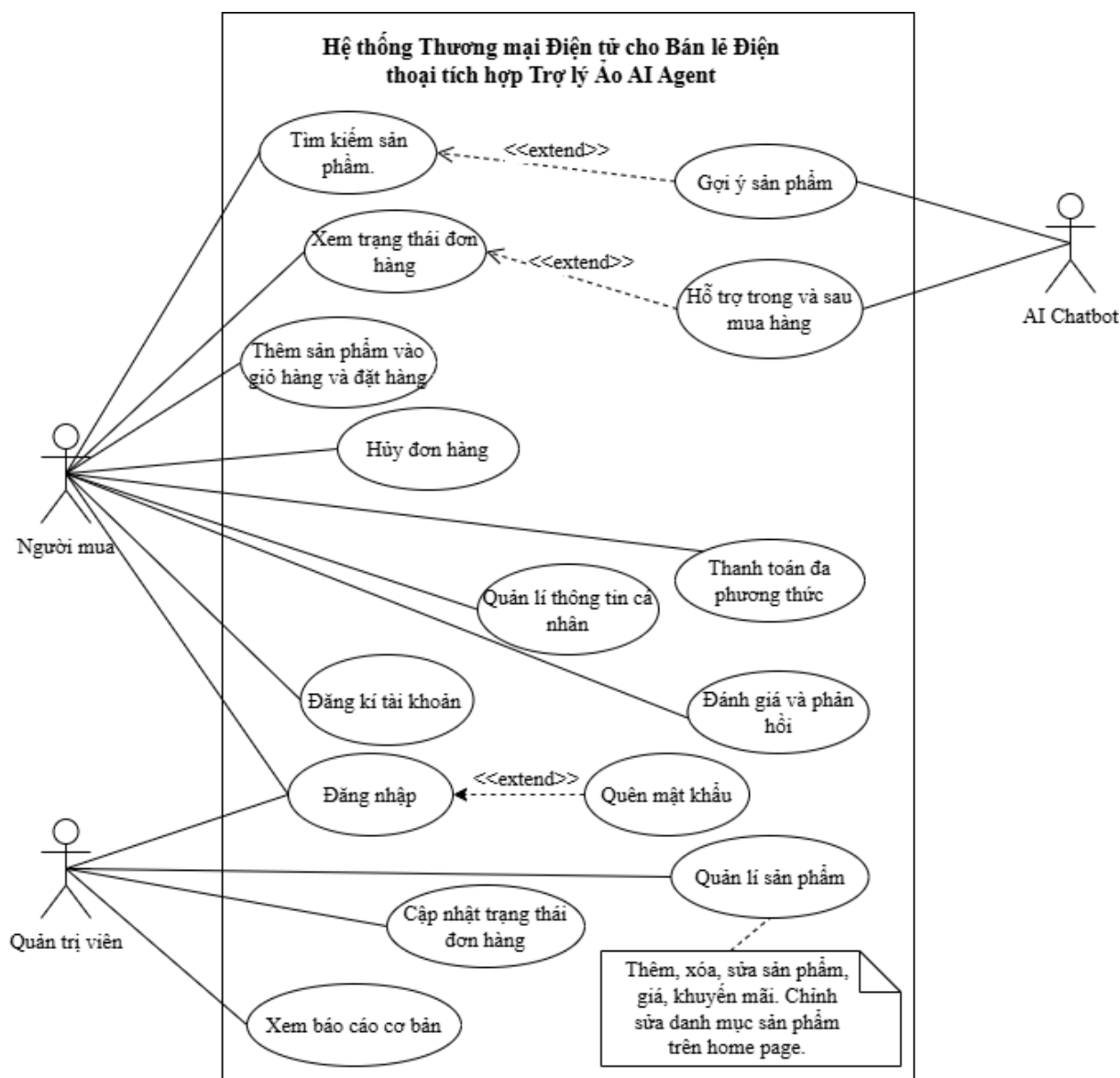
2.4.2. Bảo mật, tin cậy và khả năng bảo trì

- **[NFR4] Bảo mật thông tin** *(Ưu tiên: Cao)*
Triển khai cơ chế bảo mật cho mật khẩu người dùng bằng các thuật toán mã hóa một chiều trước khi lưu trữ. Đảm bảo an toàn quyền truy cập thông qua các cơ chế xác thực được tích hợp trong hệ thống.
- **[NFR5] Khả năng quản trị và mở rộng** *(Ưu tiên: Cao)*
Kiến trúc Microservices cho phép tách biệt hoàn toàn giữa dịch vụ nghiệp vụ (Java) và dịch vụ AI (Python), giúp việc bảo trì hoặc nâng cấp từng thành phần không gây ảnh hưởng đến toàn bộ hệ thống.
- **[NFR6] Tính toàn vẹn dữ liệu** *(Ưu tiên: Cao)*
Sử dụng cơ chế truyền tải sự kiện thông qua Message Broker để đảm bảo trạng thái dữ liệu (như đơn hàng, sản phẩm) được đồng bộ hóa nhất quán giữa các dịch vụ phân tán.

2.5 Sơ đồ và đặc tả các trường hợp sử dụng

2.5.1. Sơ đồ các trường hợp sử dụng

Sơ đồ trường hợp sử dụng (use case diagram) cung cấp cái nhìn tổng quan, cấp cao về ranh giới hệ thống và tất cả các chức năng chính mà hệ thống sẽ thực hiện để đáp ứng nhu cầu của các tác nhân. Hình 2.5 dưới đây minh họa chi tiết mối quan hệ tương tác này trong ngữ cảnh hệ thống của nhóm.



Hình 2.5: Use case diagram tổng thể của hệ thống

2.5.2. Bảng mô tả chi tiết các trường hợp sử dụng

Dựa trên danh mục các yêu cầu chức năng đã xác lập, nhóm tiến hành đặc tả chi tiết các kịch bản tương tác thông qua các bảng mô tả trường hợp sử dụng. Việc đặc tả này nhằm làm rõ trình tự thực hiện, các điều kiện ràng buộc cũng như kết quả mong đợi của từng nghiệp vụ, giúp đảm bảo tính nhất quán giữa khâu phân tích và triển khai thực tế.

Nhằm tối ưu hóa dung lượng của báo cáo và tập trung vào các đặc thù công nghệ của hệ thống, trong phạm vi chương này, nhóm trình bày các mô tả cho những trường hợp sử dụng cốt lõi, bao gồm các nghiệp vụ tìm kiếm sản phẩm, thêm sản phẩm vào giỏ hàng, đặt hàng, thanh toán và các luồng tương tác thông minh với trợ lý ảo AI thông qua các bảng 2.2, 2.3, 2.4, 2.5 và bảng 2.6. Các bảng mô tả chi tiết cho các chức năng quản trị và những nghiệp vụ hỗ trợ khác được trình bày đầy đủ tại phần phụ lục ?? của đề án.

Bảng 2.2: Đặc tả trường hợp sử dụng: Tìm kiếm sản phẩm

Tên trường hợp sử dụng	[UC5] Tìm kiếm sản phẩm
Tóm tắt mô tả	Cho phép khách hàng tìm kiếm sản phẩm theo từ khóa hoặc các tiêu chí bộ lọc.
Tác nhân	Khách hàng.
Tiền điều kiện	1. Hệ thống hoạt động bình thường. 2. Cơ sở dữ liệu được kết nối với hệ thống. 3. Kết nối với internet được đảm bảo.
Sự kiện kích hoạt	Khách hàng nhập từ khóa vào thanh tìm kiếm hoặc lựa chọn các tiêu chí lọc sản phẩm.
Các bước thực hiện	1. Khách hàng nhập từ khóa hoặc chọn bộ lọc sản phẩm (giá, cấu hình, hãng...). 2. Hệ thống thực hiện truy vấn cơ sở dữ liệu để tìm kiếm sản phẩm phù hợp. 3. Hệ thống hiển thị danh sách các sản phẩm thỏa mãn điều kiện.
Hậu điều kiện	Danh sách sản phẩm hiển thị phù hợp với tiêu chí tìm kiếm của khách hàng.
Luồng thay thế	Không.
Luồng ngoại lệ	Ngoại lệ 1: Tại bước 2 2a. Nếu xảy ra lỗi kết nối cơ sở dữ liệu, hệ thống thông báo lỗi và yêu cầu người dùng thử lại. Ngoại lệ 2: Tại bước 3 3a. Nếu không tìm thấy sản phẩm nào phù hợp, hệ thống hiển thị thông báo "Không tìm thấy sản phẩm" kèm theo các gợi ý khác.

Bảng 2.3: Đặc tả trường hợp sử dụng: [UC7] Thêm sản phẩm vào giỏ hàng và đặt hàng

Tên trường hợp sử dụng	Thêm sản phẩm vào giỏ hàng và đặt hàng
Tóm tắt mô tả	Cho phép khách hàng lựa chọn sản phẩm, thêm vào giỏ hàng và thực hiện quy trình đặt hàng với nhiều phương thức thanh toán khác nhau.
Tác nhân	Khách hàng
Tiền điều kiện	1. Hệ thống hoạt động bình thường. 2. Cơ sở dữ liệu được kết nối với hệ thống. 3. Kết nối với internet được đảm bảo. 4. Sản phẩm còn hàng trong kho.
Sự kiện kích hoạt	Khách hàng chọn sản phẩm và nhấn nút “Thêm vào giỏ hàng” hoặc “Đặt hàng”.

Các bước thực hiện	1. Khách hàng chọn sản phẩm cần mua. 2. Hệ thống hiển thị chi tiết sản phẩm và nút “Thêm vào giỏ hàng”. 3. Khách hàng thêm sản phẩm vào giỏ. 4. Hệ thống cập nhật giỏ hàng và hiển thị tổng số lượng, giá tiền. 5. Khách hàng chọn “Đặt hàng”. 6. Hệ thống yêu cầu khách hàng xác nhận địa chỉ giao hàng, phương thức thanh toán. 7. Hệ thống kiểm tra tính hợp lệ của thông tin và số lượng tồn kho. 8. Hệ thống tạo đơn hàng và gửi xác nhận cho khách hàng.
Hậu điều kiện	Đơn hàng được tạo thành công và lưu trong hệ thống, khách hàng nhận thông báo xác nhận.
Luồng thay thế	Luồng thay thế 1: Tại bước 4 4a. Khách hàng không đặt hàng ngay mà chỉ thêm sản phẩm vào giỏ hàng và tiếp tục mua sắm.
Luồng ngoại lệ	Ngoại lệ 1: Tại bước 6 6a. Nếu phương thức thanh toán hiện tại không hỗ trợ hoặc xảy ra lỗi cổng thanh toán, hệ thống báo lỗi và yêu cầu khách hàng chọn phương thức thanh toán khác.

Bảng 2.4: Đặc tả trường hợp sử dụng: Thanh toán đa phương thức

Tên trường hợp sử dụng	[UC8] Thanh toán đa phương thức
Tóm tắt mô tả	Cho phép khách hàng thanh toán đơn hàng bằng nhiều phương thức khác nhau như: tiền mặt khi nhận hàng (COD), thẻ ngân hàng, ví điện tử, hoặc các cổng thanh toán trực tuyến.
Tác nhân	Khách hàng.
Tiền điều kiện	1. Hệ thống hoạt động bình thường. 2. Cơ sở dữ liệu được kết nối với hệ thống. 3. Kết nối với Internet được đảm bảo. 4. Đơn hàng đã được tạo thành công. 5. Hệ thống kết nối thành công với các cổng thanh toán bên thứ ba (nếu có).
Sự kiện kích hoạt	Khách hàng nhấn chọn “Thanh toán” sau khi xác nhận đơn hàng đối với các phương thức trả trước.

Các bước thực hiện	1. Tùy vào phương thức thanh toán mà khách hàng chọn, hệ thống sẽ hiển thị biểu mẫu hoặc trang thanh toán tương ứng. 2. Khách hàng thực hiện các thao tác thanh toán theo hướng dẫn. 3. Hệ thống hiển thị trạng thái giao dịch và cập nhật trạng thái đơn hàng tương ứng.
Hậu điều kiện	Thanh toán được xử lý thành công và thông tin giao dịch được lưu trữ chính xác trong hệ thống.
Luồng thay thế	Luồng thay thế 1: Tại bước 2 2a. Nếu phương thức thanh toán là “Thanh toán khi nhận hàng” (COD), khách hàng bỏ qua bước thanh toán trực tuyến, hệ thống tự động cập nhật trạng thái đơn hàng sang “Chờ giao hàng”.
Luồng ngoại lệ	Ngoại lệ 1: Tại bước 2 2a. Giao dịch thất bại (sai mã OTP, số dư không đủ...), hệ thống thông báo lỗi và yêu cầu người dùng thử lại hoặc chọn phương thức khác. 2b. Lỗi kết nối với cổng thanh toán bên thứ ba, hệ thống hiển thị thông báo sự cố và giữ đơn hàng ở trạng thái “Chờ thanh toán”.

Bảng 2.5: Đặc tả trường hợp sử dụng: Đánh giá và phản hồi của khách hàng

Tên trường hợp sử dụng	Đánh giá và phản hồi của khách hàng
Tóm tắt mô tả	Khách hàng có thể gửi đánh giá (số sao, nhận xét) về sản phẩm hoặc dịch vụ sau khi nhận hàng để giúp doanh nghiệp cải thiện chất lượng phục vụ.
Tác nhân	Khách hàng.
Tiền điều kiện	1. Hệ thống hoạt động bình thường. 2. Cơ sở dữ liệu được kết nối với hệ thống. 3. Kết nối với Internet được đảm bảo. 4. Khách hàng đã đăng nhập thành công vào hệ thống.
Sự kiện kích hoạt	Khách hàng nhấn chọn chức năng “Đánh giá” hoặc “Phản hồi” tại danh mục các đơn hàng đã giao thành công.
Các bước thực hiện	1. Hệ thống hiển thị biểu mẫu đánh giá sản phẩm tương ứng với đơn hàng. 2. Khách hàng thực hiện chọn số sao đánh giá và nhập nội dung nhận xét hoặc góp ý. 3. Hệ thống kiểm tra, thực hiện lưu phản hồi vào cơ sở dữ liệu và gửi thông báo xác nhận thành công.
Hậu điều kiện	Nội dung đánh giá và phản hồi được lưu trữ trong hệ thống và sẵn sàng để hiển thị hoặc phục vụ mục đích báo cáo nội bộ.

Luồng thay thế	Luồng thay thế 1: Tại bước 1 1a. Nếu khách hàng bỏ qua bước đánh giá và thoát màn hình, đơn hàng vẫn được giữ nguyên trạng thái hoàn tất nhưng không có dữ liệu phản hồi kèm theo.
Luồng ngoại lệ	Không.

Bảng 2.6: Đặc tả trường hợp sử dụng: Xem báo cáo cơ bản

Tên trường hợp sử dụng	Xem báo cáo cơ bản
Tóm tắt mô tả	Cho phép quản trị viên xem báo cáo tổng hợp về doanh thu, số lượng đơn hàng, các sản phẩm bán chạy và phản hồi từ phía khách hàng.
Tác nhân	Quản trị viên.
Tiền điều kiện	1. Hệ thống hoạt động bình thường. 2. Cơ sở dữ liệu được kết nối với hệ thống. 3. Kết nối với Internet được đảm bảo. 4. Quản trị viên đã đăng nhập thành công vào hệ thống quản trị.
Sự kiện kích hoạt	Quản trị viên nhấn chọn chức năng “Xem báo cáo” hoặc “Thống kê” từ menu điều hướng quản trị.
Các bước thực hiện	1. Quản trị viên truy cập vào mục quản lý báo cáo và thống kê. 2. Hệ thống hiển thị danh sách các loại báo cáo hiện có (doanh thu theo ngày/tháng, sản phẩm bán chạy, tổng hợp phản hồi khách hàng). 3. Quản trị viên chọn loại báo cáo và các tiêu chí lọc (thời gian, danh mục) cần xem. 4. Hệ thống truy xuất dữ liệu và hiển thị báo cáo dưới dạng bảng số liệu hoặc biểu đồ trực quan. 5. Quản trị viên có thể chọn chức năng xuất báo cáo dưới định dạng PDF hoặc Excel để lưu trữ.
Hậu điều kiện	Quản trị viên nắm bắt được dữ liệu thống kê phục vụ cho việc phân tích kinh doanh và quản lý.
Luồng thay thế	Luồng thay thế 1: Tại bước 3 3a. Quản trị viên thay đổi bộ lọc hoặc tiêu chí thời gian, hệ thống tự động cập nhật lại số liệu báo cáo tương ứng.
Luồng ngoại lệ	Ngoại lệ 1: Tại bước 4 4a. Không có dữ liệu trong khoảng thời gian được chọn, hệ thống hiển thị thông báo “Không có dữ liệu phù hợp”. 4b. Nếu xảy ra lỗi truy xuất dữ liệu từ máy chủ, hệ thống hiển thị thông báo lỗi kỹ thuật.

Bảng 2.7: Đặc tả trường hợp sử dụng: Gợi ý sản phẩm

Tên trường hợp sử dụng	Gợi ý sản phẩm
Tóm tắt mô tả	Hệ thống AI ChatBot phân tích hành vi, lịch sử mua sắm và từ khóa tìm kiếm để đưa ra gợi ý sản phẩm cá nhân hóa. Đây là trường hợp sử dụng mở rộng từ “Tìm kiếm sản phẩm”.
Tác nhân	AI ChatBot.
Tiền điều kiện	1. Hệ thống hoạt động bình thường. 2. Cơ sở dữ liệu được kết nối với hệ thống. 3. Kết nối với internet được đảm bảo. 4. Trường hợp sử dụng “Tìm kiếm sản phẩm” đã được khởi động. 5. Dữ liệu hành vi và lịch sử khách hàng được lưu trữ. 6. AI ChatBot hoạt động và có quyền truy cập dữ liệu sản phẩm.
Sự kiện kích hoạt	Khách hàng tương tác với ChatBot.
Các bước thực hiện	1. AI ChatBot phân tích từ khóa, lịch sử mua sắm, và hành vi duyệt web của người dùng. 2. Hệ thống hiển thị câu trả lời chứa gợi ý sản phẩm phù hợp ngay trong hộp thoại chat.
Hậu điều kiện	Danh sách gợi ý sản phẩm cá nhân hóa được hiển thị thành công cho khách hàng.
Luồng thay thế	Không.
Luồng ngoại lệ	Ngoại lệ 1: Tại bước 1 1a. Nếu xảy ra lỗi kết nối cơ sở dữ liệu hoặc dịch vụ AI, hệ thống thông báo lỗi và yêu cầu người dùng thử lại. Ngoại lệ 2: Tại bước 2 2a. Nếu không tìm thấy sản phẩm nào phù hợp với sở thích hoặc lịch sử của người dùng, hệ thống hiển thị thông báo mặc định hoặc gợi ý các sản phẩm phổ biến nhất.

Bảng 2.8: Đặc tả trường hợp sử dụng: Hỗ trợ trong và sau mua hàng

Tên trường hợp sử dụng	Hỗ trợ trong và sau mua hàng
Tóm tắt mô tả	AI ChatBot cung cấp thông tin, tư vấn và giải đáp thắc mắc liên quan đến đơn hàng trong quá trình xử lý hoặc sau khi đã hoàn tất. Đây là trường hợp sử dụng mở rộng từ “Xem trạng thái đơn hàng”.
Tác nhân	AI ChatBot.
Tiền điều kiện	1. Hệ thống hoạt động bình thường. 2. Cơ sở dữ liệu được kết nối với hệ thống. 3. Kết nối với Internet được đảm bảo. 4. Trường hợp sử dụng “Xem trạng thái đơn hàng” đã được khởi động. 5. AI ChatBot ở trạng thái sẵn sàng phục vụ. 6. Dữ liệu đơn hàng và lịch sử tương tác được lưu trữ đầy đủ trong hệ thống.
Sự kiện kích hoạt	Khách hàng yêu cầu Chatbot hỗ trợ khi đang xem thông tin chi tiết của một đơn hàng.
Các bước thực hiện	1. Khách hàng nhấn chọn nút “Trợ giúp” hoặc gửi tin nhắn trực tiếp trong giao diện ChatBot. 2. AI ChatBot phân tích nội dung yêu cầu và đưa ra phản hồi tư vấn tự động dựa trên dữ liệu đơn hàng.
Hậu điều kiện	Khách hàng nhận được thông tin hỗ trợ hoặc giải pháp phù hợp cho các vấn đề liên quan đến đơn hàng.
Luồng thay thế	Luồng thay thế 1: Tại bước 2 2a. Nếu khách hàng cần tham khảo các câu hỏi thường gặp (FAQ), AI ChatBot cung cấp liên kết trực tiếp đến trang hướng dẫn hoặc chính sách tương ứng.
Luồng ngoại lệ	Ngoại lệ 1: Tại bước 2 2a. Nếu ChatBot không sẵn dụng hoặc xảy ra lỗi kết nối dịch vụ AI, hệ thống thông báo lỗi và cung cấp phương thức gửi yêu cầu hỗ trợ qua email cho khách hàng.

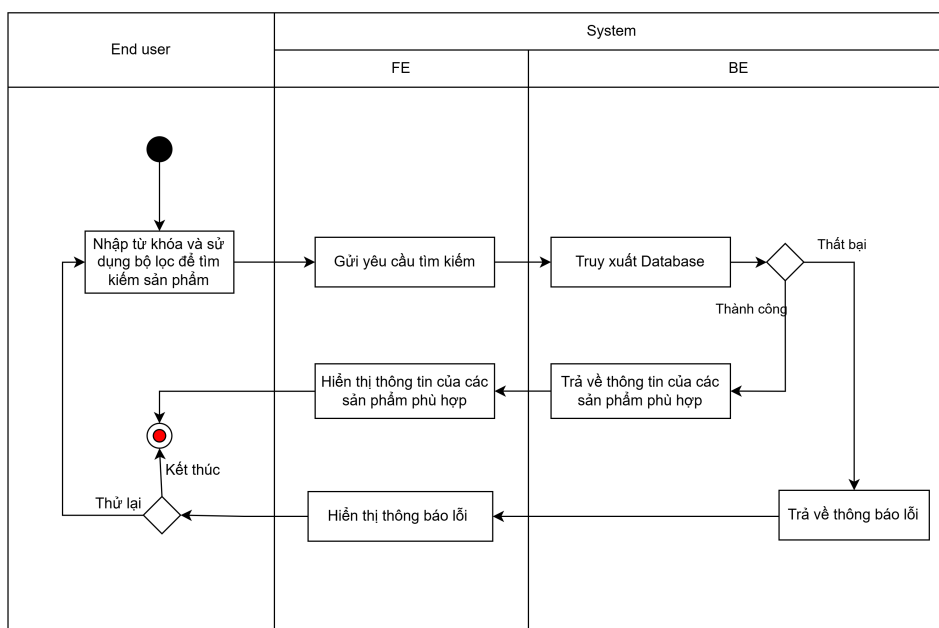
2.6 Sơ đồ hoạt động

Trong khi các bảng mô tả trường hợp sử dụng tập trung vào việc xác định các thành phần nghiệp vụ, sơ đồ hoạt động đóng vai trò minh họa các khía cạnh động của hệ thống. Các sơ đồ này chi tiết hóa luồng xử lý dữ liệu, các điểm quyết định và trình tự tương tác giữa các tác nhân với các thành phần hệ thống. Qua đó, nhóm có thể kiểm chứng tính logic của quy trình trước khi tiến hành cài đặt mã nguồn.

Tương tự như phần đặc tả chi tiết, trong phạm vi chương này nhóm trình bày các sơ đồ hoạt động cho những trường hợp sử dụng cốt lõi bao gồm các luồng tìm kiếm, đặt hàng, thanh toán và báo cáo quản trị thông qua các hình từ 2.6 đến 2.10. Các sơ đồ hoạt động

cho những nghiệp vụ hỗ trợ và quản lý khác được trình bày đầy đủ tại phần phụ lục ?? của đề án.

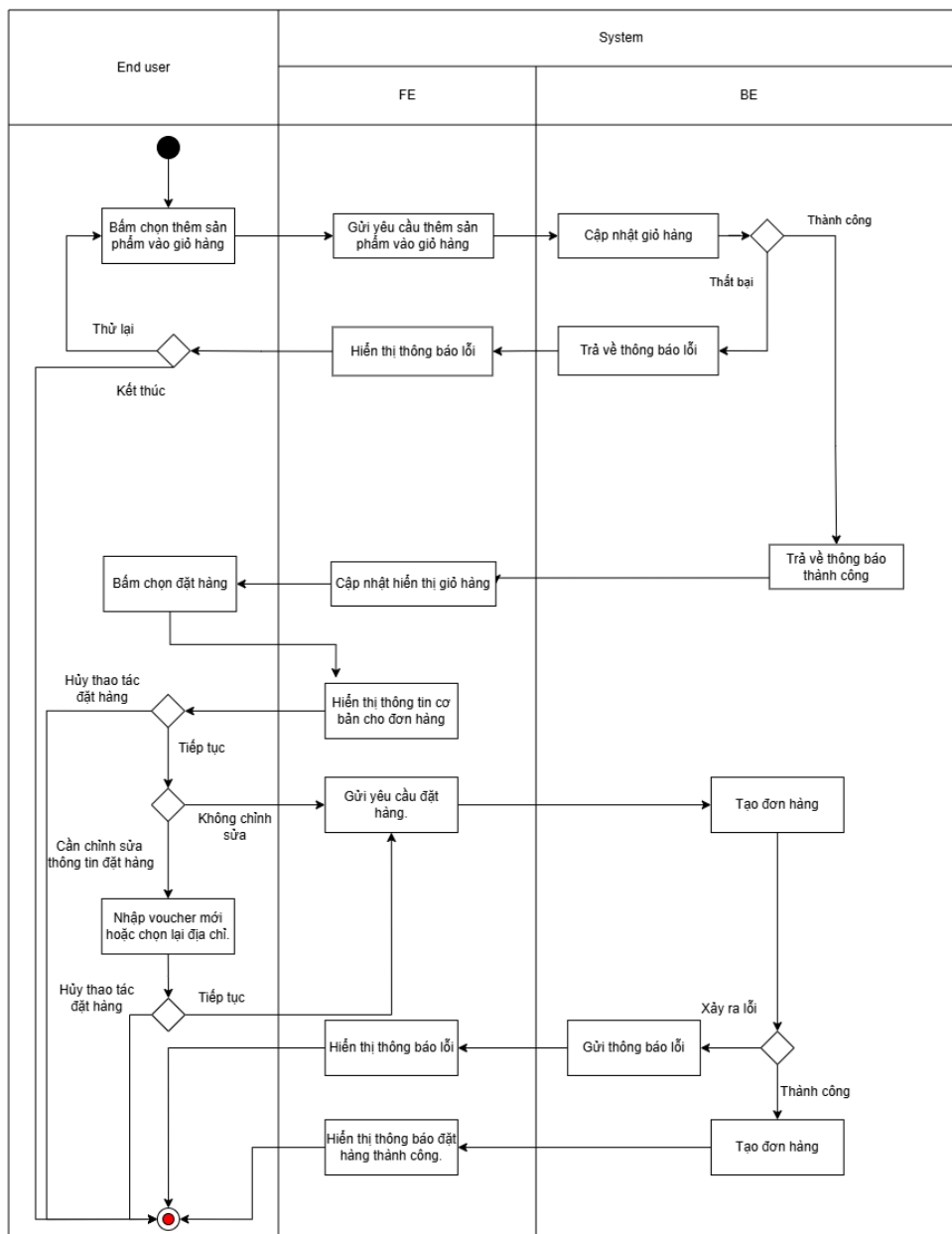
Tim kiếm sản phẩm



Hình 2.6: Sơ đồ hoạt động Tìm kiếm sản phẩm

Hình 2.6 mô tả luồng người dùng nhập từ khóa hoặc chọn danh mục, hệ thống lọc dữ liệu và hiển thị danh sách sản phẩm phù hợp cùng chi tiết sản phẩm.

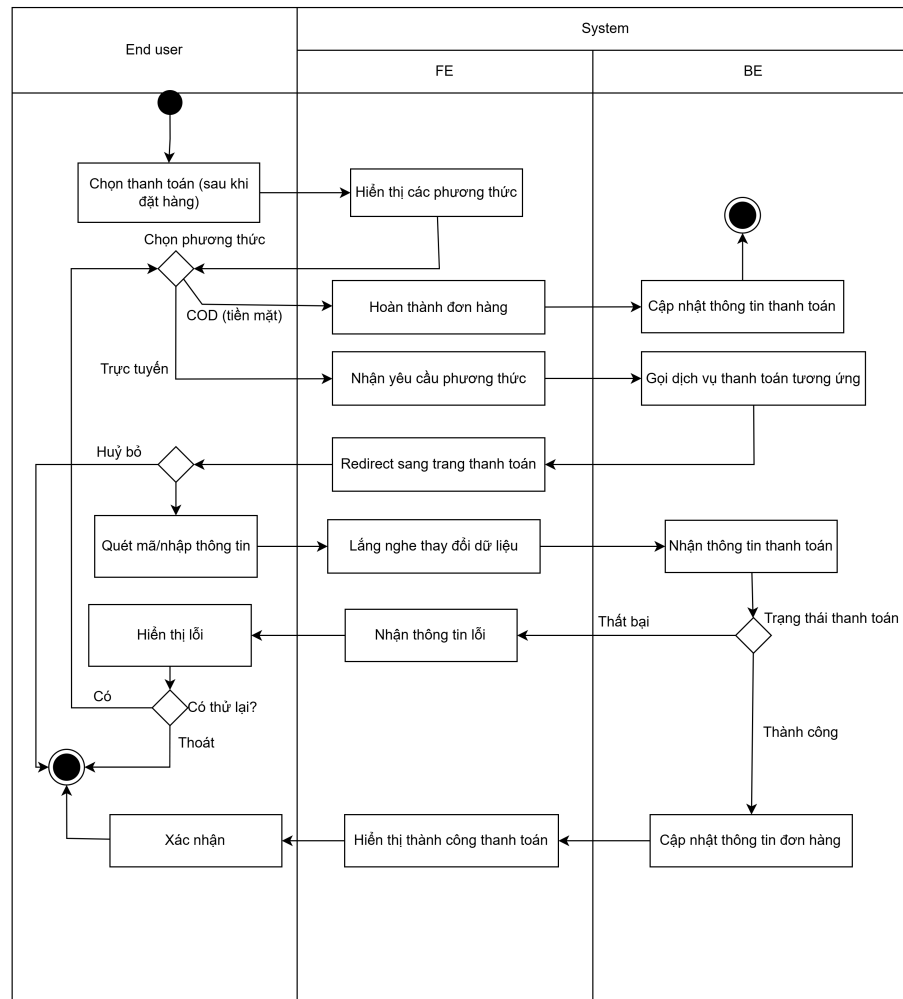
Thêm sản phẩm vào giỏ hàng và đặt hàng



Hình 2.7: Sơ đồ hoạt động Thêm vào giỏ hàng và Đặt hàng

Hình 2.7 mô tả quy trình cốt lõi: Người dùng chọn sản phẩm, thêm vào giỏ, kiểm tra lại giỏ hàng, nhập thông tin giao hàng và xác nhận tạo đơn hàng.

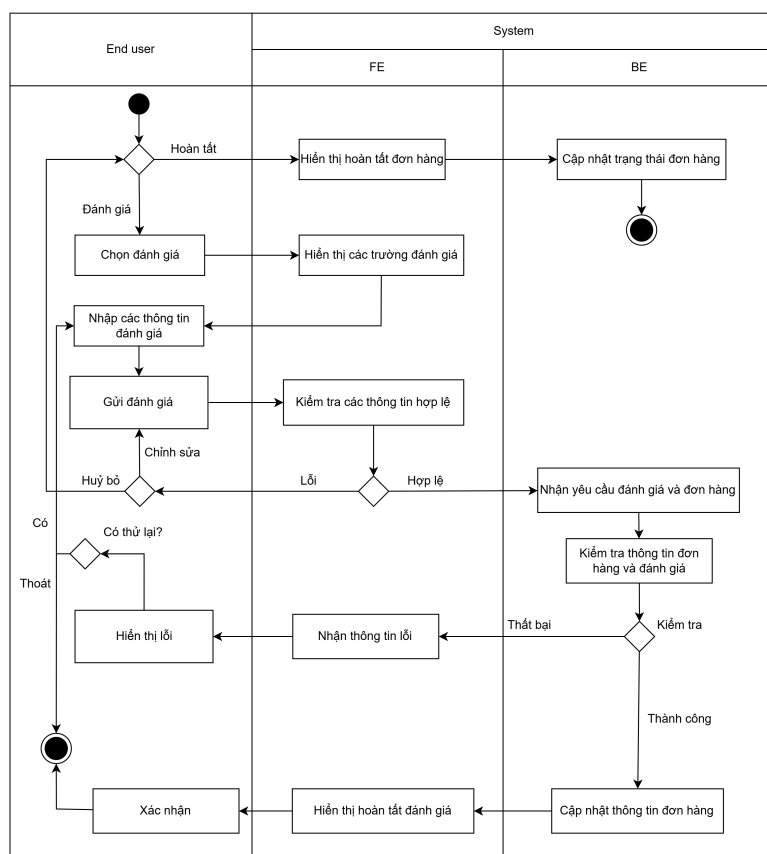
Thanh toán đa phương thức



Hình 2.8: Sơ đồ hoạt động Lựa chọn phương thức thanh toán

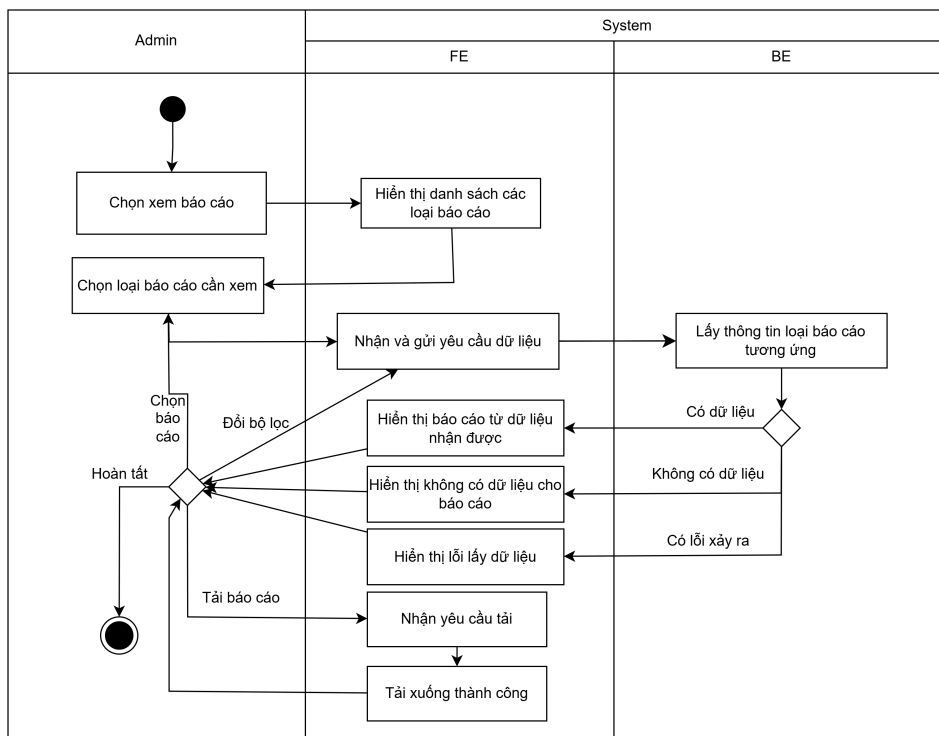
Hình ?? chi tiết hóa bước thanh toán, cho phép người dùng chọn giữa các phương thức (COD, Ví điện tử, Chuyển khoản) và xử lý luồng giao dịch tương ứng.

Đánh giá và phản hồi của khách hàng



Hình 2.9: Sơ đồ hoạt động Phản hồi ý kiến khách hàng

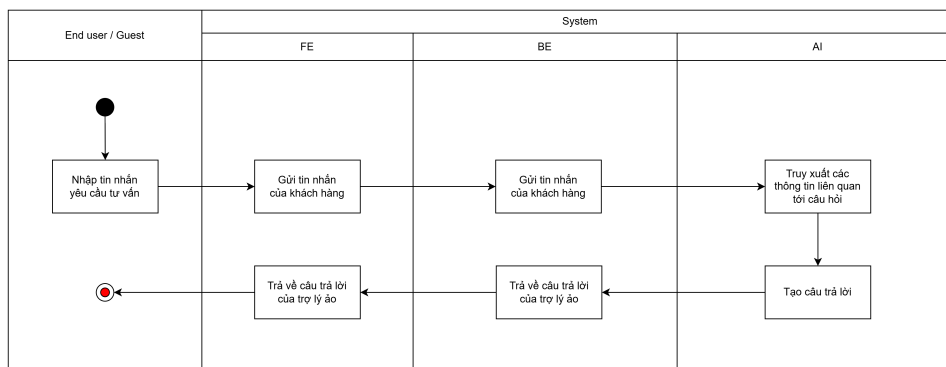
Quy trình ghi nhận đánh giá, nhận xét của khách hàng về sản phẩm hoặc dịch vụ để cải thiện chất lượng hệ thống được biểu diễn ở hình 2.9.



Hình 2.10: Sơ đồ hoạt động Xem báo cáo thống kê

Chức năng tổng hợp dữ liệu, hiển thị báo cáo doanh thu, số lượng đơn hàng và tồn kho cho người quản lý được biểu diễn qua hình 2.10.

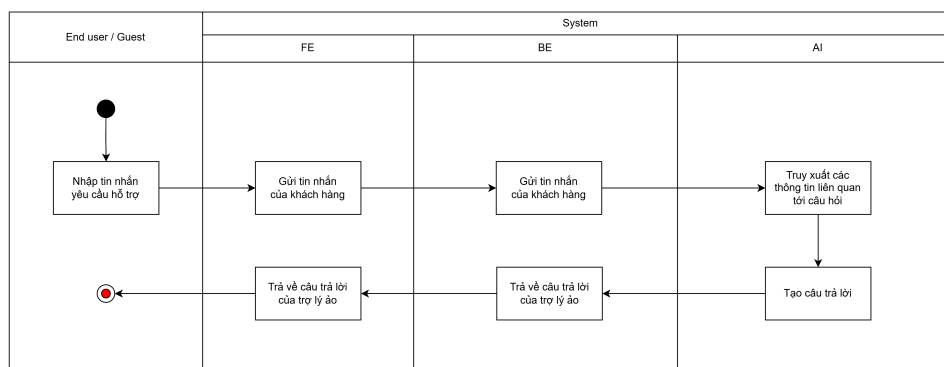
Tư vấn mua hàng



Hình 2.11: Sơ đồ hoạt động Tư vấn mua sắm

Mô tả tương tác giữa khách hàng và nhân viên/chatbot để nhận tư vấn về thông tin sản phẩm trước khi quyết định mua.

Hỗ trợ trong và sau mua hàng



Hình 2.12: Sơ đồ hoạt động Hỗ trợ sau bán hàng

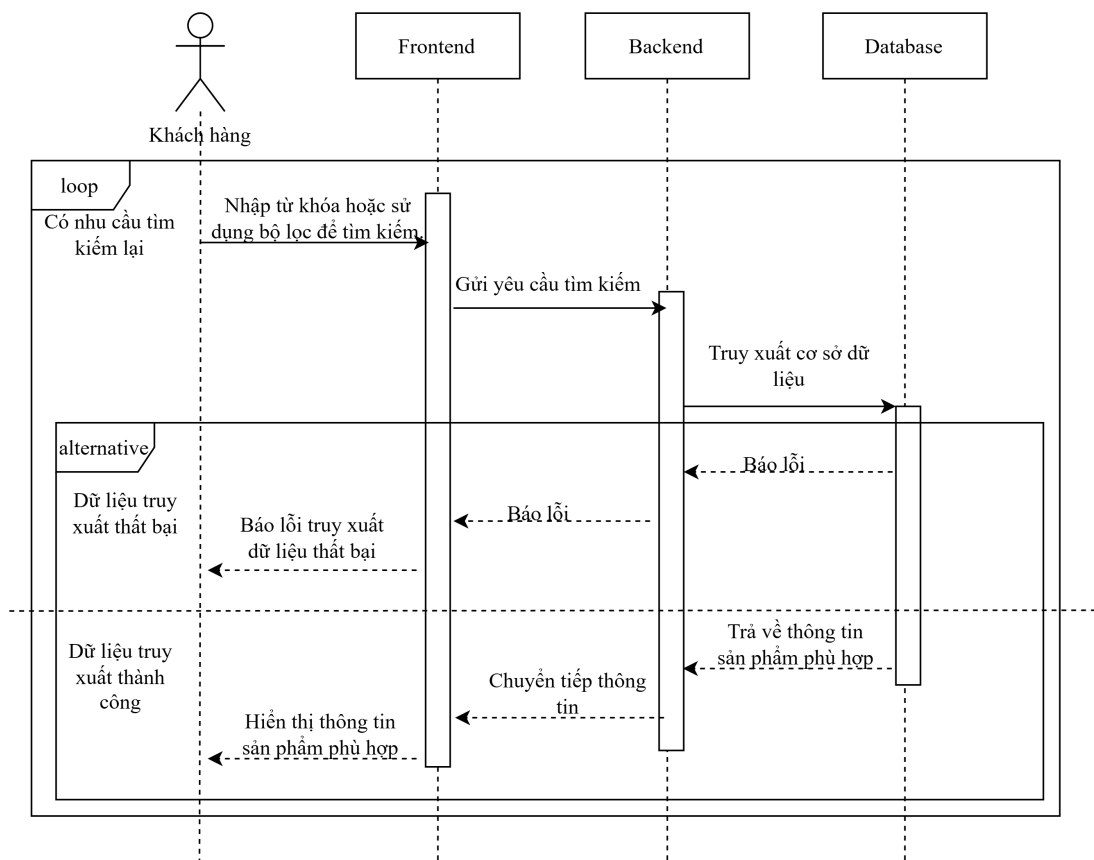
Xử lý các yêu cầu khiếu nại, bảo hành hoặc các vấn đề phát sinh sau khi khách hàng đã nhận được sản phẩm.

2.7 Sơ đồ tuần tự

Trong khi sơ đồ hoạt động tập trung vào luồng xử lý logic và các điểm quyết định, sơ đồ tuần tự đóng vai trò chi tiết hóa sự tương tác giữa các đối tượng trong hệ thống theo trình tự thời gian. Các sơ đồ này làm rõ cách thức các dịch vụ Backend, cơ sở dữ liệu và trợ lý ảo AI phối hợp truyền tải thông điệp để hoàn thành một nghiệp vụ cụ thể. Đây là cơ sở quan trọng để nhóm xác định các phương thức và cấu trúc giao tiếp giữa các Microservices.

Các hình từ 2.13 đến 2.17 trình bày các sơ đồ hoạt động cho những trường hợp sử dụng cốt lõi của hệ thống. Các sơ đồ còn lại sẽ được biểu diễn ở phần phụ lục ??.

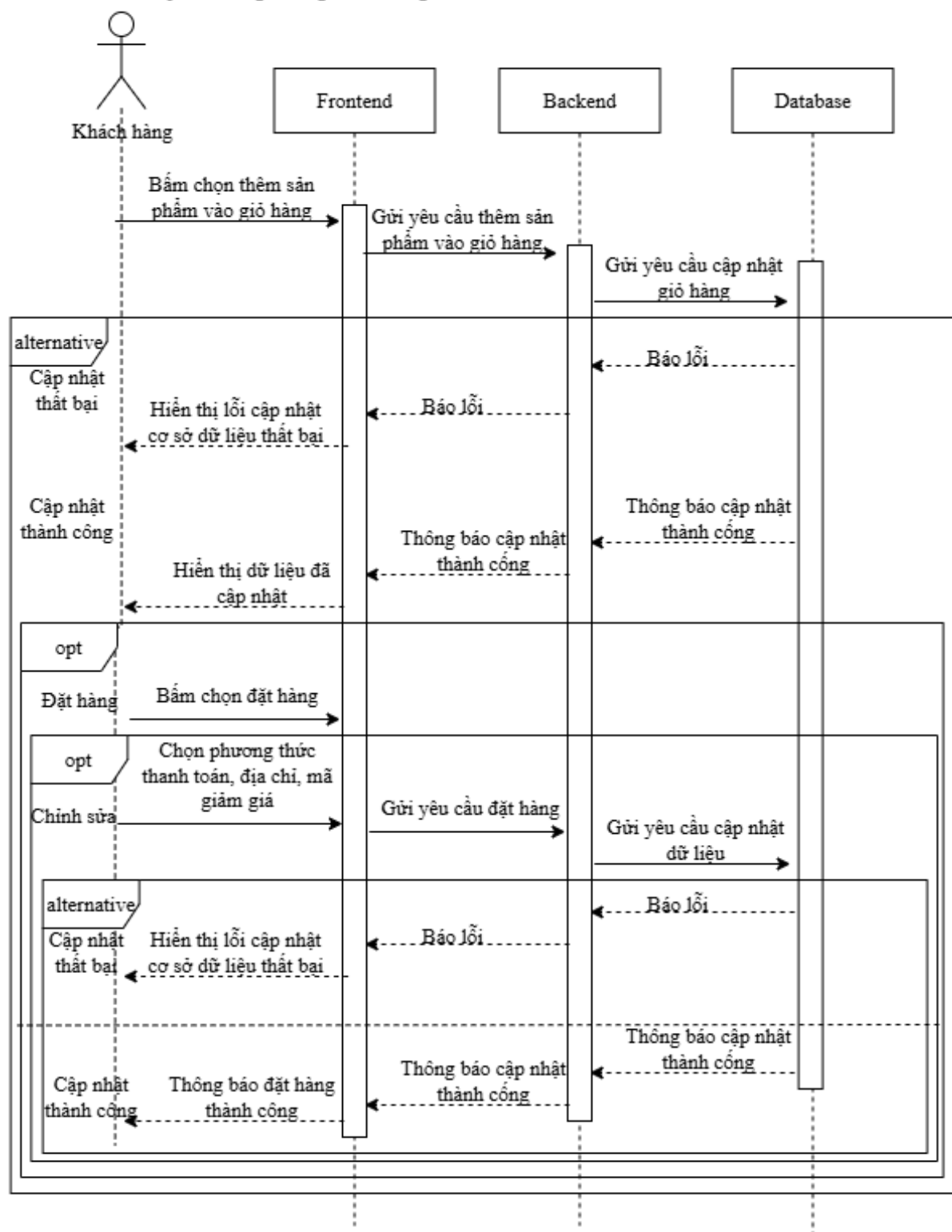
Usecase: Tìm kiếm sản phẩm



Hình 2.13: Sơ đồ tuần tự Tìm kiếm sản phẩm

Hình 2.13 mô tả luồng khi người dùng gửi từ khóa hoặc bộ lọc. Controller xử lý yêu cầu, truy vấn danh sách sản phẩm từ Database thỏa mãn điều kiện và trả về danh sách kết quả hiển thị cho người dùng.

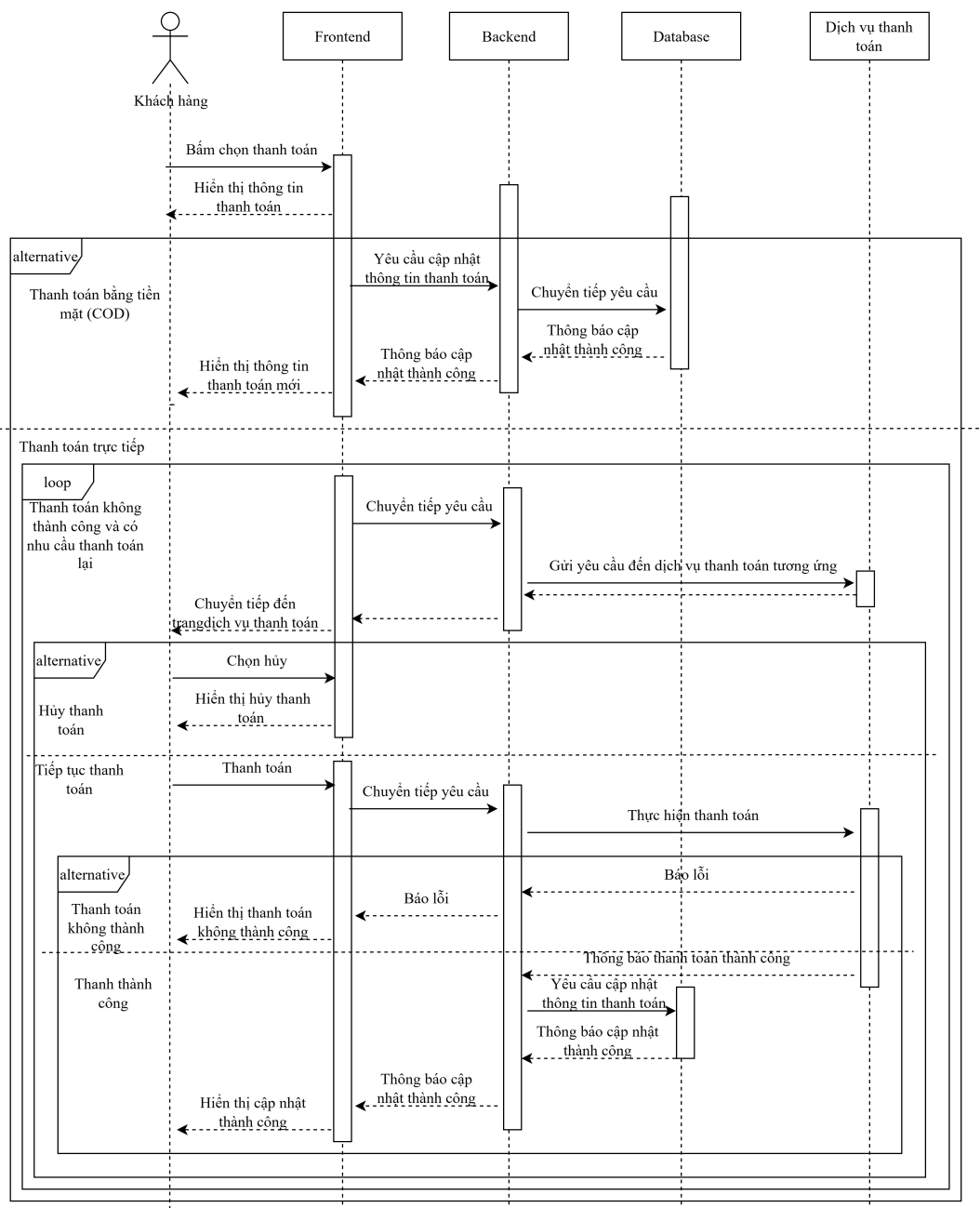
Usecase: Thêm sản phẩm vào giỏ hàng và đặt hàng



Hình 2.14: Sơ đồ tuần tự Thêm vào giỏ và Đặt hàng

Hình 2.14 mô tả sơ đồ phức tạp mô tả chuỗi hành động: Kiểm tra tồn kho khi thêm vào giỏ -> Cập nhật giỏ hàng -> Khởi tạo đơn hàng (Order) -> Lưu chi tiết đơn hàng (Order Details) và xác nhận đặt hàng thành công.

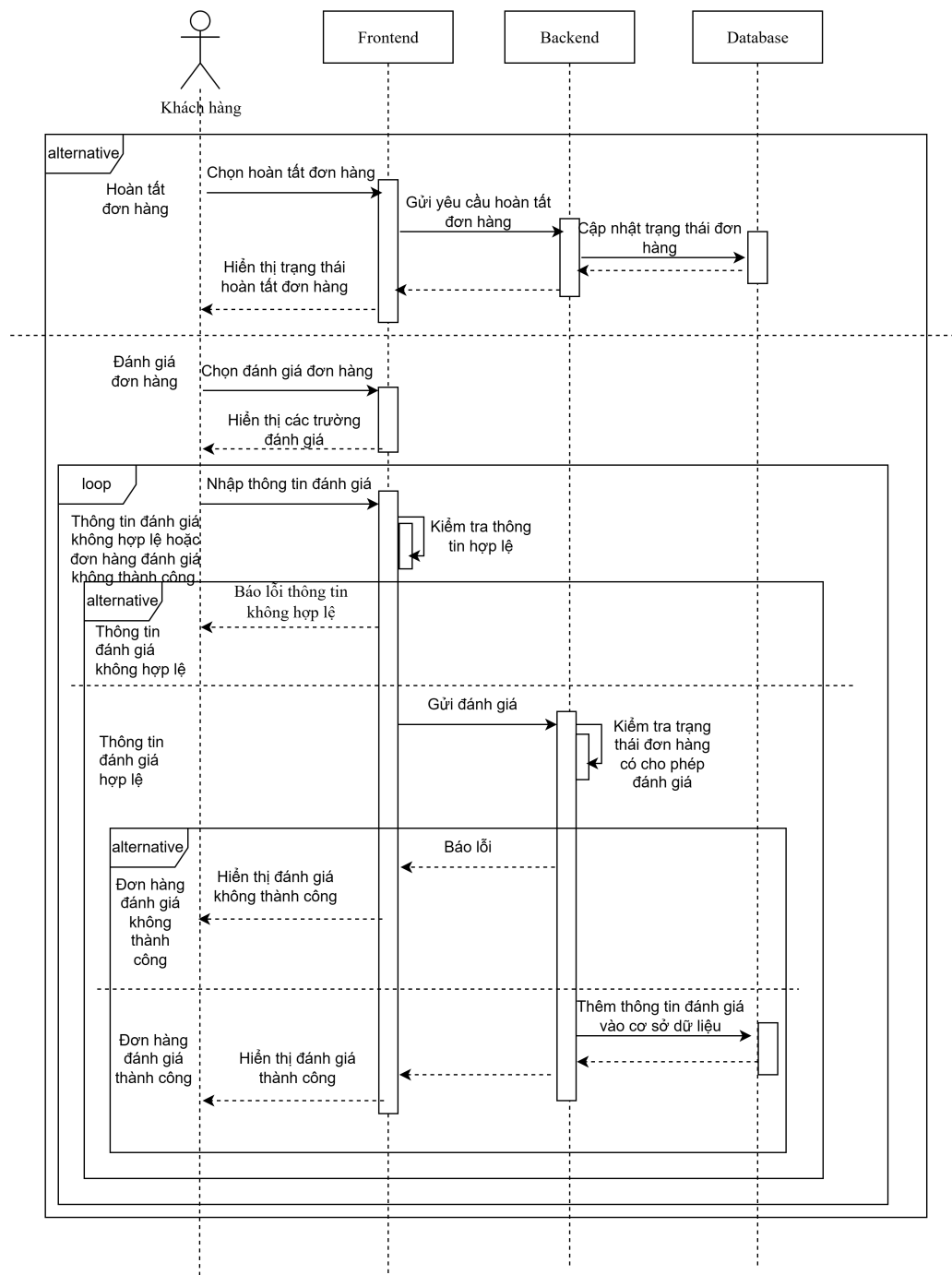
Usecase: Thanh toán đa phương thức



Hình 2.15: Sơ đồ tuần tự Thanh toán đa phương thức

Hình 2.15 mô tả việc người dùng chọn phương thức thanh toán. Hệ thống có thể tương tác với cổng thanh toán bên ngoài (API) hoặc xử lý thanh toán nội bộ, sau đó cập nhật trạng thái thanh toán cho đơn hàng.

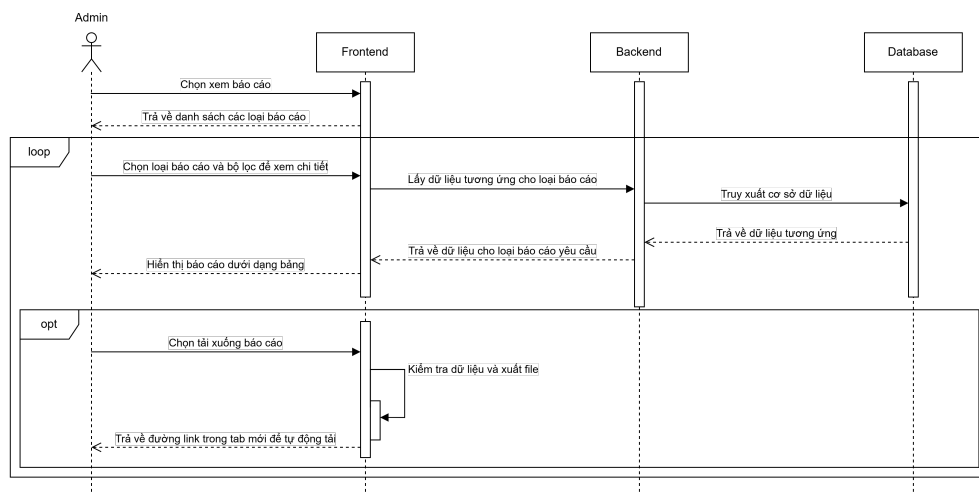
Usecase: Đánh giá và phản hồi



Hình 2.16: Sơ đồ tuần tự Gửi phản hồi khách hàng

Hình 2.16 cho thấy sau khi hoàn tất đơn hàng, người dùng gửi đánh giá. Hệ thống lưu trữ nội dung phản hồi và liên kết nó với sản phẩm hoặc đơn hàng tương ứng trong cơ sở dữ liệu.

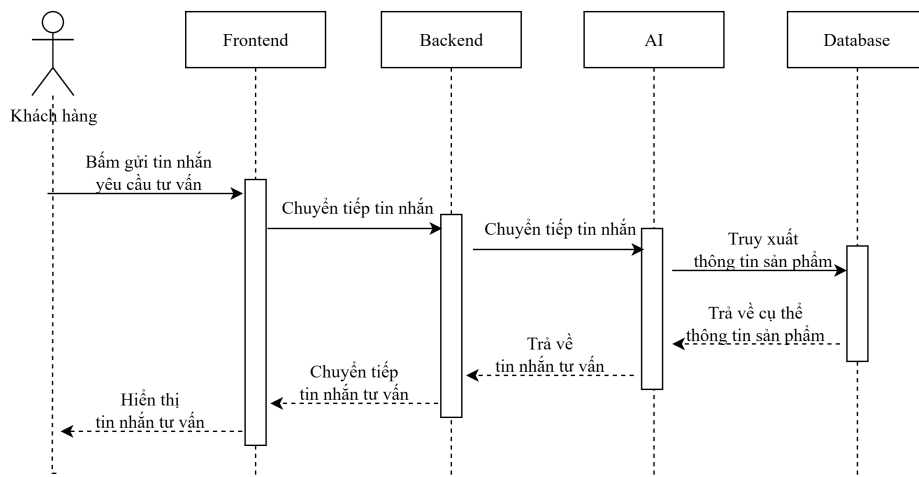
Xem báo cáo cơ bản



Hình 2.17: Sơ đồ tuần tự Xem báo cáo thống kê

Hình 2.17 biểu diễn luồng khi admin chọn loại báo cáo và khoảng thời gian. Hệ thống tổng hợp dữ liệu từ các bảng Order, Product, Payment,... tính toán các chỉ số và hiển thị biểu đồ hoặc bảng số liệu.

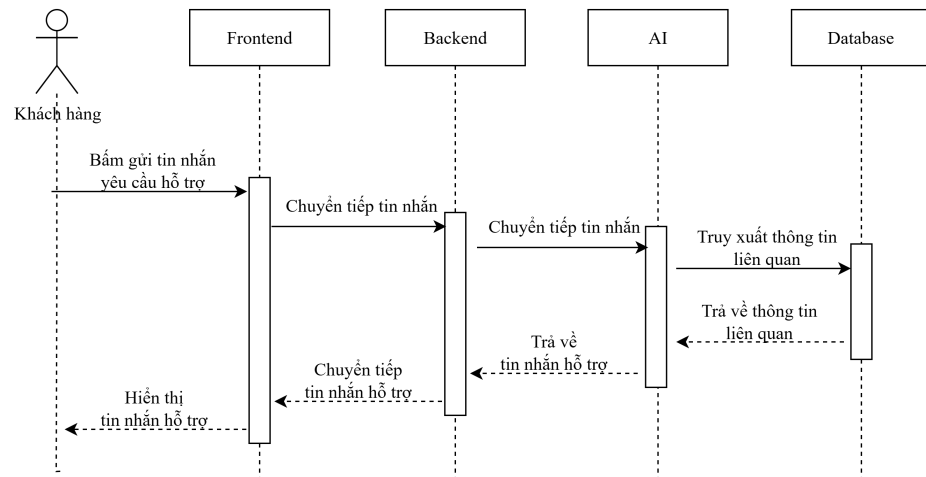
Usecase: Tư vấn mua hàng



Hình 2.18: Sơ đồ tuần tự Tư vấn mua sắm

Mô tả luồng tin nhắn giữa Khách hàng và Nhân viên hỗ trợ/Hệ thống Chatbot để giải đáp thắc mắc về sản phẩm trước khi mua.

Usecase: Hỗ trợ trong và sau mua hàng



Hình 2.19: Sơ đồ tuần tự Hỗ trợ sau mua hàng

Quy trình xử lý các khiếu nại hoặc yêu cầu bảo hành. Nhân viên kiểm tra thông tin đơn hàng trên hệ thống và ghi nhận kết quả xử lý hỗ trợ.

3 Cơ sở lý thuyết

Chương này tập trung vào các cơ sở lý thuyết quan trọng về kiến trúc hệ thống và AI Agent. Những cơ sở lý thuyết này đóng vai trò làm nền tảng cho việc thực hiện đề tài.

3.1 Lý thuyết về kiến trúc hệ thống

3.1.1. Kiến trúc Monolithic

Kiến trúc Monolithic là mô hình kiến trúc trong đó toàn bộ các phần của hệ thống như giao diện, xử lý nghiệp vụ và truy cập dữ liệu được gộp chung trong một ứng dụng duy nhất. Ứng dụng được xây dựng và triển khai như một khối thống nhất. Mọi thành phần của hệ thống chạy trong cùng tiến trình, dùng chung tài nguyên và phụ thuộc chặt chẽ vào nhau.

Đặc điểm:

- Mọi chức năng nằm trong **một mã nguồn và một tiến trình chạy**[1].
- Các phần hệ thống phụ thuộc chặt chẽ vào nhau.
- Thường tổ chức theo các tầng: Presentation – Business – Data Access[1].

Ưu điểm:

- Dễ phát triển ban đầu, phù hợp nhóm nhỏ.
- Triển khai đơn giản, ít yêu cầu hạ tầng[1].
- Hiệu năng tốt cho hệ thống nhỏ.
- Kiểm thử tổng thể dễ dàng.

Nhược điểm:

- Khó mở rộng, phải mở rộng cả hệ thống.
- Khó bảo trì khi lớn dần, dễ bị rối.
- Triển khai chậm, khi cần thay đổi nhỏ cũng phải re-deploy toàn bộ hệ thống.
- Khả năng chịu lỗi thấp.
- Khó áp dụng công nghệ mới cho từng phần riêng.

3.1.2. Kiến trúc Microservices

Kiến trúc Microservices là kiến trúc mà ở đó hệ thống được phân tách thành các dịch vụ (service) nhỏ, độc lập — mỗi dịch vụ đảm nhận một chức năng riêng và có thể phát triển cũng như triển khai mà không ảnh hưởng đến phần còn lại.

Các dịch vụ giao tiếp với nhau qua API hoặc các kênh trao đổi nhẹ, cho phép hệ thống linh hoạt, dễ mở rộng, dễ bảo trì và thuận tiện khi cần nâng cấp hoặc mở rộng từng phần riêng lẻ.

Đặc điểm:

- Mỗi dịch vụ có **mã nguồn và vòng đời triển khai riêng**[2].
- Dịch vụ có thể dùng **ngôn ngữ hoặc công nghệ khác nhau**.
- Dễ phân tách theo nghiệp vụ, phù hợp tổ chức theo nhóm nhỏ độc lập[2].

Ưu điểm:

- Dễ mở rộng, có thể mở rộng từng dịch vụ theo nhu cầu.
- Triển khai linh hoạt, thay đổi dịch vụ này không ảnh hưởng dịch vụ khác.
- Tăng tính ổn định, lỗi dịch vụ không làm sập toàn hệ thống.
- Cho phép áp dụng công nghệ khác nhau cho từng phần.

Nhược điểm:

- Phức tạp hơn kiến trúc Monolithic, đòi hỏi DevOps và hạ tầng tốt.
- Theo dõi và kiểm thử khó hơn vì hệ thống phân tán[2].
- Chi phí giao tiếp giữa các dịch vụ cao hơn[2].
- Cần thêm các thành phần như API Gateway, dịch vụ quản lý cấu hình, giám sát phân tán.

3.1.3. Kiến trúc EDA + Cloud-Native

Kiến trúc EDA (Event-Driven Architecture) + Cloud-Native được phát triển dựa trên kiến trúc Microservices mà ở đó có hai điểm khác biệt rõ nhất đó là tổ chức hệ thống theo mô hình phát sinh sự kiện (event) – truyền sự kiện – xử lý sự kiện và hướng tới xây dựng ứng dụng tối ưu cho môi trường điện toán đám mây, thường dựa trên container và nền tảng điều phối như Docker hoặc Kubernetes.

Khi kết hợp, hệ thống trở nên linh hoạt, mở rộng tốt, xử lý bất đồng bộ hiệu quả và phù hợp với môi trường phân tán hiện đại.

Đặc điểm:

- Giao tiếp qua sự kiện, thông qua hàng đợi hoặc hệ thống truyền tin[3].
- Sử dụng container và hoạt động trên môi trường Cloud nên có khả năng **tự động mở rộng, tự phục hồi**[3].

- Hoạt động tốt trong môi trường nhiều dịch vụ, khối lượng dữ liệu lớn và thay đổi nhanh.

Ưu điểm:

- Xử lý bất đồng bộ, phù hợp tải cao[3].
- Khả năng mở rộng rất tốt, tự tăng/giảm tài nguyên theo nhu cầu[3].
- Tính chịu lỗi cao, sự kiện được lưu lại ngay cả khi dịch vụ xử lý tạm lỗi[3].
- Dễ thêm chức năng mới, chỉ cần thêm dịch vụ tiêu thụ sự kiện.
- Tối ưu chi phí, phân bổ tài nguyên linh hoạt.

Nhược điểm:

- Thiết kế và vận hành phức tạp, đòi hỏi hiểu biết sâu về truyền tin, hàng đợi, container[3].
- Khó theo dõi luồng xử lý, cần công cụ giám sát mạnh.
- Việc thiết kế sự kiện không hợp lý có thể gây trùng dữ liệu hoặc rối luồng xử lý[3].
- Cần hạ tầng cloud và DevOps tốt[3].

3.1.4. So sánh giữa các kiến trúc hệ thống

Bảng 3.1: So sánh giữa các kiến trúc hệ thống

Tiêu chí	Monolithic	Microservices	EDA + Cloud-Native
Khả năng mở rộng	Tệ	Tốt	Rất tốt
Mức độ phức tạp	Thấp	Trung bình	Cao
Độ tin cậy	Thấp	Ổn	Rất tốt
Chi phí vận hành	Thấp	Trung bình	Cao
Tốc độ triển khai	Chậm	Nhanh	Rất nhanh
Tính linh hoạt	Thấp	Cao	Rất cao
Phù hợp quy mô hệ thống	Nhỏ – trung bình	Lớn	Rất lớn / phân tán
Hiệu quả khi chạy trên Cloud	Tệ	Tốt	Rất tốt

Nhìn chung, các kiến trúc Monolithic, Microservices và EDA + Cloud-Native khác nhau chủ yếu ở mức độ phân tách thành phần và cách các thành phần giao tiếp với nhau. Mỗi kiến trúc thể hiện một mức độ tiến hóa khác nhau trong cách tổ chức hệ thống phần mềm hiện đại. Trong tiến trình phát triển kiến trúc phần mềm, xu hướng đang chuyển dần từ mô hình nguyên khối sang các mô hình phân tán và sự kiện nhằm đáp ứng nhu cầu mở rộng, độ tin cậy và tính linh hoạt cao hơn. Các kiến trúc hiện đại như Microservices và EDA + Cloud-Native được xem là phù hợp với môi trường triển khai đa nền tảng, tự động hóa và điện toán đám mây.

3.2 Lý thuyết về AI Agent

3.2.1. Trí tuệ nhân tạo

Trí tuệ nhân tạo (Artificial Intelligence – AI) là công nghệ cho phép máy tính và các hệ thống kỹ thuật mô phỏng những năng lực cốt lõi của trí tuệ con người như học tập, hiểu biết, giải quyết vấn đề, ra quyết định, sáng tạo và hành động tự chủ. Các hệ thống AI có khả năng nhận biết môi trường, hiểu và phản hồi ngôn ngữ tự nhiên, học hỏi từ dữ liệu và kinh nghiệm, đồng thời thực hiện các hành động mà không cần hoặc cần rất ít sự can thiệp của con người[4].

AI được xây dựng dựa trên nhiều kỹ thuật và mô hình tính toán khác nhau, trong đó học máy và học sâu đóng vai trò nền tảng. Các ứng dụng được trang bị AI có thể phân tích dữ liệu phức tạp, đưa ra khuyến nghị chi tiết và hỗ trợ hoặc thay thế con người trong nhiều lĩnh vực như giao thông tự hành, chăm sóc khách hàng và phân tích dữ liệu[4].

Các lĩnh vực chính của Trí tuệ nhân tạo bao gồm:

- Học máy (Machine Learning)
- Học sâu (Deep Learning)
- Trí tuệ nhân tạo sinh (Generative AI)

Phần lớn các hệ thống AI hiện nay thuộc nhóm AI hẹp (Weak AI hay Narrow AI), được thiết kế để thực hiện hiệu quả một nhiệm vụ hoặc một nhóm nhiệm vụ cụ thể. Trí tuệ nhân tạo tổng quát (Strong AI hay AGI), với khả năng hiểu và áp dụng tri thức trên nhiều lĩnh vực ở mức tương đương hoặc vượt con người, hiện vẫn mang tính lý thuyết và chưa được hiện thực hóa[4].

3.2.2. AI Agent

AI Agent là mô hình Trí tuệ nhân tạo được mở rộng thêm khả năng hành động thông qua các công cụ được tích hợp. Thay vì chỉ sinh văn bản, Agent có thể phân tích yêu cầu, lập kế hoạch, lựa chọn hành động phù hợp và truy xuất dữ liệu thực tế từ các nguồn bên ngoài[5].

Một AI Agent điển hình bao gồm:

- **Bộ tư duy (reasoning loop)** để phân tích và hiểu mục tiêu[5].
- **Bộ lập kế hoạch (planner)** để xác định các bước thực hiện[5].
- **Cơ chế tool/function calling** để gọi API, truy vấn cơ sở dữ liệu hoặc xử lý thông tin[5].

3.2.3. ReAct

ReAct (Reasoning and Acting) là một phương pháp prompting cho mô hình ngôn ngữ lớn (LLMs), được đề xuất nhằm kết hợp lập luận bằng ngôn ngữ và hành động tương tác với môi trường trong cùng một tiến trình giải quyết nhiệm vụ[6].

Trong ReAct, mô hình hoạt động như một agent tương tác theo chuỗi xen kẽ Thought – Action – Observation. Ở mỗi bước, mô hình sinh ra:

- **Thought:** bước suy luận bằng ngôn ngữ tự nhiên để phân tích ngữ cảnh và lập kế hoạch[6].
- **Action:** hành động cụ thể để tương tác với môi trường hoặc công cụ bên ngoài (ví dụ: truy vấn thông tin)[6].
- **Observation:** kết quả quan sát được từ hành động vừa thực hiện, làm đầu vào cho bước tiếp theo[6].

Điểm đặc trưng của ReAct là các bước suy luận không trực tiếp tác động đến môi trường, nhưng được sử dụng để định hướng hành động, trong khi các hành động giúp thu thập thông tin mới nhằm hỗ trợ quá trình suy luận tiếp theo[6].

ReAct thường được triển khai dưới dạng prompting với một vài ví dụ (few-shot prompting), không yêu cầu huấn luyện lại mô hình, và đã cho thấy hiệu quả trong các tác vụ như hỏi đáp nhiều bước, truy xuất tri thức và điều hướng môi trường văn bản[6].

Nhờ khả năng kết hợp lập luận và hành động, ReAct giúp tăng tính minh bạch của quá trình suy luận và giảm hiện tượng sinh thông tin sai lệch so với các phương pháp chỉ dựa trên suy luận nội bộ[6].

3.2.4. RAG

RAG (Retrieval-Augmented Generation) là kỹ thuật kết hợp mô hình ngôn ngữ với hệ thống truy xuất thông tin nhằm nâng cao độ chính xác và tính nhất quán của câu trả lời. Thay vì dựa hoàn toàn vào kiến thức có sẵn trong mô hình, RAG tìm kiếm các tài liệu liên quan từ nguồn dữ liệu bên ngoài rồi đưa chúng vào ngữ cảnh (context) để mô hình sinh câu trả lời[7].

Quy trình RAG gồm ba bước:

- **Embedding:** chuyển tài liệu thành dạng vector số[7].
- **Retrieval:** tìm các vector gần nhất với vector ứng với câu hỏi của người dùng[7].
- **Augmentation:** đưa nội dung truy xuất được vào ngữ cảnh trước khi cho mô hình trả lời[7].

RAG đặc biệt hữu ích trong các hệ thống yêu cầu thông tin chính xác, cập nhật và bám sát dữ liệu nội bộ, giúp giảm thiểu hiện tượng mô hình sinh ra những thông tin, sự kiện không có thật[7].

3.2.5. Đồ thị tri thức

Đồ thị tri thức (Knowledge Graph) là mô hình biểu diễn tri thức dưới dạng đồ thị, trong đó các thực thể được biểu diễn bằng các nút (nodes) và các quan hệ giữa chúng bằng

cạnh (edges)[8]. Cấu trúc này cho phép lưu trữ thông tin có tính liên kết cao và hỗ trợ suy luận logic.

Một đồ thị tri thức thường bao gồm:

- **Nút:** để biểu diễn các thực thể có dữ liệu và quan hệ với các thực thể khác[8].
- **Cạnh:** để thể hiện mối quan hệ giữa các nút[8].
- **Nhãn (Label):** lưu trữ thông tin, dữ liệu của nút hoặc cạnh[8].

Đồ thị tri thức giúp các hệ thống giảm thời gian truy xuất các dữ liệu có tính liên kết cao. Bên cạnh đó, nhờ đặc tính lưu trữ dữ liệu theo quan hệ nên đồ thị tri thức mang lại khả năng suy luận ra các tri thức, quan hệ mới dựa trên các quan hệ đã có.

3.2.6. Hệ thống đề xuất

Hệ thống đề xuất (Recommendation System) là những hệ thống nhằm cung cấp trải nghiệm cá nhân hóa cho người dùng bằng cách đề xuất các sản phẩm phù hợp với nhu cầu hoặc sở thích. Các phương pháp phổ biến bao gồm:

- **Lọc cộng tác (Collaborative filtering):** lọc và đề xuất gợi ý cho một người dùng dựa trên sự tương đồng về đặc điểm và hành vi người dùng của họ so với những người dùng đã sử dụng sản phẩm, dịch vụ[9].
- **Lọc theo nội dung (Content-based filtering):** lọc và đề xuất gợi ý cho một người dùng dựa trên đặc điểm của các sản phẩm, dịch vụ mà họ đã sử dụng[9].
- **Phương pháp đề xuất lai (Hybrid):** kết hợp hai phương pháp trên nhằm tối ưu độ chính xác[9].

Hệ thống đề xuất có ý nghĩa to lớn cho cả doanh nghiệp và người dùng. Hệ thống đề xuất giúp cải thiện trải nghiệm người dùng, từ đó giúp tăng khả năng giữ chân khách hàng và nâng cao tỉ lệ chuyển đổi từ người dùng thành khách hàng[9].

4 Công nghệ

Trong chương này, nhóm giới thiệu về các công nghệ mà nhóm đã tìm hiểu, nghiên cứu và quyết định lựa chọn để thực hiện đồ án.

4.1 Backend

Trong giai đoạn nghiên cứu, nhóm đã đánh giá nhiều giải pháp công nghệ để xây dựng các dịch vụ nghiệp vụ cho hệ thống. Quyết định lựa chọn cuối cùng được đưa ra dựa trên sự cân bằng giữa ưu điểm kỹ thuật của từng công nghệ và mức độ phù hợp với thể mạnh chuyên môn của nhóm. Dưới đây là phần phân tích chi tiết.

4.1.1. Framework

Để xây dựng các API và xử lý logic nghiệp vụ cốt lõi, hệ thống yêu cầu một nền tảng framework mạnh mẽ, có khả năng mở rộng tốt và đảm bảo tính toàn vẹn dữ liệu. Nhóm đã cân nhắc ba hệ sinh thái phổ biến nhất hiện nay là Node.js, Python và Java. Mỗi nền tảng đều mang lại những giá trị kiến trúc riêng, nhưng cũng đi kèm với những hạn chế nhất định khi áp dụng vào bài toán thương mại điện tử đặc thù. Bảng 4.1 dưới đây tóm tắt các đánh giá và quyết định cuối cùng của nhóm.

Bảng 4.1: So sánh các framework phát triển Backend

Công nghệ	Ưu điểm	Nhược điểm	Lý do lựa chọn
NestJS	- Tận dụng cơ chế Non-blocking I/O của Node.js, tối ưu hóa tài nguyên khi xử lý lượng lớn yêu cầu đồng thời. - Hỗ trợ sẵn gói Microservices chuyên dụng, giúp tích hợp Message Broker một cách nhất quán và dễ dàng quản lý luồng sự kiện.	- Hiệu năng chịu giới hạn bởi cơ chế Single-threaded của Node.js khi xử lý các tác vụ tính toán logic nặng. - Việc quản lý các giao dịch phân tán vẫn phức tạp và chưa có giải pháp tích hợp sẵn mạnh mẽ như Spring Boot.	Không lựa chọn do hạn chế trong việc xử lý song song các nghiệp vụ phức tạp và thiếu các bộ công cụ quản lý giao dịch chặt chẽ để đảm bảo an toàn dữ liệu.

Bảng 4.1: So sánh các framework phát triển Backend (tt)

Công nghệ	Ưu điểm	Nhược điểm	Lý do lựa chọn
Python FastAPI	- Hiệu suất thực thi cao vượt trội nhờ chuẩn ASGI và lập trình bất đồng bộ. - Tích hợp sẵn chuẩn OpenAPI, tự động hóa việc tạo tài liệu và kiểm thử trực tiếp trên Swagger UI.	- Cơ chế GIL gây hạn chế trong việc tận dụng tối đa sức mạnh đa nhân CPU cho các tác vụ xử lý song song. - Tiêu tốn nhiều tài nguyên hệ thống và gặp hạn chế về hiệu năng khi xử lý đồng thời lượng lớn giao dịch phức tạp.	Không chọn vì bản chất thông dịch và cơ chế quản lý luồng chưa đáp ứng tốt yêu cầu xử lý các nghiệp vụ phức tạp và tính khắt khe trong kiểm soát tài nguyên.
Java Spring Boot	- Hỗ trợ hoàn hảo cho mô hình CQRS nhờ sự kết hợp linh hoạt giữa JPA (ghi) và JDBC (đọc). - Quản lý giao dịch xuất sắc, đảm bảo an toàn dữ liệu tuyệt đối. - Hệ sinh thái Spring Cloud và Spring Security mạnh mẽ, tối ưu cho Microservices phức tạp.	- Mức độ tiêu thụ tài nguyên bộ nhớ cao. - Cấu hình ban đầu phức tạp và thời gian khởi động chậm.	Được chọn nhờ khả năng hỗ trợ kiến trúc CQRS ổn định, đảm bảo tính toàn vẹn dữ liệu và ưu thế OOP giúp mã nguồn dễ bảo trì, xử lý hiệu quả các nghiệp vụ lỗi.

Spring Boot là một framework dựa trên ngôn ngữ Java giúp đơn giản hóa quá trình phát triển các ứng dụng cấp doanh nghiệp mạnh mẽ và sẵn sàng cho môi trường vận hành thực tế [10]. Nền tảng này kế thừa sức mạnh từ hệ sinh thái Spring Framework với các tính năng cốt lõi như tiêm phụ thuộc và quản lý dịch vụ linh hoạt. Với triết lý cấu hình ưu tiên sự tiện lợi, Spring Boot giúp nhóm tập trung hoàn toàn vào việc hiện thực hóa các luồng logic nghiệp vụ. Đặc biệt, Spring Boot cung cấp bộ công cụ Spring Web mạnh mẽ giúp việc xây dựng các RESTful API trở nên nhanh chóng thông qua các cơ chế xử lý dữ liệu tự động và quản lý lỗi hệ thống một cách nhất quán.

Hệ thống yêu cầu một giải pháp backend hoạt động ổn định và hỗ trợ kiến trúc Microservices hiệu quả. Trong khi NestJS và FastAPI bộc lộ những hạn chế về khả năng kiểm soát tài nguyên khi hệ thống mở rộng, Spring Boot khẳng định vị thế nhờ hệ sinh thái Spring Cloud mạnh mẽ giúp điều phối các dịch vụ phân tán dễ dàng. Khả năng tích hợp sẵn với các thư viện như Spring Data JPA giúp việc thao tác với cơ sở dữ liệu trở nên mạch lạc và an toàn hơn. Sự cân bằng giữa hiệu năng thực thi, khả năng bảo mật tích hợp sẵn và ưu thế của lập trình hướng đối tượng chính là lý do nhóm quyết định lựa chọn Java Spring Boot làm nền tảng cốt lõi cho backend nghiệp vụ của dự án.

4.1.2. Message Broker

Để hiện thực hóa kiến trúc Microservices kết hợp mô hình Event-driven, việc thiết lập một kênh giao tiếp bất đồng bộ là yêu cầu cốt lõi nhằm triệt tiêu sự phụ thuộc trực tiếp giữa các dịch vụ. Bảng 4.2 dưới đây là bản đánh giá quyết định cuối cùng của nhóm.

Bảng 4.2: So sánh các message broker

Công nghệ	Ưu điểm	Nhược điểm	Lý do lựa chọn
Redis Pub/Sub	- Hoạt động hoàn toàn trên bộ nhớ đệm, mang lại tốc độ truyền tin cực cao với độ trễ gần như bằng không. - Mô hình phát tán đơn giản, phù hợp cho các tính năng cần phản hồi tức thì như thông báo hoặc cập nhật trạng thái nhanh.	- Không hỗ trợ lưu trữ sự kiện bền vững; dữ liệu sẽ bị mất vĩnh viễn nếu dịch vụ tiêu thụ đang ngoại tuyến tại thời điểm phát tin. - Thiếu cơ chế đảm bảo an toàn dữ liệu và quản lý luồng phức tạp cho Microservices quy mô lớn.	Không chọn do tính chất "phát và quên" không đảm bảo tính toàn vẹn và an toàn dữ liệu cho các luồng nghiệp vụ quan trọng của hệ thống.
RabbitMQ	- Hỗ trợ các cơ chế định tuyến thông điệp phức tạp và đảm bảo độ tin cậy cao thông qua việc xác nhận nhận tin. - Phù hợp cho các tác vụ xử lý nền và các luồng công việc yêu cầu sự phân phối tin nhắn chính xác đến từng đích cụ thể.	- Quản lý trạng thái tin nhắn tại Broker gây áp lực lên tài nguyên khi số lượng sự kiện lớn hoặc nhiều bên đăng ký nhận tin. - Không hỗ trợ lưu trữ sự kiện bền vững để đọc lại, gây khó khăn khi cần đồng bộ dữ liệu giữa các dịch vụ.	Không lựa chọn vì hạn chế trong việc lưu trữ luồng sự kiện lâu dài và khả năng mở rộng kém linh hoạt hơn so với kiến trúc nhật ký phân tán của Kafka.
Apache Kafka	- Kiến trúc nhật ký phân tán giúp lưu trữ thông điệp bền vững, đảm bảo dữ liệu không bị mất ngay cả khi dịch vụ tiêu thụ gặp sự cố tạm thời. - Khả năng xử lý thông tải lớn và mở rộng linh hoạt, tối ưu cho việc điều phối dữ liệu giữa dịch vụ lõi và các dịch vụ bổ trợ.	- Cấu hình hạ tầng và quy trình triển khai phức tạp hơn so với các giải pháp truyền tin truyền thống. - Độ trễ phản hồi có thể cao hơn các hệ thống thuần In-memory do phải thực hiện ghi dữ liệu xuống đĩa cứng để đảm bảo an toàn.	Được chọn nhờ khả năng lưu trữ thông điệp tin cậy, giúp tách biệt hoàn toàn các dịch vụ và đảm bảo luồng dữ liệu nghiệp vụ luôn được truyền tải ổn định trong kiến trúc Microservices.

Apache Kafka là một nền tảng truyền thông điệp phân tán mạnh mẽ được thiết kế dựa trên kiến trúc nhật ký phân tán giúp lưu trữ dữ liệu bền vững và có khả năng chịu lỗi cao [11]. Thay vì vận hành theo mô hình đẩy tin nhắn truyền thống, Kafka cho phép các dịch vụ tiêu thụ chủ động kéo dữ liệu theo năng lực xử lý riêng giúp hệ thống duy trì sự ổn định ngay cả khi luồng sự kiện tăng đột biến. Nền tảng này nổi bật với khả năng xử lý thông tải cực lớn và cơ chế sao chép dữ liệu trên các cụm máy chủ nhằm đảm bảo tính nhất quán và an toàn dữ liệu tuyệt đối cho toàn bộ hệ thống.

Hệ thống yêu cầu một giải pháp truyền tin mạnh mẽ để đảm bảo sự tách biệt giữa các dịch vụ có tốc độ xử lý khác nhau. Trái với Redis và RabbitMQ bộc lộ những hạn chế về khả năng lưu trữ bền vững và quản lý trạng thái khi quy mô mở rộng, Apache Kafka mạnh mẽ nhờ khả năng lưu giữ thông điệp trên đĩa cứng giúp các dịch vụ bổ trợ tiêu thụ dữ liệu mà không gây áp lực lên dịch vụ nghiệp vụ chính. Việc tích hợp hoàn hảo với hệ sinh thái Spring Cloud cùng khả năng mở rộng ngang linh hoạt giúp nhóm dễ dàng điều phối các luồng dữ liệu phức tạp trong kiến trúc Microservices. Sự cân bằng giữa hiệu suất thực thi, tính bền vững của dữ liệu và khả năng chịu lỗi phân tán chính là lý do nhóm quyết định lựa chọn Apache Kafka làm trục xương sống cho toàn bộ cơ chế giao tiếp hướng sự kiện của dự án.

4.1.3. Reactive Framework

Reactive Framework đóng vai trò then chốt trong việc xây dựng các ứng dụng phân tán có khả năng phản hồi nhanh và chịu tải cao. Thay vì mô hình lập trình truyền thống, khung làm việc này ưu tiên cơ chế xử lý bất đồng bộ và không nghẽn luồng, giúp hệ thống tận dụng tối đa tài nguyên để xử lý hàng nghìn yêu cầu đồng thời.

Bên cạnh Framework chính là Spring Boot, nhóm tích hợp thêm Eclipse Vert.x như một giải pháp để xử lý các tác vụ đòi hỏi hiệu suất cao bên trong từng dịch vụ. Vert.x hoạt động dựa trên mô hình hướng sự kiện và cơ chế vào ra dữ liệu không nghẽn luồng, giúp tối ưu hóa việc sử dụng tài nguyên hệ thống thông qua cơ chế vòng lặp sự kiện.

Trong kiến trúc của dự án, Vert.x Event Bus được sử dụng như một trục giao tiếp nội bộ giữa các thành phần bên trong một microservice. Thay vì sử dụng các lời gọi hàm trực tiếp gây nghẽn luồng xử lý, các thành phần có thể gửi và nhận thông điệp một cách bất đồng bộ thông qua Event Bus. việc kết hợp này cho phép hệ thống tận dụng được khả năng quản lý nghiệp vụ mạnh mẽ của Spring Boot, đồng thời đạt được tốc độ xử lý phản ứng của Vert.x cho các tác vụ tính toán hoặc điều phối dữ liệu nội bộ một cách hiệu quả.

4.1.4. Tổng hợp các công nghệ sử dụng tại Backend

Bảng 4.3 dưới đây là bảng tổng hợp các công nghệ then chốt được áp dụng tại tầng Backend nhằm hiện thực hóa hệ thống microservice phân tán:

Bảng 4.3: Tổng hợp các công nghệ sử dụng tại Backend

Thành phần	Công nghệ	Vai trò cốt lõi
Ngôn ngữ chính	Java 21	Nền tảng lập trình chính, hỗ trợ Virtual Threads giúp tối ưu hiệu năng xử lý.
Framework chính	Spring Boot 3.x	Xây dựng, cấu hình và điều phối hoạt động các dịch vụ nghiệp vụ độc lập.
Reactive Framework	Eclipse Vert.x	Xử lý bất đồng bộ và điều phối thông điệp nội bộ dịch vụ qua cơ chế Event Bus.
Truyền tải sự kiện	Apache Kafka (KRaft)	Message Broker hỗ trợ giao tiếp hướng sự kiện và đồng bộ dữ liệu giữa các dịch vụ.
Truy cập dữ liệu	Spring Data JPA, JdbcTemplate	Quản lý thực thể dữ liệu quan hệ và tối ưu hóa các truy vấn truy xuất cơ sở dữ liệu.

4.2 AI Agent

Quá trình lựa chọn được thực hiện dựa trên việc so sánh các giải pháp phổ biến cho từng bài toán, với các tiêu chí chính bao gồm: mức độ phù hợp với yêu cầu hệ thống, khả năng mở rộng, độ phức tạp triển khai và chi phí vận hành.

Các công nghệ chính được sử dụng trong hệ thống bao gồm:

- PostgreSQL / Supabase (cơ sở dữ liệu quan hệ)
- Neo4j / AuraDB (cơ sở dữ liệu đồ thị)
- ChromaDB (cơ sở dữ liệu vector)
- LangGraph (xây dựng AI Agent)
- FastAPI (xây dựng API)
- Railway (triển khai hệ thống)

4.2.1. Cơ sở dữ liệu quan hệ: PostgreSQL / Supabase

Hệ thống cần một cơ sở dữ liệu để lưu trữ dữ liệu sản phẩm có cấu trúc và hỗ trợ các truy vấn lọc theo điều kiện như giá, thương hiệu và thông số kỹ thuật.

Các lựa chọn phổ biến được xem xét bao gồm:

Công nghệ	Ưu điểm	Nhược điểm
MySQL	Phổ biến, dễ sử dụng	Hạn chế extension nâng cao
PostgreSQL	Hỗ trợ mạnh (index, extension, JSON)	Cấu hình phức tạp hơn
MongoDB	Linh hoạt schema	Không tối ưu cho truy vấn quan hệ

PostgreSQL được lựa chọn do khả năng hỗ trợ truy vấn phức tạp, hệ thống index mạnh và khả năng mở rộng thông qua extension. Những đặc điểm này phù hợp với yêu cầu xử lý dữ liệu sản phẩm có nhiều thuộc tính và cần lọc theo nhiều điều kiện.

Trong triển khai thực tế, PostgreSQL được sử dụng thông qua nền tảng Supabase. Việc sử dụng Supabase giúp tận dụng một dịch vụ database được quản lý sẵn, giảm đáng kể chi phí vận hành và cấu hình, đồng thời cung cấp các tính năng hỗ trợ như quản lý dữ liệu và backup.

4.2.2. Cơ sở dữ liệu đồ thị: Neo4j / AuraDB

Đối với các bài toán liên quan đến quan hệ giữa các thực thể, cơ sở dữ liệu đồ thị là lựa chọn phù hợp hơn so với mô hình quan hệ truyền thống.

Các giải pháp được xem xét:

Công nghệ	Ưu điểm	Nhược điểm
PostgreSQL (self-join)	Không cần thêm hệ thống mới	Query phức tạp, kém hiệu quả
Neo4j	Tối ưu cho graph, query trực quan (Cypher)	Cần học thêm ngôn ngữ
ArangoDB	Multi-model	Ít phổ biến hơn

Neo4j được lựa chọn do là hệ quản trị đồ thị phổ biến, hỗ trợ ngôn ngữ truy vấn Cypher trực quan và tối ưu cho các truy vấn quan hệ phức tạp.

Trong triển khai, hệ thống sử dụng Neo4j AuraDB — dịch vụ Neo4j được quản lý trên cloud. Điều này giúp loại bỏ nhu cầu tự vận hành database và đơn giản hóa quá trình triển khai.

4.2.3. Cơ sở dữ liệu vector: ChromaDB

Để hỗ trợ tìm kiếm ngữ nghĩa trong các câu hỏi tự nhiên, hệ thống cần một cơ sở dữ liệu vector có khả năng lưu trữ và truy vấn embedding.

Các lựa chọn phổ biến:

Công nghệ	Ưu điểm	Nhược điểm
FAISS	Hiệu năng cao	Không có persistence mặc định
Pinecone	Managed, dễ dùng	Chi phí cao
ChromaDB	Nhẹ, dễ tích hợp, hỗ trợ local	Ít tính năng nâng cao

ChromaDB được lựa chọn do dễ tích hợp với Python và hỗ trợ lưu trữ cục bộ. Điều này cho phép hệ thống triển khai đơn giản mà không cần thêm một dịch vụ bên ngoài, đồng thời vẫn đáp ứng tốt nhu cầu tìm kiếm ngữ nghĩa với quy mô dữ liệu của đồ án.

4.2.4. Công nghệ xây dựng AI Agent: LangGraph

Hệ thống hội thoại yêu cầu khả năng điều phối giữa mô hình ngôn ngữ và các công cụ truy xuất dữ liệu theo nhiều bước. Do đó, cần một framework hỗ trợ orchestration và quản lý trạng thái.

Các lựa chọn:

Công nghệ	Ưu điểm	Nhược điểm
Prompt chaining (thuần)	Đơn giản	Khó mở rộng, khó kiểm soát
LangChain Agent	Linh hoạt	Khó kiểm soát luồng
LangGraph	Kiểm soát tốt, hỗ trợ stateful	Phức tạp hơn

LangGraph được lựa chọn do cho phép xây dựng AI Agent dưới dạng đồ thị có trạng thái, giúp kiểm soát rõ ràng luồng xử lý và dễ dàng mở rộng.

4.2.5. Công nghệ xây dựng API: FastAPI

Hệ thống cần một framework để xây dựng API có hiệu năng cao và hỗ trợ tốt cho các tác vụ bất đồng bộ.

Các lựa chọn:

Công nghệ	Ưu điểm	Nhược điểm
Flask	Đơn giản, dễ dùng	Không hỗ trợ async tốt
Django	Full-stack, nhiều tính năng	Nặng, dư thừa
FastAPI	Hiệu năng cao, hỗ trợ async, typing	Cộng đồng nhỏ hơn Django

FastAPI được lựa chọn do hỗ trợ bất đồng bộ, hiệu năng cao và tích hợp tốt với Python typing.

4.2.6. Công nghệ triển khai: Railway

Hệ thống cần một nền tảng triển khai đơn giản, hỗ trợ nhanh việc đưa ứng dụng lên môi trường chạy thực tế.

Các lựa chọn:

Công nghệ	Ưu điểm	Nhược điểm
VPS	Toàn quyền kiểm soát	Cần tự cấu hình nhiều
Heroku	Dễ deploy	Hạn chế free tier
Railway	Triển khai nhanh, tích hợp tốt	Ít tùy chỉnh nâng cao

Railway được lựa chọn do khả năng triển khai trực tiếp từ GitHub và cấu hình đơn giản, phù hợp với quy mô hệ thống.

4.3 Hạ tầng và triển khai

Các thành phần hạ tầng như cơ sở dữ liệu, công cụ đóng gói, nền tảng triển khai và hệ thống quản lý phiên bản là những yếu tố cần thiết đối với hệ thống thương mại điện tử bán lẻ điện thoại tích hợp AI Agent của nhóm. Đây là lớp công nghệ bảo đảm dữ liệu được lưu trữ an toàn, môi trường thực thi của các dịch vụ duy trì tính nhất quán, quá trình triển khai diễn ra ổn định và vòng đời phát triển phần mềm được kiểm soát chặt chẽ.

4.3.1. Cơ sở dữ liệu

Cơ sở dữ liệu là thành phần nền tảng có vai trò lưu trữ, quản lý và truy xuất thông tin phục vụ các nghiệp vụ cốt lõi của ứng dụng. Việc triển khai hợp lý giúp đảm bảo tính toàn vẹn, nhất quán và an toàn dữ liệu. Để tìm ra giải pháp phù hợp nhất cho hệ thống, nhóm đã tiến hành nghiên cứu và đánh giá các hệ quản trị cơ sở dữ liệu quan hệ phổ biến hiện nay. Dữ liệu tóm tắt được biểu diễn ở bảng 4.4.

Bảng 4.4: So sánh các hệ quản trị cơ sở dữ liệu

Công nghệ	Ưu điểm	Nhược điểm	Lý do lựa chọn
Oracle Database	- Khả năng xử lý khối lượng dữ liệu khổng lồ với tính bảo mật và độ ổn định gần như tuyệt đối. - Cung cấp các công cụ quản trị chuyên sâu tối ưu cho doanh nghiệp lớn.	- Chi phí bản quyền cực kỳ đắt đỏ và yêu cầu cấu hình phần cứng rất cao. - Quy trình triển khai và bảo trì phức tạp, không linh hoạt cho các dự án vừa và nhỏ.	Không lựa chọn do chi phí vận hành quá lớn và mức độ phức tạp vượt quá nhu cầu thực tế của dự án.
MySQL	- Tốc độ thực thi nhanh đối với các truy vấn đọc và ghi dữ liệu đơn giản. - Cấu hình nhẹ nhàng, dễ triển khai và có cộng đồng hỗ trợ rộng lớn.	- Khả năng xử lý các truy vấn lồng nhau phức tạp và các kiểu dữ liệu hiện đại còn hạn chế. - Chưa đáp ứng tốt các tiêu chuẩn khắt khe về tính toàn vẹn dữ liệu trong các kịch bản phức tạp.	Không chọn do cần một hệ thống mạnh mẽ hơn để quản lý các giao dịch nghiệp vụ và dữ liệu đa dạng.
PostgreSQL	- Tuân thủ chặt chẽ tiêu chuẩn SQL và đặc tính ACID, đảm bảo an toàn giao dịch tuyệt đối. - Tích hợp tốt với các thư viện hiện đại như Spring Data JPA, hỗ trợ quản lý chỉ mục tự động hiệu quả. - Hỗ trợ kiểu dữ liệu JSONB linh hoạt cho dữ liệu phi cấu trúc.	- Mức độ tiêu thụ tài nguyên hệ thống cao hơn so với các giải pháp cơ sở dữ liệu tối giản. - Tốc độ đọc dữ liệu cơ bản đôi khi chậm hơn MySQL trong một số kịch bản truy vấn đơn lẻ.	Được chọn nhờ khả năng xử lý dữ liệu tin cậy, hỗ trợ tốt các dịch vụ trực tuyến miễn phí và tính tương thích cao với hệ sinh thái Spring.

PostgreSQL được nhóm đánh giá là hệ quản trị cơ sở dữ liệu quan hệ mã nguồn mở tiên tiến nhất hiện nay, mang trong mình sức mạnh xử lý tương đương các giải pháp thương mại đắt đỏ. Nền tảng này nổi bật với khả năng tuân thủ chặt chẽ các đặc tính ACID, đảm bảo an toàn tuyệt đối cho các giao dịch quan trọng như quản lý đơn hàng và kho vận. Ngoài ra, PostgreSQL còn cung cấp các tính năng mở rộng mạnh mẽ như kiểu dữ liệu JSONB, tạo điều kiện thuận lợi để hệ thống có thể tích hợp và quản lý các thông số kỹ thuật đa dạng của sản phẩm điện thoại trong các giai đoạn phát triển tiếp theo mà không cần tái cấu trúc toàn bộ cơ sở dữ liệu.

Thay vì phải tự vận hành hạ tầng phức tạp, nhóm có thể dễ dàng tận dụng các nền tảng đám mây hiện đại như Supabase hoặc Neon. Các dịch vụ này cung cấp các gói lưu trữ trực tuyến miễn phí với hiệu suất ổn định và khả năng kết nối linh hoạt, cực kỳ phù hợp với quy mô triển khai và nguồn lực của một đồ án sinh viên. So với sự tốn kém của Oracle hay những hạn chế về tính năng nâng cao của MySQL, PostgreSQL chính là sự lựa chọn tối ưu khi vừa đảm bảo tính kinh tế, vừa đáp ứng trọn vẹn các tiêu chuẩn khắt khe về an toàn dữ liệu và khả năng mở rộng lâu dài cho hệ thống.

4.3.2. Công cụ đóng gói

Công cụ đóng gói đóng vai trò quyết định trong việc chuẩn hóa môi trường thực thi, giúp loại bỏ các sai khác giữa máy tính cá nhân và môi trường triển khai thực tế. Việc container hóa cho phép mỗi dịch vụ hoạt động độc lập, từ đó đơn giản hóa quy trình cập nhật và duy trì tính ổn định của toàn hệ thống.

Docker đóng vai trò là giải pháp container hóa hàng đầu, cho phép đóng gói mỗi microservice cùng với toàn bộ môi trường thực thi của nó vào một container độc lập. Việc sử dụng Docker giúp loại bỏ hoàn toàn các lỗi phát sinh do sự khác biệt về môi trường hệ điều hành hoặc xung đột thư viện giữa các máy tính trong nhóm phát triển.

Tuy việc vận hành container có thể làm tăng nhẹ mức độ tiêu thụ tài nguyên hệ thống, nhưng Docker mang lại khả năng chuẩn hóa môi trường Cloud-Native tuyệt vời cho cả dịch vụ Java và Python. Nhờ vào Docker, nhóm có thể đảm bảo tính nhất quán từ giai đoạn phát triển đến khi triển khai thực tế, đồng thời tạo tiền đề vững chắc cho việc quản lý vòng đời ứng dụng một cách chuyên nghiệp và linh hoạt.

4.3.3. Công cụ quản lý phiên bản

Hệ thống quản lý phiên bản là thành phần không thể thiếu nhằm đảm bảo tính đồng nhất của mã nguồn và hỗ trợ làm việc nhóm một cách có tổ chức. Công cụ này cho phép kiểm soát lịch sử thay đổi, giúp tăng tính minh bạch và an toàn cho tài sản số của dự án.

GitHub là nền tảng quản lý mã nguồn tập trung dựa trên Git, đóng vai trò then chốt trong việc kiểm soát lịch sử thay đổi và hỗ trợ cộng tác giữa các thành viên trong nhóm. Nền tảng này cung cấp các công cụ quản lý quy trình phát triển chuyên nghiệp như Pull Request và Code Review, giúp tăng tính minh bạch và đảm bảo tính toàn vẹn của mã nguồn xuyên suốt dự án.

Tuy đòi hỏi các thành viên phải tuân thủ nghiêm ngặt các quy tắc về phân nhánh và kiểm tra mã nguồn, nhưng GitHub mang lại khả năng phối hợp nhóm cực kỳ hiệu quả. Việc

sử dụng GitHub làm kho lưu trữ tập trung không chỉ giúp bảo vệ tài sản số của dự án mà còn hỗ trợ nhóm theo dõi tiến độ và xử lý các xung đột mã nguồn một cách nhanh chóng, đảm bảo luồng phát triển luôn diễn ra ổn định và chuyên nghiệp.

4.3.4. Tóm tắt công nghệ

Bảng 4.5 dưới đây là bảng tổng hợp các công nghệ then chốt được áp dụng ở hạ tầng nhằm hiện thực hóa hệ thống microservice phân tán:

Bảng 4.5: Tổng hợp các công nghệ sử dụng tại Hạ tầng

Thành phần	Công nghệ	Vai trò cốt lõi
Cơ sở dữ liệu	PostgreSQL	Lưu trữ dữ liệu giao dịch chính, hỗ trợ tối ưu truy vấn cho luồng đọc/ghi.
Containerization	Docker	Chuẩn hóa môi trường Cloud-Native cho các dịch vụ Java và Python.
Quản lý Phiên bản	GitHub	Quản lý mã nguồn tập trung và cộng tác phát triển nhóm.

5 Thiết kế hệ thống

Chương này thiết kế hệ thống dựa trên kiến trúc đã chọn. Các nội dung về thiết kế kiến trúc, cơ sở dữ liệu và giao diện được trình bày chi tiết nhằm tạo cơ sở kỹ thuật để triển khai và vận hành hệ thống.

5.1 Thiết kế kiến trúc

5.1.1. Lựa chọn kiến trúc

Hệ thống thương mại điện tử tích hợp Trợ lý Ảo AI của nhóm yêu cầu một kiến trúc **hiện đại, linh hoạt và có khả năng mở rộng**, đủ để xử lý lượng lớn yêu cầu đồng thời, hỗ trợ phản hồi theo thời gian thực (real-time) và dễ dàng mở rộng theo từng phần.

Từ những phân tích về nhu cầu kỹ thuật và khả năng phát triển lâu dài, kiến trúc hệ thống cần đáp ứng các mục tiêu sau:

- Mở rộng linh hoạt cho từng dịch vụ độc lập.
- Hỗ trợ xử lý bất đồng bộ và realtime.
- Đảm bảo tính cô lập giữa các thành phần.
- Cho phép triển khai, cập nhật, bảo trì riêng từng dịch vụ mà không ảnh hưởng toàn hệ thống.
- Duy trì độ tin cậy cao, tránh tình trạng “sập dây chuyền” khi một dịch vụ gặp lỗi.
- Tận dụng hạ tầng Cloud để tự động hóa triển khai, cân bằng tải và tối ưu chi phí.
- Hỗ trợ tích hợp Trợ lý Ảo một cách dễ dàng.

Sau khi tìm hiểu, nghiên cứu về cơ sở lý thuyết của các mô hình kiến trúc phổ biến như *Monolithic*, *Microservices* và *EDA + Cloud-Native*, nhóm lựa chọn kiến trúc **EDA + Cloud-Native** vì những lý do sau:

- Phù hợp với hệ thống có nhiều sự kiện phát sinh và cần xử lý bất đồng bộ như hệ thống thương mại điện tử.
- Đảm bảo khả năng phản hồi nhanh và độ trễ thấp.
- Dễ dàng mở rộng hoặc thêm mới dịch vụ mà không ảnh hưởng hệ thống hiện hữu.
- Giúp giảm phụ thuộc giữa các thành phần và tăng tính mở rộng.
- Cho phép tích hợp AI Agent hoạt động với các nền tảng vector database và graph database một cách dễ dàng.

5.1.2. Các lựa chọn thiết kế trong kiến trúc EDA + Cloud-Native

Các mô hình thiết kế (pattern) trong kiến trúc EDA + Cloud-Native là những mô hình được thiết kế để giúp các dịch vụ giao tiếp và phản ứng với nhau thông qua các sự kiện một cách bất đồng bộ, hiệu quả và an toàn. Cách tiếp cận này giúp xây dựng hệ thống có khả năng mở rộng cao, độ chịu lỗi tốt và mức độ phụ thuộc thấp giữa các thành phần. Những mô hình này mô tả cách sự kiện được tạo ra, truyền đi và tiêu thụ trong môi trường phân tán, qua đó đáp ứng các yêu cầu về khả năng mở rộng theo tải, độ tin cậy, và tính phản hồi của hệ thống.

Các mô hình tiêu biểu gồm Publish–Subscribe, Message Queue, Event Sourcing, CQRS, Saga và Choreography[3], mỗi mô hình giải quyết một nhóm vấn đề đặc thù trong quá trình thiết kế và vận hành hệ thống theo kiến trúc EDA + Cloud-Native.

Về mặt luân chuyển sự kiện

Các mô hình luân chuyển sự kiện (Event-Delivery Patterns) mô tả cách sự kiện được truyền từ dịch vụ phát sinh (producer) đến dịch vụ xử lý (consumer). Hai mô hình phổ biến nhất là Publish–Subscribe (Pub/Sub) và Message Queue (MQ). Các cơ chế này giúp hệ thống duy trì sự tách biệt giữa các dịch vụ, đảm bảo độ tin cậy và khả năng mở rộng khi vận hành trên hạ tầng phân tán.

Publish – Subscribe (Pub/Sub) là mô hình truyền sự kiện trong đó producer gửi sự kiện lên một kênh (topic), và nhiều consumer có thể đồng thời nhận được sự kiện này. Mô hình này giúp hệ thống linh hoạt hơn và dễ mở rộng. Mô hình này có những đặc điểm sau:

- Producer không cần biết số lượng hay danh tính consumer[3].
- Một sự kiện có thể kích hoạt nhiều dịch vụ cùng lúc[3].
- Mô hình topic hỗ trợ mở rộng theo chiều ngang.
- Thích hợp với xử lý song song nhiều luồng.

Pub/Sub được ứng dụng trong các hệ thống cần phân phối sự kiện đến nhiều dịch vụ cùng lúc, đặc biệt khi nhiều thành phần trong kiến trúc phải phản ứng đồng thời với các thay đổi. Mô hình này hỗ trợ mở rộng linh hoạt, giảm phụ thuộc giữa các dịch vụ và cho phép xử lý song song ở quy mô lớn, nhờ đó phù hợp với những luồng nghiệp vụ đòi hỏi khả năng phản ứng tức thời và lan tỏa.

Message Queue là mô hình hàng đợi giúp đảm bảo mỗi thông điệp chỉ được xử lý bởi một consumer, phù hợp với mô hình truyền sự kiện điểm–đến–điểm (point-to-point). Đặc điểm của mô hình này là:

- Mỗi message được một consumer duy nhất xử lý[3].
- Không mất message nhờ cơ chế lưu trữ tạm.
- Hỗ trợ retry khi lỗi xảy ra.

- Giúp phân phối tải giữa nhiều worker.

Message Queue được sử dụng trong các nghiệp vụ yêu cầu xử lý tuần tự, đảm bảo tính tin cậy và mức độ cam kết cao trong việc tiêu thụ sự kiện. Mô hình này giúp duy trì sự ổn định của hệ thống bằng cách kiểm soát tải, đảm bảo mỗi sự kiện được xử lý trọn vẹn, đồng thời hạn chế rủi ro mất mát dữ liệu khi dịch vụ gặp sự cố hoặc gián đoạn.

Sau khi phân tích hai mô hình truyền sự kiện phổ biến là Publish/Subscribe và Message Queue, nhóm quyết định lựa chọn Pub/Sub làm mô hình luân chuyển sự kiện cho hệ thống. Việc lựa chọn này xuất phát từ đặc thù của hệ thống thương mại điện tử, nơi mỗi sự kiện thường cần được nhiều dịch vụ cùng lúc lắng nghe và xử lý. Ví dụ, khi một đơn hàng được tạo ra (order_created), Inventory Service cần cập nhật tồn kho, Payment Service tiếp nhận để xử lý thanh toán, Notification Service gửi thông báo cho người dùng, và Analytics Service ghi nhận dữ liệu phục vụ thống kê. Với Pub/Sub, dịch vụ phát sinh sự kiện không cần biết có bao nhiêu dịch vụ quan tâm, và mỗi consumer có thể xử lý độc lập mà không ảnh hưởng đến phần còn lại của hệ thống.

Mô hình Pub/Sub giúp giảm mạnh sự phụ thuộc giữa các dịch vụ và tạo điều kiện cho việc mở rộng hệ thống về sau. Khi bổ sung dịch vụ mới, chỉ cần đăng ký (subscribe) vào topic sự kiện mà không cần thay đổi bất kỳ thành phần đang hoạt động. Điều này phù hợp với tư duy mở rộng linh hoạt của kiến trúc Cloud-Native và giúp hệ thống vận hành bất đồng bộ hiệu quả. Vì các lợi ích trên, nhóm thống nhất lựa chọn Pub/Sub làm mô hình truyền sự kiện duy nhất cho toàn bộ hệ thống.

Về mặt quản lý trạng thái dữ liệu

Trong môi trường phân tán, quản lý trạng thái dữ liệu đóng vai trò then chốt. Do đó, để hạn chế sự bất đồng bộ trong trạng thái dữ liệu giữa các dịch vụ, chúng ta cần thiết kế hệ thống theo các mô hình quản lý trạng thái (State Management Patterns) hiện đại và phù hợp. Hai mô hình thông dụng là Event Sourcing và CQRS, mỗi mô hình phù hợp cho các loại nghiệp vụ khác nhau.

Event Sourcing lưu toàn bộ thay đổi dưới dạng chuỗi sự kiện thay vì trạng thái cuối cùng. Đặc điểm của mô hình này là:

- Lưu lịch sử dữ liệu đầy đủ[3].
- Khôi phục trạng thái bằng replay sự kiện.
- Hỗ trợ kiểm toán và truy vết mạnh[3].
- Yêu cầu thiết kế và lưu trữ phức tạp hơn[3].

Event Sourcing thích hợp cho các hệ thống cần duy trì lịch sử thay đổi chi tiết, hỗ trợ kiểm toán hoặc phục hồi trạng thái với độ chính xác cao. Nhờ lưu lại toàn bộ chuỗi sự kiện, mô hình này cho phép tái xây dựng dữ liệu, phân tích hành vi theo thời gian và đáp ứng các yêu cầu về minh bạch, nhất quán và khả năng truy vết trong môi trường phân tán.

CQRS (Command Query Responsibility Segregation) tách biệt luồng ghi và luồng đọc, giúp tối ưu từng thành phần riêng biệt. Mô hình này có những đặc điểm sau:

- Tách biệt đường đi của đọc và ghi[3].
- Cho phép tối ưu từng luồng theo nhu cầu[3].
- Hỗ trợ mở rộng độc lập.
- Tăng cường bảo mật nhờ phân tách chức năng.

CQRS phù hợp với những hệ thống có tải truy vấn lớn, nơi nhu cầu đọc dữ liệu vượt trội so với ghi. Việc tách biệt hai luồng xử lý giúp tối ưu hiệu năng, cải thiện tốc độ phản hồi và cho phép mở rộng độc lập từng phần của hệ thống, đồng thời tăng cường khả năng kiểm soát truy cập và tính linh hoạt trong thiết kế dữ liệu.

Trong hai mô hình quản lý trạng thái là Event Sourcing và CQRS, nhóm lựa chọn CQRS làm mô hình quản lý trạng thái dữ liệu cho hệ thống. Lý do là hệ thống thương mại điện tử thường có tỷ lệ truy vấn đọc lớn hơn rất nhiều so với tỷ lệ ghi. Người dùng liên tục xem sản phẩm, xem giỏ hàng, xem lịch sử đơn hàng và tra cứu thông tin chi tiết. Các thao tác này yêu cầu phản hồi nhanh, ổn định và có khả năng xử lý tải cao. CQRS cho phép tách phần ghi (command) và phần đọc (query) thành hai thành phần độc lập, giúp tối ưu từng luồng theo đúng đặc thù của nó.

Với CQRS, phần ghi tập trung đảm bảo tính toàn vẹn dữ liệu và xử lý đúng logic nghiệp vụ, trong khi phần đọc có thể được tối ưu bằng cache, denormalized view hoặc cơ sở dữ liệu chuyên biệt cho truy vấn nhanh. Điều này làm giảm tải cho hệ thống ghi và cải thiện tốc độ truy vấn lên giao diện người dùng.

Event Sourcing mặc dù mạnh về khả năng lưu vết, nhưng lại phức tạp trong thiết kế và không phù hợp với đề tài của nhóm. Do đó, nhóm chọn CQRS làm mô hình quản lý trạng thái duy nhất để đảm bảo cân bằng giữa hiệu năng, sự đơn giản và khả năng triển khai thực tế.

Về mặt điều phối hệ thống

Các mô hình điều phối (Orchestration Patterns) mô tả cách các dịch vụ phối hợp để xử lý quy trình nghiệp vụ phức tạp. Hai mô hình phổ biến là Saga và Choreography.

Saga chia tác vụ lớn thành nhiều bước nhỏ, mỗi bước có thể rollback khi lỗi xảy ra. Mô hình này có những đặc điểm sau:

- Chia tác vụ thành nhiều bước tuần tự.
- Có hành động để xử lý lỗi nếu có ở từng bước[3].
- Có trung tâm điều phối (orchestrator) để tổ chức thực hiện tác vụ[3].
- Hỗ trợ xử lý lỗi và rollback có kiểm soát.

Saga được áp dụng trong các quy trình nghiệp vụ phức tạp đòi hỏi nhiều bước xử lý có liên quan và cần duy trì tính nhất quán trong môi trường phân tán. Mô hình này hỗ trợ điều phối luồng công việc theo từng giai đoạn, đảm bảo hệ thống có thể xử lý lỗi an toàn, khôi phục từng phần khi cần thiết và duy trì tính toàn vẹn của các giao dịch dài.

Choreography không dùng orchestrator; các dịch vụ tự lắng nghe sự kiện và thực thi tương ứng. Đặc điểm của mô hình này là:

- Không có bộ điều phối trung tâm[3].
- Mở rộng tự nhiên theo kiến trúc phân tán.
- Khó kiểm soát khi quy trình phức tạp[3].

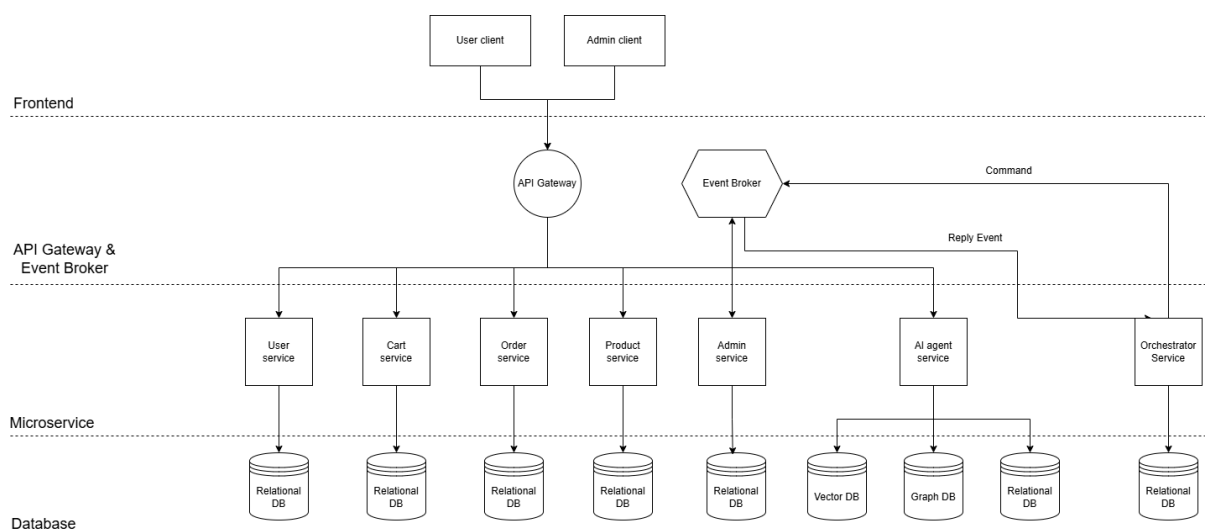
Choreography phù hợp với các quy trình nhẹ, mang tính linh hoạt cao và không yêu cầu sự điều phối tập trung. Mỗi dịch vụ phản ứng độc lập theo sự kiện, giúp hệ thống giảm bớt ràng buộc, dễ mở rộng và hoạt động theo hướng phi tập trung. Mô hình này đặc biệt hữu ích trong những luồng nghiệp vụ đơn giản hoặc các tác vụ hỗ trợ trong kiến trúc sự kiện.

Trong nhóm các mô hình điều phối nghiệp vụ, sau khi xem xét Saga và Choreography, nhóm quyết định lựa chọn Saga làm mô hình điều phối hệ thống. Quy trình đặt hàng trong thương mại điện tử là quy trình phức tạp, bao gồm nhiều bước phụ thuộc lẫn nhau như kiểm tra tồn kho, xử lý thanh toán, tạo đơn hàng, cập nhật trạng thái và gửi thông báo. Mỗi bước đều có khả năng phát sinh lỗi và yêu cầu cơ chế rollback phù hợp để đảm bảo tính nhất quán của toàn hệ thống.

Saga Orchestration có một orchestrator trung tâm chịu trách nhiệm điều phối từng bước của quy trình, theo dõi trạng thái và quyết định thực hiện hành động bù trừ (compensation) khi xảy ra lỗi. Mô hình này giúp kiểm soát luồng nghiệp vụ rõ ràng, dễ quan sát, dễ debug và phù hợp với yêu cầu thực tế của đề tài. So với Choreography, Saga tránh được tình trạng “event chaos” khi có nhiều dịch vụ phản ứng lẫn nhau, đồng thời mang lại sự minh bạch và khả năng kiểm soát cao hơn.

Vì các lý do trên, nhóm lựa chọn Saga Orchestration làm mô hình điều phối cho toàn bộ các quy trình nghiệp vụ quan trọng trong hệ thống, đặc biệt là quy trình đặt hàng.

5.1.3. Các tầng của hệ thống



Hình 5.1: Sơ đồ kiến trúc hệ thống

Hình 5.1 bên trên biểu diễn kiến trúc tổng thể của hệ thống của nhóm. Sơ đồ 5.1 cho thấy hệ thống của nhóm được tổ chức bao gồm bốn tầng chính:

- Frontend
- API Gateway & Event Broker
- Microservice
- Database

Mỗi tầng đảm nhiệm một vai trò riêng biệt trong việc xử lý yêu cầu, điều phối hoạt động và lưu trữ dữ liệu.

Tầng Frontend

Tầng Frontend đóng vai trò là điểm tương tác đầu tiên giữa người dùng với hệ thống. Tầng này bao gồm hai thành phần chính: Giao diện dành cho khách hàng (User client) và giao diện dành cho quản trị viên của hệ thống (Admin client). User client là nơi diễn ra toàn bộ các hoạt động của khách hàng như xem sản phẩm, thêm vào giỏ hàng, đặt hàng hoặc theo dõi tình trạng đơn hàng. Trong khi đó, Admin client phục vụ quản trị viên trong việc quản lý danh mục sản phẩm, kiểm soát đơn hàng, điều chỉnh thông tin và xử lý các thao tác vận hành nội bộ.

Cả hai giao diện đều giao tiếp với các phần khác của hệ thống thông qua một thành phần duy nhất là API Gateway, đảm bảo toàn bộ yêu cầu được thống nhất về đường đi và cách xử lý.

Tầng API Gateway và Event Broker

Tầng này bao gồm hai thành phần chính: API Gateway và Event Broker, mỗi thành phần đảm nhiệm một vai trò khác nhau trong cơ chế vận hành của hệ thống.

API Gateway hoạt động như một lớp trung gian giữa Frontend và Microservices. Đây là nơi tiếp nhận tất cả các yêu cầu từ User client và Admin client, sau đó định tuyến chúng đến đúng microservice phía dưới. Việc đặt API Gateway giữa giao diện và các service giúp kiểm soát luồng dữ liệu, hợp nhất các điểm truy cập và tạo điều kiện thuận lợi cho việc áp dụng các chính sách hệ thống trong tương lai như phân quyền, kiểm soát tốc độ và ghi log.

Song song với API Gateway, hệ thống còn sử dụng một Event Broker. Event Broker đảm nhiệm việc truyền tải và phân phối các sự kiện nội bộ giữa các service theo mô hình Publish/Subscribe. Điều này cho phép các microservice xử lý bất đồng bộ, tránh việc phải gọi trực tiếp lẫn nhau và giảm thiểu sự phụ thuộc giữa các module.

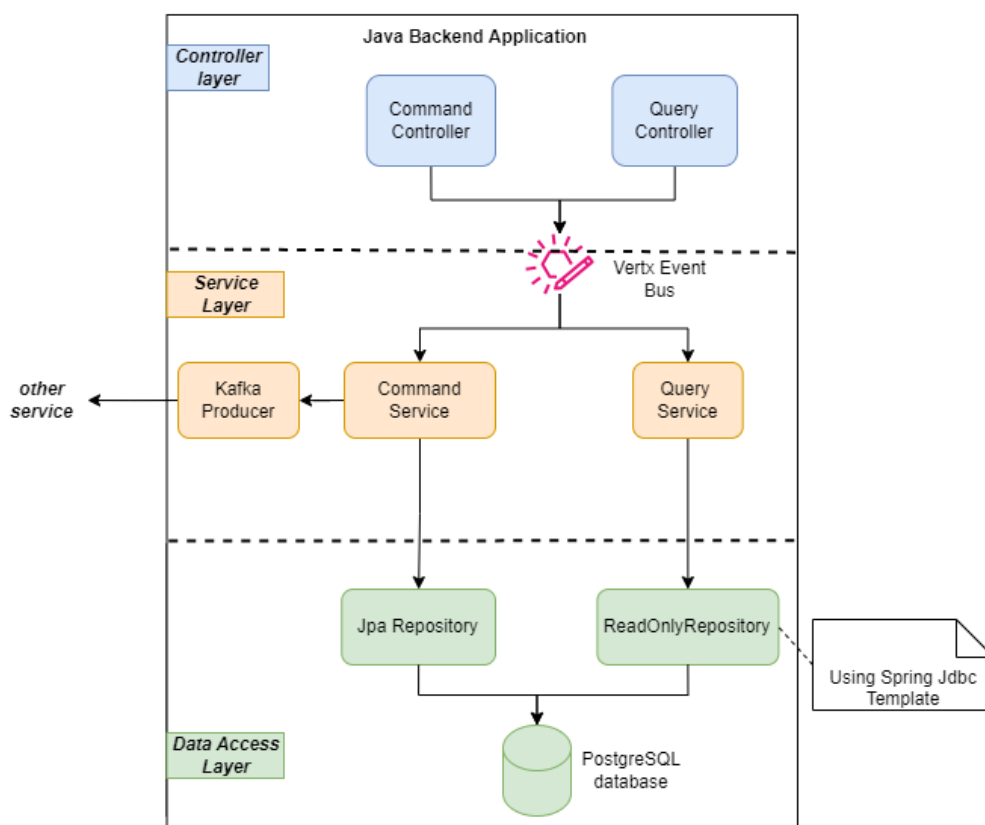
Tầng Microservice

Tầng này bao gồm bảy dịch vụ độc lập theo chức năng, trong đó có một service điều phối điều phối Saga để duy trì tính nhất quán dữ liệu trong các giao dịch phân tán, năm service nghiệp vụ thương mại điện tử và một service chuyên biệt về AI được thể hiện rõ trên sơ đồ:

- User service: Quản lý thông tin người dùng và các thao tác liên quan đến tài khoản.
- Cart service: Xử lý toàn bộ hoạt động liên quan đến giỏ hàng của người dùng.
- Order service: Tiếp nhận và xử lý yêu cầu liên quan đến đặt hàng và lưu đơn hàng.
- Product service: Lưu trữ và cung cấp thông tin về sản phẩm cho hệ thống.
- Admin service: Hỗ trợ các chức năng quản trị.
- AI agent service: Thực hiện các hoạt động của Trợ lý Ảo như tư vấn sản phẩm, kiểm tra thông tin, ...
- Orchestrator Service: Thực hiện mô hình Saga Orchestration, chuyên trách điều phối các quy trình nghiệp vụ phức tạp bằng cách quản lý Trạng thái Saga, gửi Commands và xử lý Reply Events qua Event Broker. Dịch vụ này có Database riêng để lưu trữ Metadata Điều phối.

Các microservice không có bất kỳ liên kết trực tiếp nào với nhau trong sơ đồ, thay vào đó toàn bộ giao tiếp đều phải thông qua Event Broker. Điều này giúp cho mỗi service được vận hành tách biệt, giúp giảm thiểu tình trạng xung đột logic, tăng tính chủ động khi triển khai và bảo trì, và bên cạnh đó còn giúp hỗ trợ nhóm phân chia công việc dễ dàng hơn trong giai đoạn hiện thực hệ thống.

Ngoài ra, về mặt cấu trúc nội bộ, mỗi Microservice nghiệp vụ được nhóm tổ chức theo mô hình phân tầng cơ bản.



Hình 5.2: Cấu trúc nội bộ Microservice của các dịch vụ Java

Sơ đồ 5.2 mô tả cấu trúc nội bộ của các dịch vụ nghiệp vụ trong hệ thống. Nhóm sử dụng Vert.x Event Bus làm bộ điều phối nội bộ để chuyển giao các yêu cầu một cách bất đồng bộ. Luồng ghi dùng JPA Repository để quản lý các thực thể phức tạp và đảm bảo tính nhất quán dữ liệu trong cơ sở dữ liệu PostgreSQL. Ngược lại, luồng đọc sử dụng một kho lưu trữ riêng với Spring JdbcTemplate để thực hiện các câu lệnh SQL thuần, giúp loại bỏ độ trễ của các bộ thư viện ánh xạ thực thể và tăng tốc độ phản hồi. Thêm vào đó, Kafka Producer ở luồng ghi sẽ phát đi các sự kiện nghiệp vụ. Đây là mắt xích quan trọng để kích hoạt các quy trình liên dịch vụ như SAGA, đảm bảo tính toàn vẹn dữ liệu cho toàn bộ hệ thống phân tán.

Tầng Database

Tầng Database chịu trách nhiệm lưu trữ dữ liệu cho từng microservice. Mỗi microservice có cơ sở dữ liệu riêng của nó:

- User service → Lưu trữ toàn bộ thông tin người dùng và dữ liệu tài khoản liên quan.
- Cart service → Lưu dữ liệu giỏ hàng của từng người dùng, bao gồm danh sách và số lượng sản phẩm.
- Order service → Lưu thông tin đơn hàng, trạng thái xử lý và các dữ liệu liên quan đến quá trình đặt hàng.
- Product service → Lưu trữ danh sách sản phẩm, thuộc tính và thông tin mô tả sản phẩm.

- Admin service → Lưu dữ liệu phục vụ thao tác quản trị và điều chỉnh thông tin hệ thống.
- AI agent service → Lưu trữ các vector embedding dùng cho xử lý truy vấn và gợi ý của Trợ lý Ảo. Bên cạnh đó, lưu các quan hệ dạng đồ thị giữa sản phẩm, hành vi và dữ liệu cần phân tích cho Trợ lý Ảo.
- Orchestrator service → Lưu trữ metadata điều phối Saga, bao gồm trạng thái hiện tại của các Saga đang chạy, thông tin về các bước đã hoàn thành và các dữ liệu cần thiết để quản lý quy trình nghiệp vụ phân tán.

Các service chính như User, Cart, Order, Product và Admin đều sử dụng cơ sở dữ liệu quan hệ, đảm bảo lưu trữ dữ liệu theo cấu trúc bảng truyền thống.

Riêng AI agent service sở hữu hai cơ sở dữ liệu độc lập: cơ sở dữ liệu Vector và cơ sở dữ liệu Graph, tương ứng với hai loại dữ liệu mà service này cần quản lý. Điều này cho phép dịch vụ Trợ lý Ảo vận hành trên những tập dữ liệu phù hợp với nhu cầu xử lý của nó.

5.2 Thiết kế cơ sở dữ liệu

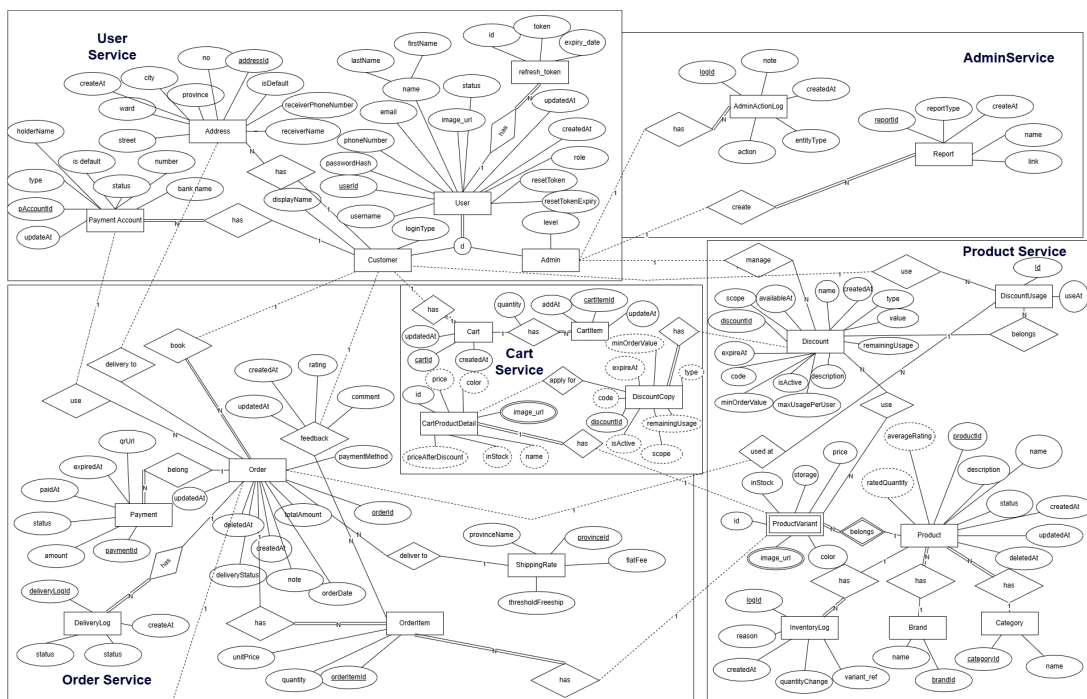
Trong kiến trúc microservice, mỗi dịch vụ quản lý một cơ sở dữ liệu độc lập theo đúng ranh giới domain. Vì vậy, mô hình dữ liệu không sử dụng quá nhiều ràng buộc khóa ngoại như kiến trúc nguyên khối. Thay vào đó, các quan hệ giữa dữ liệu được thể hiện ở mức logic hoặc thông qua các cơ chế đồng bộ sự kiện giữa các dịch vụ.

Mô hình ERD ban đầu cho các service nghiệp vụ được nhóm xây dựng dựa trên kiến trúc nguyên khối tổng thể, sau đó được phân tách và nhóm các thực thể cùng quan hệ tương ứng vào các cơ sở dữ liệu riêng biệt dựa trên ranh giới của từng Microservice.

Bên cạnh các dịch vụ nghiệp vụ chính, hệ thống còn có orchestrator service, chịu trách nhiệm điều phối các quy trình phân tán theo mô hình saga. Do vai trò đặc thù của nó, ERD của orchestrator service không mô tả các thực thể nghiệp vụ như các service thương mại điện tử khác, mà tập trung vào việc lưu vết trạng thái của từng saga, từng bước xử lý và các sự kiện cần gửi đi thông qua Kafka. Các thực thể như saga_instance, saga_step, outbox_event và compensation_action được tổ chức nhằm hỗ trợ điều phối, theo dõi và khôi phục giao dịch phân tán một cách tin cậy.

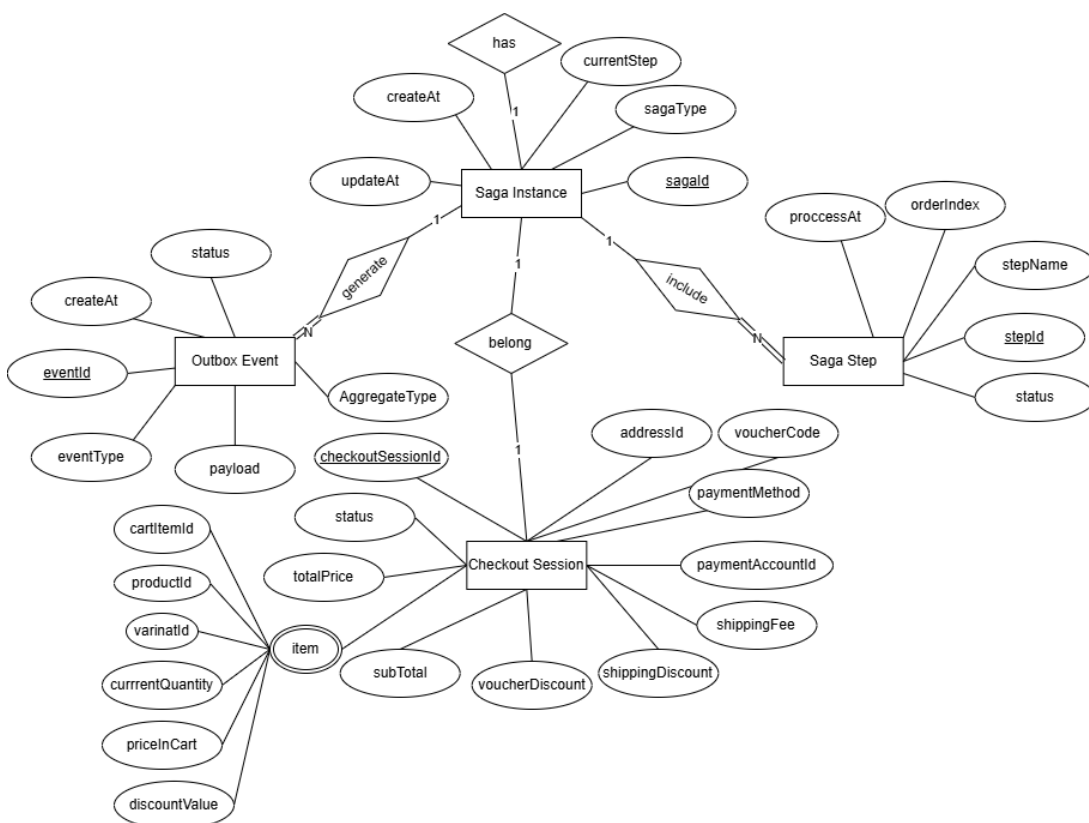
Hình 5.3 chính là sơ đồ ERD mô tả các thực thể và quan hệ giữa chúng trong từng service của hệ thống. Sơ đồ ERD tổng quan mô tả các thực thể chính của từng domain trong hệ thống. Mỗi nhóm thực thể được tổ chức tách biệt theo chức năng. Các mối quan hệ giữa chúng chỉ mang tính chất tham chiếu logic nhằm mô tả dòng dữ liệu, không phải là ràng buộc vật lý trong cơ sở dữ liệu.

Sơ đồ mapping của các ERD sẽ được miêu tả cụ thể ở phần phụ lục



Hình 5.3: Sơ đồ ERD

Sơ đồ ERD của orchestrator service được trình bày bên dưới ở hình 5.4 nhằm làm rõ cách hệ thống triển khai Saga orchestration và cách các thành phần này phối hợp để đảm bảo tính nhất quán xuyên suốt nhiều microservice.



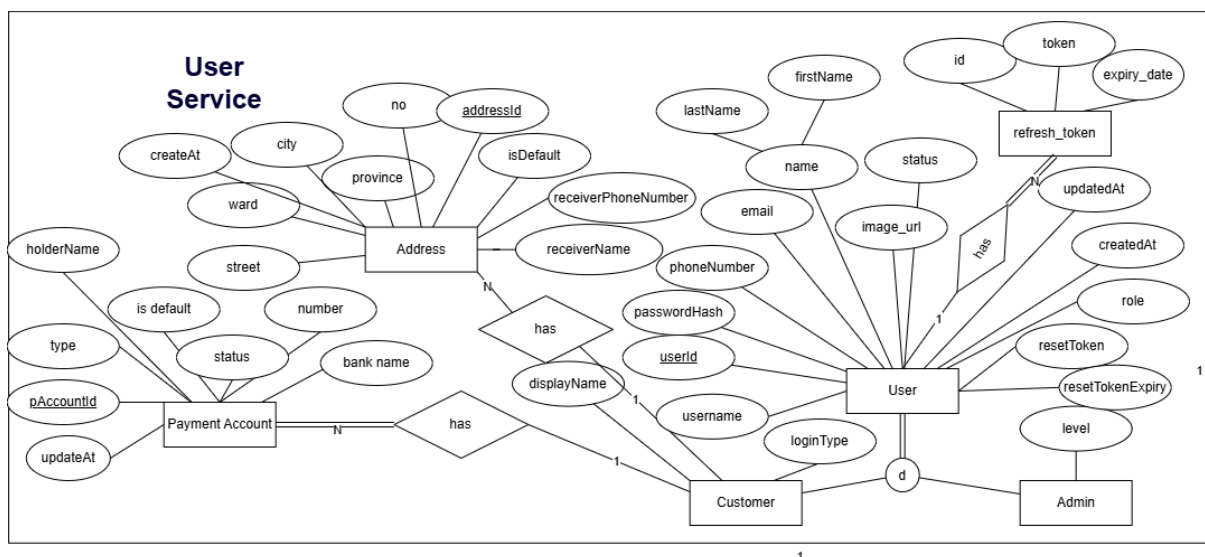
Hình 5.4: ERD của orchestrator service

5.2.1. ERD chi tiết cho user service

Hình 5.5 mô tả cấu trúc dữ liệu của user service, đơn vị quản lý danh tính và thông tin giao dịch trong hệ thống.

User Service là domain chịu trách nhiệm quản lý danh tính và thông tin giao dịch cá nhân của tất cả người dùng trong hệ thống. Thực thể User đóng vai trò là cốt lõi, lưu trữ các thông tin định danh và có quan hệ disjoint để phân loại rõ ràng giữa Customer và Admin, thể hiện sự phân quyền trong hệ thống. Để hỗ trợ bảo mật, thực thể Refresh Token có quan hệ 1:N với user giúp quản lý phiên đăng nhập lâu dài thông qua mã token và thời gian hết hạn.

Thực thể Customer mở rộng vai trò của người dùng thông thường và là trung tâm của các quan hệ giao dịch cá nhân. Mỗi Khách hàng có quan hệ 1:N với thực thể Address, cho phép lưu trữ nhiều địa chỉ giao hàng khác nhau. Đồng thời, Khách hàng cũng có quan hệ 1:N với thực thể Payment Account, quản lý các phương thức thanh toán đã liên kết.



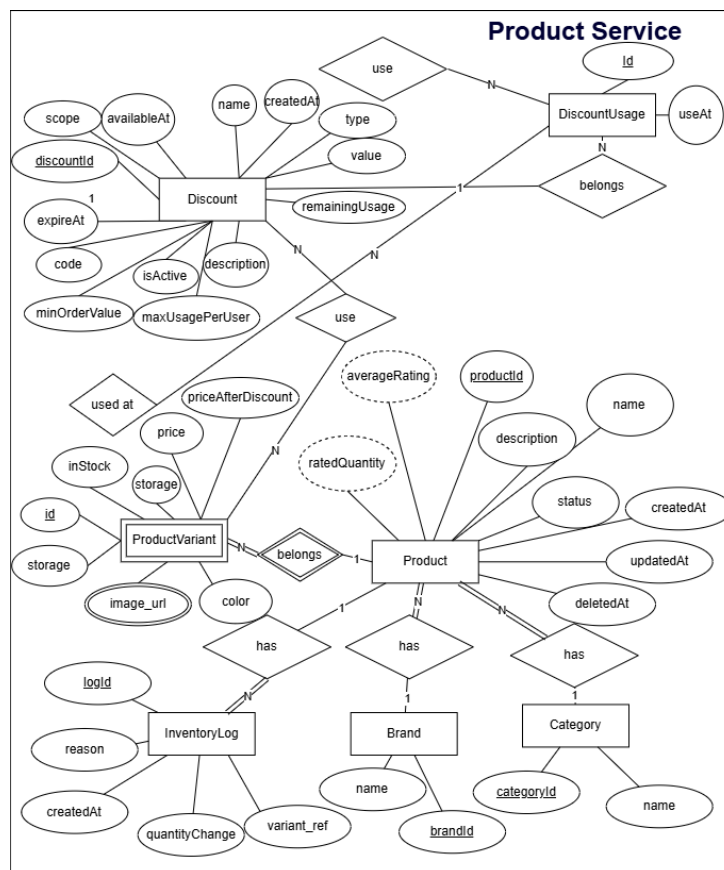
Hình 5.5: ERD chi tiết cho user service

User service độc lập và chỉ quản lý các thực thể thuộc phạm vi người dùng. Mọi service khác trong kiến trúc Microservice, như Order Service, sẽ chỉ tham chiếu đến dữ liệu này bằng ID (ví dụ: userId, addressId) mà không cần lưu trữ lại thông tin chi tiết.

5.2.2. ERD chi tiết cho product service

Hình 5.6 mô tả cấu trúc dữ liệu của Product Service, domain cốt lõi quản lý danh mục sản phẩm, tồn kho và các chính sách khuyến mãi.

Thực thể Product đóng vai trò trung tâm, liên kết với Brand và Category qua các quan hệ N-1 để phân loại sản phẩm. Để tối ưu việc quản lý thuộc tính như màu sắc hay dung lượng, hệ thống sử dụng thực thể yếu ProductVariant có quan hệ N:1 với Product, giúp theo dõi chi tiết tồn kho và hình ảnh riêng biệt cho từng phiên bản. Mọi biến động hàng hóa đều được lưu vết qua thực thể InventoryLog.



Hình 5.6: ERD chi tiết của product service

Hệ thống quản lý khuyến mãi thông qua thực thể Discount và DiscountUsage. Trong đó, DiscountUsage đóng vai trò quan trọng trong việc thiết lập các quan hệ logic liên dịch vụ: kết nối với thực thể Customer thuộc user service qua quan hệ use và thực thể Order thuộc order service qua quan hệ used at. Điều này giúp kiểm soát chặt chẽ việc áp dụng mã giảm giá cho từng khách hàng trên mỗi đơn hàng cụ thể.

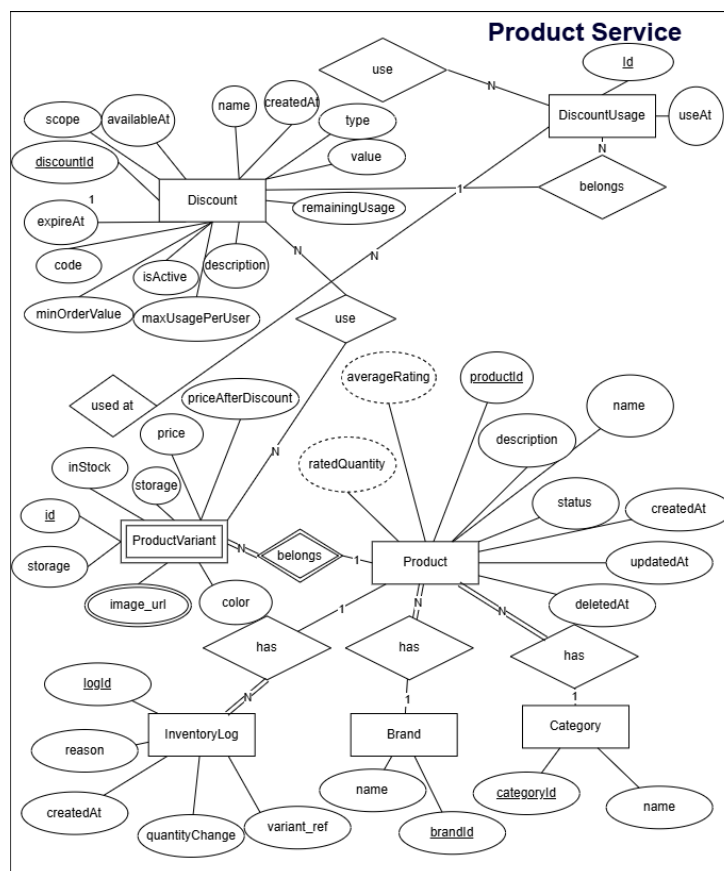
5.2.3. ERD chi tiết cho cart service

Cart service được thiết kế tối giản nhằm quản lý trạng thái giỏ hàng tạm thời với hiệu suất cao. Thiết kế dữ liệu tập trung vào thực thể Cart, đại diện cho giỏ hàng duy nhất của mỗi khách hàng.

Thực thể Cart có quan hệ 1:1 với thực thể Customer thuộc user service thông qua quan hệ has để xác định chủ sở hữu. Thực thể CartItem lưu trữ danh sách mặt hàng khách hàng đã chọn kèm số lượng. Để tối ưu tốc độ truy xuất và giảm tải cho việc gọi API liên dịch vụ, Cart Service lưu trữ các bản sao dữ liệu được đồng bộ từ product service thông qua Apache Kafka. Thay vì thiết lập quan hệ trực tiếp với thực thể Product ở dịch vụ khác, CartItem liên kết với thực thể bản sao CartProductDetail. Việc này nhằm đảm bảo thông tin sản phẩm luôn sẵn dùng ngay cả khi product service gặp sự cố, đồng thời loại bỏ độ trễ mạng khi phải truy vấn liên dịch vụ để hiển thị danh sách giỏ hàng.

- CartProductDetail: Bản sao tổng hợp từ Product và Product Variant giúp hiển thị giỏ hàng tức thời.

- Discount: Bản sao thông tin khuyến mãi giúp tính toán tổng tiền trực tiếp tại dịch vụ mà không cần gọi API sang Product Service.



Hình 5.7: ERD chi tiết của cart service

5.2.4. ERD chi tiết cho order service

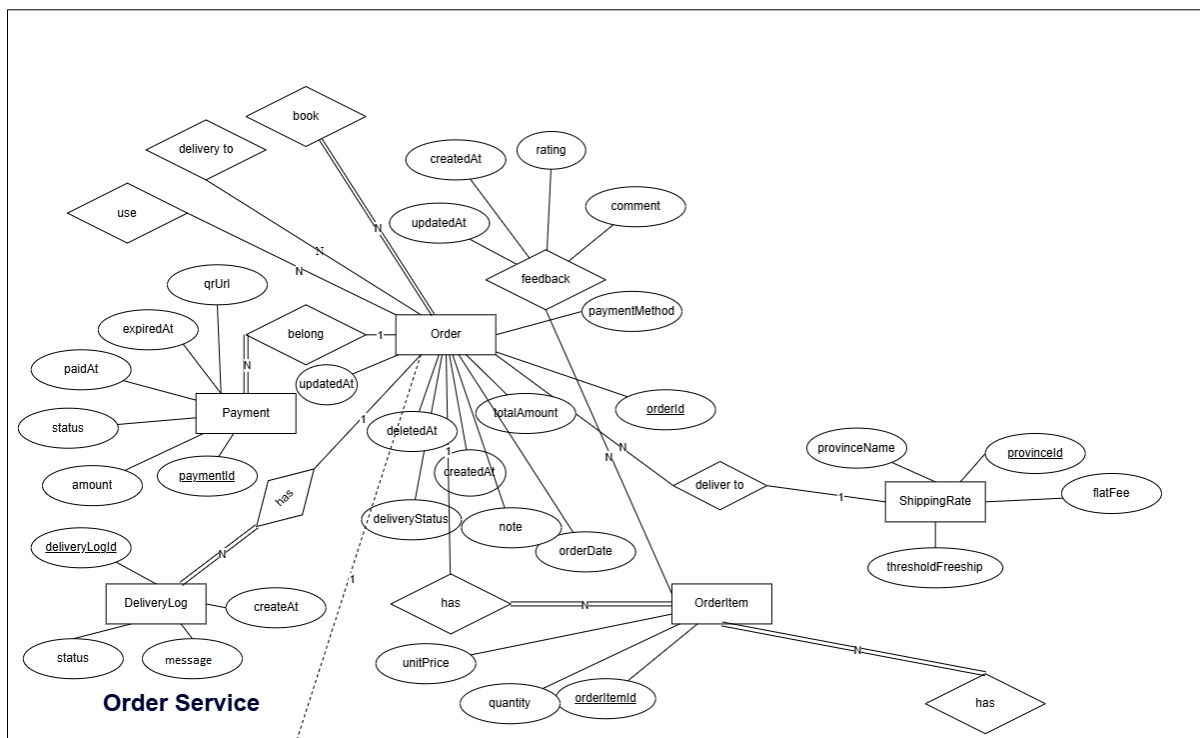
Order Service chịu trách nhiệm quản lý toàn bộ vòng đời giao dịch từ đặt hàng, thanh toán đến hậu mãi. Thực thể Order đóng vai trò trung tâm, lưu trữ thông tin tổng quan và trạng thái đơn hàng.

Hệ thống quản lý chi tiết đơn hàng thông qua thực thể OrderItem có quan hệ 1:N với Order. Để theo dõi quá trình xử lý, mỗi đơn hàng liên kết với thực thể Payment qua quan hệ 1-1 để quản lý giao dịch tiền tệ và quan hệ 1-N với DeliveryLog nhằm ghi vết lịch sử vận chuyển. Ngoài ra, thực thể ShippingRate được sử dụng để xác định phí giao hàng dựa trên từng khu vực cụ thể.

Hình 5.8 minh họa các quan hệ ngoại lai logic để đảm bảo tính độc lập giữa các dịch vụ:

- book / feedback: Tham chiếu đến Customer ở user service để xác định chủ đơn hàng và người thực hiện đánh giá.
- deliver to: Liên kết với Address ở user service và ShippingRate để xác định điểm đến và chi phí vận chuyển.

- use: Tham chiếu đến Payment Account cũng ở user service để xác định nguồn thanh toán.
- belong: Liên kết giữa Payment và Order để đối soát giao dịch.
- has: Tham chiếu từ OrderItem đến Product thuộc product service để xác định mặt hàng.



Hình 5.8: ERD chi tiết của order service

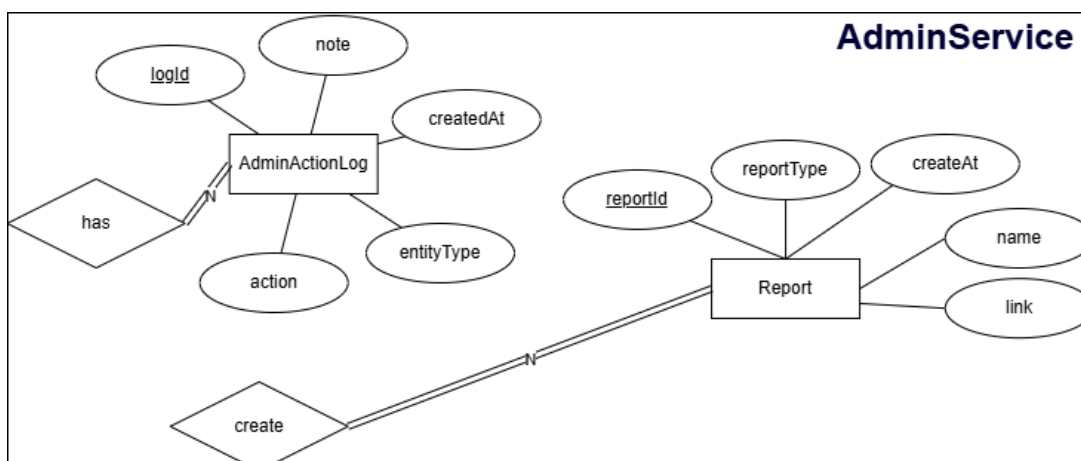
Việc chỉ lưu trữ các ID ngoại lai mà không có ràng buộc vật lý giúp order service duy trì tính độc lập cao, phù hợp với nguyên tắc thiết kế Database per Service trong kiến trúc Microservices.

5.2.5. ERD chi tiết cho admin service

Admin Service đóng vai trò là domain phụ trợ, chịu trách nhiệm duy trì tính minh bạch qua việc ghi vết hành động quản trị và quản lý các báo cáo hệ thống. Thực thể AdminActionLog đóng vai trò lưu trữ nhật ký thao tác chi tiết của quản trị viên, ghi nhận cụ thể loại hành động, thực thể bị tác động cùng các ghi chú liên quan để phục vụ mục đích kiểm soát tính minh bạch. Bên cạnh đó, thực thể Report chịu trách nhiệm quản lý danh mục các báo cáo thống kê đã được kết xuất. Thay vì lưu trữ trực tiếp dữ liệu thô gây tốn kém tài nguyên, thực thể này chỉ lưu giữ thông tin về loại báo cáo và đường dẫn liên kết đến tệp kết quả hoàn chỉnh.

Hình 5.9 mô tả các mối liên kết ngoại lai nhằm duy trì tính nhất quán dữ liệu giữa các dịch vụ. Cụ thể, quan hệ has từ thực thể AdminActionLog và quan hệ create từ thực thể Report đều tham chiếu trực tiếp đến thực thể Admin thuộc User Service. Việc thiết lập

các liên kết logic này giúp hệ thống định danh chính xác người chịu trách nhiệm cho từng thao tác quản trị cũng như xác định tác giả của các báo cáo thống kê được tạo lập.



Hình 5.9: ERD chi tiết của admin service

5.3 Thiết kế API

5.3.1. Mục tiêu của thiết kế API

Mục tiêu chung của việc thiết kế API là cung cấp điểm truy cập thống nhất cho ứng dụng frontend và đảm bảo tính nhất quán cho các giao dịch phân tán nội bộ.

Hệ thống triển khai hai loại API chính dựa trên mục đích sử dụng:

1. External API (Frontend-Facing): Là các API công khai, phục vụ giao tiếp trực tiếp với ứng dụng frontend. Hầu hết các API này là điểm khởi đầu cho các luồng xử lý command hoặc kích hoạt một giao dịch phân tán.
2. Internal API (Backend/Saga Orchestration): Là các API nội bộ, được gọi đồng bộ bởi orchestrator service để thực hiện các bước tuần tự của một giao dịch phân tán phức tạp. Các API này đảm bảo tính nhất quán giữa nhiều microservice theo mô hình Saga Orchestration.

5.3.2. Nguyên tắc thiết kế API

Để đảm bảo một hệ thống API đồng bộ, dễ hiểu, dễ mở rộng và an toàn, việc tuân thủ một bộ các nguyên tắc thiết kế thống nhất là điều tối quan trọng. Các nguyên tắc dưới đây tập trung vào việc áp dụng mô hình Resource-Oriented và chuẩn mực RESTful cùng với các biện pháp bảo mật và quản lý phiên bản hiệu quả đối với các External API.

- Theo mô hình Resource-Oriented (dựa trên tài nguyên: users, products, carts, orders...).
- RESTful: sử dụng GET/POST/PUT/DELETE.
- Áp dụng DTO cho mọi request/response, hạn chế lộ entity gốc.
- Sử dụng token-based authentication cho những API yêu cầu xác thực.

5.3.3. Cấu trúc mẫu của Request/Response trong API

Để đảm bảo tính thống nhất giữa các microservice và giảm thiểu trùng lặp trong quá trình thiết kế API, hệ thống chuẩn hóa toàn bộ phản hồi và truy vấn phân trang thông qua ba lớp cơ sở: `BaseResponse`, `BasePaginationResponse`, và `BasePaginationRequest`. Các thành phần này đóng vai trò làm khuôn mẫu chung được sử dụng xuyên suốt tất cả service nghiệp vụ.

a/ `BaseResponse<T>` – Cấu trúc phản hồi chuẩn

`BaseResponse` được sử dụng cho tất cả API không yêu cầu phân trang. Đây là cấu trúc phản hồi đơn giản nhưng thống nhất, giúp frontend xử lý kết quả và lỗi một cách dễ dàng.

Các trường thông tin:

- `status (int)`: Mã trạng thái HTTP hoặc mã nội bộ xác định kết quả xử lý, ví dụ: 200, 400, 404,...
- `data (T)`: Payload chứa dữ liệu cần trả về cho frontend, có thể là DTO hoặc bất kỳ kiểu dữ liệu nào.
- `message (String)`: Thông tin mô tả kết quả xử lý, ví dụ: "Success", "Invalid credentials".

Việc áp dụng các nguyên tắc này có ý nghĩa quan trọng trong việc chuẩn hóa giao diện API giữa các service, đảm bảo tính đồng nhất và khả năng tích hợp dễ dàng. Sự chuẩn hóa này cũng trực tiếp hỗ trợ logging và tracking dễ dàng hơn, giúp hệ thống theo dõi và xử lý lỗi hiệu quả. Hơn nữa, nó còn giúp giảm thiểu số lượng schema response lặp lại, tối ưu hóa mã nguồn và đơn giản hóa việc phát triển cho các ứng dụng client.

`BasePaginationResponse<T>` – Cấu trúc phản hồi phân trang

`BasePaginationResponse` được sử dụng cho các API trả về danh sách có phân trang (ví dụ: danh sách sản phẩm, danh sách đơn hàng, danh sách người dùng admin).

Lớp này mở rộng cấu trúc `BaseResponse` bằng cách bổ sung các thông tin phân trang cần thiết.

Các trường thông tin:

- `status (int)`: Mã trạng thái HTTP hoặc mã nội bộ xác định kết quả xử lý, ví dụ: 200, 400, 404,...
- `page_number (int)`: Số trang hiện tại (zero-based).
- `page_size (int)`: Số lượng item trên mỗi trang.
- `count_items (long)`: Tổng số item lấy được từ truy vấn.
- `count_pages (int)`: Tổng số trang có thể hiển thị.

- data (T): Payload chứa dữ liệu cần trả về cho frontend, có thể là DTO hoặc bất kỳ kiểu dữ liệu nào.
- message (String): Thông tin mô tả kết quả xử lý, ví dụ: "Success", "Invalid credentials".

Việc thống nhất cơ chế phân trang mang lại ý nghĩa then chốt là chuẩn hóa mọi API phân trang trên các service, tạo ra một giao diện đồng nhất. Điều này không chỉ giúp frontend hiển thị giao diện phân trang dễ dàng hơn nhờ vào cấu trúc response đã được định trước, mà còn hỗ trợ backend sử dụng các công cụ như Spring Data xây dựng phân trang hiệu quả và nhất quán xuyên suốt hệ thống.

BasePaginationRequest – Chuẩn hóa Request phân trang

BasePaginationRequest là lớp DTO dùng cho toàn bộ API cần phân trang. Lớp này hỗ trợ Spring Data trực tiếp thông qua hàm toPageable(), giúp code service gọn và tránh lặp lại logic phân trang.

Các trường thông tin:

- page (int): Chỉ số trang (bắt đầu từ 0). Có validate là không được nhỏ hơn 0.
- size (int): Số lượng phần tử mỗi trang. Có validate là không được nhỏ hơn 1.
- sortBy (String): Tên trường cần sắp xếp.
- sortDirection (String): Hướng sắp xếp: "ASC" hoặc "DESC" (mặc định "DESC").

Việc chuẩn hóa cách thức backend nhận yêu cầu phân trang giúp tăng tính nhất quán giữa các API khác nhau, đảm bảo mọi dịch vụ đều xử lý yêu cầu phân trang theo cùng một quy tắc. Điều này cho phép tái sử dụng logic phân trang đã được định nghĩa trong hầu hết mọi service, từ đó tiết kiệm thời gian phát triển và giảm thiểu lỗi.

5.3.4. User Service

Nhằm đảm bảo tính súc tích cho báo cáo, nhóm chỉ tập trung trình bày các nhóm API đại diện cho các luồng nghiệp vụ cốt lõi của User Service như xác thực, quản lý hồ sơ và các thông tin giao dịch cá nhân. Chi tiết về các Endpoint quan trọng này được tổng hợp tại Bảng 5.1.

Bảng 5.1: Các API quan trọng của User Service

Nhóm chức năng	Method	Đường dẫn	Chức năng
Xác thực	POST	/api/auth/login/local	Đăng nhập hệ thống.
	POST	/api/auth/register/local	Đăng ký tài khoản mới.
Tài khoản	GET	/api/users/profile/me	Lấy thông tin cá nhân.
	PUT	/api/users/change-password/me	Đổi mật khẩu.
Địa chỉ	POST	/api/addresses	Thêm địa chỉ mới.
	GET	/api/addresses/default	Lấy địa chỉ mặc định.
Thanh toán	POST	/api/payment-accounts	Liên kết tài khoản.

5.3.5. Product Service

Dịch vụ sản phẩm chịu trách nhiệm cung cấp thông tin danh mục cho khách hàng và các công cụ quản lý nội dung cho quản trị viên. Dưới đây là các API đặc trưng phục vụ luồng nghiệp vụ chính như tìm kiếm sản phẩm và quản lý chương trình khuyến mãi, chi tiết được tổng hợp tại Bảng 5.2.

Bảng 5.2: Các API quan trọng của Product Service

Nhóm chức năng	Method	Đường dẫn	Chức năng
Khách hàng	GET	/api/products/paged	Liệt kê sản phẩm phân trang.
	GET	/api/products/search-filter	Tìm kiếm sản phẩm nâng cao.
Quản trị sản phẩm	POST	/api/products/admin	Tạo sản phẩm mới.
	POST	/api/products/admin/id/variant	Thêm biến thể cho sản phẩm.
Khuyến mãi	POST	/api/discounts/from-code	Kiểm tra khuyến mãi từ mã code.
	POST	/api/discounts/admin	Thiết lập chương trình ưu đãi mới.

5.3.6. Cart Service

Dịch vụ giỏ hàng chịu trách nhiệm quản lý các mặt hàng tạm thời của khách hàng trước khi tiến hành đặt hàng. Do tính chất tập trung vào hiệu năng xử lý các thao tác thêm, sửa, xóa thường xuyên, hệ thống cung cấp đầy đủ các điểm cuối để quản lý trạng thái giỏ hàng và tính toán giá trị trực tiếp. Toàn bộ danh mục API của dịch vụ được chi tiết tại Bảng 5.3.

Bảng 5.3: Danh mục API của Cart Service

Nhóm chức năng	Method	Đường dẫn	Chức năng
Giỏ hàng (Cart)	GET	/api/carts	Lấy thông tin giỏ hàng hiện tại.
	POST	/api/carts/add-to-cart	Thêm sản phẩm vào giỏ.
Truy vấn	GET	/api/carts/with-items-paging	Lấy danh sách item phân trang.
	POST	/api/cart-items/by-ids	Truy xuất items theo danh sách ID.
Điều chỉnh	PATCH	/api/cart-items/increase/id	Tăng số lượng sản phẩm.
	PATCH	/api/cart-items/decrease/id	Giảm số lượng sản phẩm.
	DELETE	/api/cart-items/delete/id	Xóa sản phẩm khỏi giỏ hàng.
Tính toán	POST	/api/carts/price	Tính toán tổng giá trị giỏ hàng.

5.3.7. Orchestration Service

Dịch vụ điều phối chịu trách nhiệm quản lý tính nhất quán của dữ liệu thông qua mô hình Saga. Đây là trung tâm điều khiển các giao dịch phân tán phức tạp như đặt hàng, thanh toán và xử lý hoàn tác khi có sự cố. Các API đại diện cho luồng nghiệp vụ này được tổng hợp tại Bảng 5.4.

Bảng 5.4: Các API quan trọng của Orchestration Service

Nhóm chức năng	Method	Đường dẫn	Chức năng
Checkout	POST	/api/checkout/preview/init	Khởi tạo phiên kiểm tra đơn hàng.
	POST	/api/checkout/confirm	Xác nhận đặt hàng và kích hoạt Saga.
Thanh toán	POST	/api/payments	Xử lý yêu cầu thanh toán đơn hàng.
	POST	/api/payments/simulate	Giả lập môi trường thanh toán thử nghiệm.
Saga và Order	POST	/api/sagas/cancel-order	Hủy đơn hàng và kích hoạt bù trừ (Compensating).
	GET	/api/sagas/id	Kiểm tra trạng thái tiến trình Saga.

5.3.8. Order Service

Dịch vụ đơn hàng chịu trách nhiệm lưu trữ thông tin giao dịch cuối cùng, quản lý trạng thái vận chuyển và ghi nhận phản hồi từ khách hàng. Để đảm bảo tính minh bạch trong quá trình vận hành, dịch vụ cung cấp hệ thống nhật ký giao hàng chi tiết cho từng giai đoạn của đơn hàng. Các API đại diện được trình bày tại Bảng 5.5.

Bảng 5.5: Các API quan trọng của Order Service

Nhóm chức năng	Method	Đường dẫn	Chức năng
Truy vấn đơn	GET	/api/orders/id	Xem chi tiết đơn hàng cá nhân.
	GET	/api/orders/by-delivery-status	Lọc đơn theo trạng thái vận chuyển.
Vận chuyển	POST	/api/orders/admin/id/...	Tạo nhật ký giao hàng mới (Admin).
	GET	/api/delivery-logs/latest/id	Lấy trạng thái vận chuyển mới nhất.
Feedback	POST	/api/orders/feedback	Gửi đánh giá cho sản phẩm.
	GET	/api/orders/feedback/product/id	Xem các đánh giá của sản phẩm.

5.3.9. Admin Service

Dịch vụ quản trị cung cấp các công cụ hỗ trợ theo dõi biến động hệ thống và kết xuất dữ liệu thống kê. Điểm đặc biệt của dịch vụ này là khả năng lưu vết chi tiết mọi hành động của người quản lý thông qua hệ thống Action Logs, đảm bảo tính minh bạch và an toàn dữ liệu. Danh mục toàn bộ API của dịch vụ được trình bày tại Bảng 5.6.

Bảng 5.6: Danh mục API của Admin Service

Nhóm chức năng	Method	Đường dẫn	Chức năng
Tạo báo cáo	POST	/api/admins/reports/top-selling-products	Thông kê sản phẩm bán chạy.
	POST	/api/admins/reports/monthly-revenue	Thông kê doanh thu định kỳ.
Truy vấn báo cáo	GET	/api/admins/reports	Danh sách các tệp báo cáo.
	GET	/api/admins/reports/id	Xem thông tin báo cáo chi tiết.
Nhật ký thao tác	GET	/api/admins/action-logs/filter	Lọc nhật ký thao tác nâng cao.
	GET	/api/admins/action-logs/by-id/id	Xem chi tiết một bản ghi log.

5.4 Thiết kế các topic Kafka cho hệ thống

Trong kiến trúc Microservices, việc đảm bảo tính nhất quán dữ liệu và phối hợp các giao dịch phân tán là một thách thức lớn. Hệ thống sử dụng Apache Kafka làm nền tảng trung gian để thực hiện các giao tiếp bất đồng bộ, giúp giảm thiểu sự phụ thuộc trực tiếp giữa các dịch vụ. Các Topic được thiết kế nhằm phục vụ hai mục đích chính: điều phối quy trình nghiệp vụ thông qua mô hình Saga và đồng bộ hóa dữ liệu bản sao giữa các dịch vụ.

5.4.1. Nhóm Topic đồng bộ hóa dữ liệu

Đối với danh mục sản phẩm và chương trình khuyến mãi, hệ thống sử dụng một topic duy nhất cho mỗi thực thể. Trong thông điệp gửi đi sẽ bao gồm trường UpdateType (nhận các giá trị: CREATE, UPDATE, DELETE) để dịch vụ tiêu thụ xác định hành động cần thực hiện.

- `discount.copy.command`: Cập nhật thông tin khuyến mãi sang các dịch vụ liên quan để tính toán giá.
- `product.detail.command`: Đồng bộ chi tiết sản phẩm khi có thay đổi về mô tả hoặc trạng thái.

Riêng với dữ liệu đánh giá và thao tác giải phóng sản phẩm, hệ thống tách nhỏ thành các topic riêng biệt dựa trên hành động của người dùng. Cách tiếp cận này giúp product service dễ dàng phân loại logic xử lý ngay từ tầng Listener mà không cần kiểm tra trường UpdateType bên trong code:

- `create.feedback.events`: Kích hoạt khi người dùng tạo đánh giá mới.
- `update.feedback.events`: Cập nhật số sao đánh giá trung bình và số lượng đánh giá ở sản phẩm.
- `delete.feedback.events`: Gỡ bỏ đánh giá và cập nhật lại điểm trung bình.
- `product.release.commands`: Giải phóng sản phẩm khi người dùng chọn hủy đơn hàng.

5.4.2. Nhóm Topic điều phối giao dịch Saga

Hệ thống áp dụng mô hình Saga Orchestration để quản lý các giao dịch phân tán. Quy trình dựa trên việc gửi lệnh và nhận sự kiện giữa Orchestrator và các dịch vụ thành phần.

A. Luồng đặt hàng

- `order.commands`: Lệnh yêu cầu tạo hoặc cập nhật trạng thái đơn hàng gửi đến order service.
- `order.events`: Phản hồi về kết quả xử lý bản ghi đơn hàng từ order service.
- `product.commands`: Lệnh yêu cầu giữ chỗ hoặc giảm số lượng tồn kho gửi đến product service.
- `product.events`: Phản hồi xác nhận đã trừ kho thành công hoặc thông báo hết hàng.
- `cart.commands`: Lệnh yêu cầu xóa các item đã thanh toán khỏi giỏ hàng gửi đến cart service.
- `cart.events`: Phản hồi xác nhận giỏ hàng đã được làm sạch thành công.

B. Luồng thanh toán

- `payment.commands`: Lệnh khởi tạo giao dịch thanh toán gửi đến order service.
- `payment.events`: Phản hồi khi giao dịch thanh toán được xử lý thành công.
- `payment.failed.events`: Phản hồi khi giao dịch thanh toán thất bại.
- `payment.confirm.commands`: Lệnh xác nhận hoàn tất giao dịch sau khi các dịch vụ khác đã sẵn sàng.
- `payment.confirm.events`: Phản hồi xác nhận giao dịch đã được chốt thành công.
- `payment.cancel.commands`: Lệnh hủy hoặc hoàn tác giao dịch thanh toán khi chuỗi Saga gặp lỗi.
- `payment.cancel.events`: Phản hồi xác nhận việc hủy thanh toán đã hoàn tất.

C. Luồng hủy đơn hàng

- `cancel.order.commands`: Lệnh hủy đơn hàng gửi đến order service.
- `cancel.order.success.events`: Phản hồi khi cập nhật trạng thái hủy thành công.
- `cancel.order.failed.events`: Phản hồi khi quy trình hủy gặp lỗi nghiệp vụ.
- `refund.success.events`: Thông báo hoàn tiền thành công cho khách hàng.
- `refund.failed.events`: Thông báo lỗi trong quá trình hoàn trả tiền.

5.5 Thiết kế AI Agent

5.5.1. Thiết kế cơ sở dữ liệu

Hệ thống sử dụng kiến trúc đa cơ sở dữ liệu (polyglot persistence) nhằm đáp ứng các yêu cầu khác nhau của bài toán tư vấn hội thoại. Thay vì cố gắng giải quyết mọi nhu cầu bằng một loại cơ sở dữ liệu duy nhất, mỗi loại dữ liệu được lưu trữ trong một hệ quản trị phù hợp với đặc trưng của nó.

Cụ thể, hệ thống bao gồm ba thành phần chính:

- Cơ sở dữ liệu quan hệ (PostgreSQL) cho dữ liệu có cấu trúc
- Cơ sở dữ liệu đồ thị (Neo4j) cho dữ liệu quan hệ phức tạp và hành vi người dùng
- Cơ sở dữ liệu vector (ChromaDB) cho tìm kiếm ngữ nghĩa

Thiết kế cơ sở dữ liệu quan hệ PostgreSQL

PostgreSQL được sử dụng để lưu trữ dữ liệu sản phẩm dưới dạng có cấu trúc, phục vụ các truy vấn lọc theo điều kiện như danh mục, giá và thông số kỹ thuật. Mô hình dữ liệu được tổ chức thành ba bảng chính:

- Bảng product lưu thông tin chung của sản phẩm như tên, danh mục, thương hiệu, dòng sản phẩm và đánh giá.
- Bảng variant biểu diễn các biến thể của sản phẩm (màu sắc, giá, hình ảnh)
- Bảng product_spec lưu thông số kỹ thuật theo mô hình EAV (Entity–Attribute–Value).

Việc lựa chọn mô hình EAV thay vì schema cố định xuất phát từ đặc thù dữ liệu: mỗi danh mục sản phẩm có tập thông số khác nhau (ví dụ laptop có RAM, trong khi máy in có tốc độ in). Nếu sử dụng schema cố định, số lượng cột sẽ rất lớn và phần lớn giá trị bị bỏ trống. EAV cho phép lưu trữ linh hoạt các thuộc tính mà không cần thay đổi schema.

Tuy nhiên, mô hình này cũng có nhược điểm là làm tăng độ phức tạp truy vấn. Để truy vấn theo thông số kỹ thuật, hệ thống phải sử dụng các điều kiện EXISTS hoặc JOIN nhiều lần, dẫn đến chi phí tính toán cao hơn so với schema cố định.

Để giảm thiểu vấn đề này, hệ thống áp dụng các chiến lược tối ưu:

- Tạo index tổng hợp trên (category, brand) để tăng tốc lọc cơ bản
- Tạo index riêng cho các trường thường truy vấn như price và color
- Tạo index trên (spec_key, value_num) để hỗ trợ lọc theo thông số kỹ thuật

Ngoài ra, extension `pg_trgm` được sử dụng để hỗ trợ tìm kiếm mờ theo tên sản phẩm. Điều này cho phép hệ thống xử lý các truy vấn không chính xác hoàn toàn, đặc biệt hữu ích trong bước định danh sản phẩm từ ngôn ngữ tự nhiên.

Thiết kế cơ sở dữ liệu đồ thị Neo4j

Neo4j được sử dụng để lưu trữ các mối quan hệ giữa các thực thể, đặc biệt là quan hệ phân cấp sản phẩm và hành vi mua hàng của người dùng. Khác với mô hình quan hệ, cơ sở dữ liệu đồ thị cho phép biểu diễn trực tiếp các liên kết giữa các nút, giúp việc truy vấn các quan hệ phức tạp trở nên tự nhiên hơn.

Trong hệ thống, các nút chính bao gồm:

- Product
- Variant
- Brand
- Series
- Category
- Customer

Các quan hệ như `HAS_VARIANT`, `OF_BRAND`, `IN_SERIES` và `IN_CATEGORY` mô tả cấu trúc phân cấp của sản phẩm, trong khi quan hệ `BOUGHT` biểu diễn hành vi mua hàng của người dùng.

Việc sử dụng Neo4j mang lại lợi thế rõ rệt trong các truy vấn liên quan đến hành vi, chẳng hạn như xác định thương hiệu hoặc màu sắc được người dùng mua nhiều nhất. Nếu thực hiện truy vấn tương tự trong cơ sở dữ liệu quan hệ, hệ thống sẽ cần nhiều phép `JOIN` phức tạp, trong khi Neo4j cho phép biểu diễn trực tiếp bằng `pattern matching`.

Tuy nhiên, Neo4j không được sử dụng cho các truy vấn lọc lớn theo điều kiện (ví dụ tìm tất cả sản phẩm $\text{RAM} \geq 8\text{GB}$), vì các truy vấn dạng này phù hợp hơn với cơ sở dữ liệu quan hệ. Do đó, Neo4j được giới hạn trong vai trò xử lý quan hệ và hành vi, thay vì thay thế hoàn toàn PostgreSQL.

Thiết kế cơ sở dữ liệu vector ChromaDB

ChromaDB được sử dụng để hỗ trợ tìm kiếm ngữ nghĩa trên dữ liệu văn bản, bao gồm mô tả sản phẩm, câu hỏi thường gặp (FAQ) và các chính sách của cửa hàng. Khác với truy vấn SQL truyền thống, tìm kiếm ngữ nghĩa cho phép hệ thống xử lý các câu hỏi tự nhiên như “máy này có pin dùng được lâu không” mà không cần khớp từ khóa chính xác.

Dữ liệu được lưu trữ dưới dạng các đoạn văn bản (chunks), trong đó mỗi đoạn được ánh xạ thành một vector embedding. Hai collection chính được sử dụng:

- Collection products: chứa mô tả và các cặp FAQ của sản phẩm

- Collection policies: chứa các đoạn văn bản chính sách

Việc tách riêng hai collection giúp kiểm soát phạm vi tìm kiếm và cải thiện độ chính xác của kết quả. Ví dụ, các câu hỏi về bảo hành sẽ chỉ truy vấn trong collection policies thay vì toàn bộ dữ liệu.

Hệ thống sử dụng mô hình embedding tiếng Việt để đảm bảo chất lượng biểu diễn ngữ nghĩa. Các vector được chuẩn hóa để đảm bảo tính nhất quán khi so sánh khoảng cách.

Tuy nhiên, tìm kiếm vector có chi phí tính toán cao hơn so với truy vấn SQL, và kết quả mang tính xấp xỉ. Do đó, ChromaDB chỉ được sử dụng cho các câu hỏi mang tính ngữ nghĩa, không thay thế cho các truy vấn chính xác.

Do hệ thống sử dụng nhiều cơ sở dữ liệu, việc đảm bảo tính nhất quán dữ liệu là một vấn đề quan trọng. Trong quá trình xây dựng dữ liệu, mỗi sản phẩm được gán một định danh duy nhất (UUID) và được sử dụng xuyên suốt trong PostgreSQL và Neo4j. Quá trình ghi dữ liệu được thực hiện theo thứ tự ưu tiên: trước hết ghi vào PostgreSQL, sau đó mới ghi vào Neo4j. Nếu bước ghi vào PostgreSQL thất bại, toàn bộ giao dịch bị rollback và không tiếp tục ghi sang các hệ thống khác. Cách tiếp cận này đảm bảo rằng dữ liệu quan hệ luôn là nguồn chính xác.

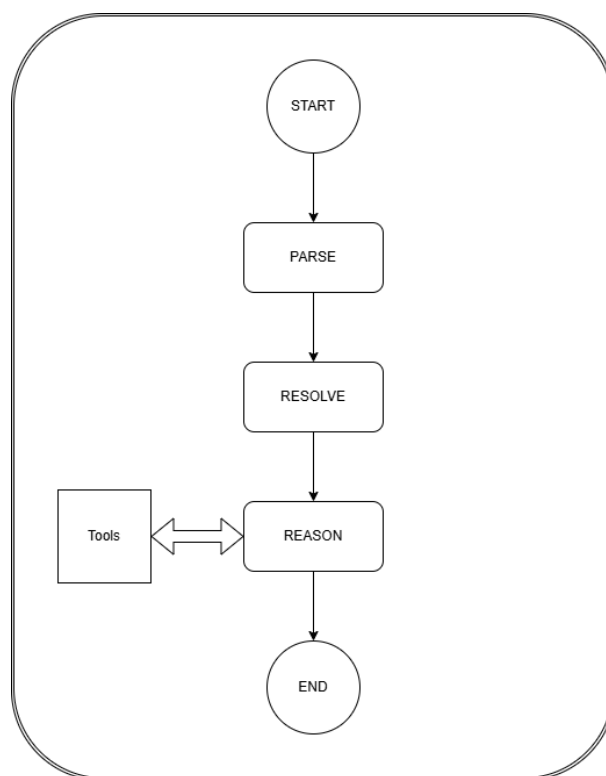
Việc sử dụng kiến trúc đa cơ sở dữ liệu giúp hệ thống tận dụng được thế mạnh của từng công nghệ, nhưng đồng thời làm tăng độ phức tạp tổng thể. Thay vì một hệ thống đơn giản, hệ thống cần quản lý ba loại database với các mô hình dữ liệu khác nhau.

PostgreSQL đảm bảo tính nhất quán và hỗ trợ truy vấn chính xác, nhưng kém linh hoạt với dữ liệu quan hệ phức tạp. Neo4j xử lý tốt các quan hệ và hành vi, nhưng không phù hợp cho truy vấn lọc lớn. ChromaDB hỗ trợ tìm kiếm ngữ nghĩa, nhưng kết quả mang tính xấp xỉ và phụ thuộc vào chất lượng embedding.

5.5.2. Thiết kế AI Agent

AI Agent được xây dựng nhằm thực hiện tư vấn sản phẩm theo ngữ cảnh hội thoại, với khả năng hiểu ý định người dùng, duy trì trạng thái qua nhiều lượt trao đổi và truy xuất dữ liệu từ nhiều nguồn khác nhau. Bài toán này có đặc trưng là luồng xử lý không tuyến tính: người dùng có thể liên tục thay đổi yêu cầu hoặc chuyển ngữ cảnh giữa các lượt hội thoại. Do đó, hệ thống không được thiết kế theo pipeline tuần tự mà sử dụng mô hình đồ thị xử lý (graph-based execution).

Luồng xử lý tổng quát của agent như sau:



Hình 5.10: Sơ đồ kiến trúc AI Agent

Trong đó, Parse thực hiện trích xuất thông tin có cấu trúc, Resolve định danh sản phẩm, Reason đóng vai trò điều phối quyết định và Tools đảm nhiệm truy xuất dữ liệu. Thiết kế này cho phép tách biệt rõ ba lớp: hiểu ngôn ngữ (LLM), điều phối logic (graph execution) và truy xuất dữ liệu (database), từ đó giảm sự phụ thuộc chéo và giúp hệ thống dễ mở rộng.

Thiết kế trạng thái hội thoại

Trạng thái hội thoại (State) là thành phần trung tâm, đóng vai trò như bộ nhớ làm việc của agent. Có thể xem trạng thái này như một “incremental query builder”, trong đó mỗi lượt hội thoại bổ sung thêm ràng buộc cho truy vấn cuối cùng.

Trạng thái bao gồm các nhóm thông tin chính:

- Thông tin định danh của khách hàng (nếu có)
- Lịch sử hội thoại gần nhất
- Các bộ lọc tìm kiếm: danh mục, thương hiệu, dòng sản phẩm, màu sắc
- Khoảng giá
- Thông số kỹ thuật dưới dạng bộ lọc số
- Danh sách sản phẩm được đề cập và danh sách đã định danh

Các thông số kỹ thuật được chuẩn hóa thành dạng (value, operator), ví dụ “RAM $\geq$ 8GB” được biểu diễn dưới dạng một cấu trúc có thể ánh xạ trực tiếp sang điều kiện SQL. Thiết

kế này giúp giảm đáng kể độ phức tạp ở bước sinh truy vấn và đảm bảo tính nhất quán giữa các thành phần của hệ thống.

node Parse

Node Parse chịu trách nhiệm chuyển đổi ngôn ngữ tự nhiên thành dữ liệu có cấu trúc. Thay vì sử dụng các phương pháp thủ công, hệ thống sử dụng mô hình ngôn ngữ với cơ chế structured output để đảm bảo đầu ra tuân theo một schema cố định.

Các thông tin được trích xuất bao gồm danh mục sản phẩm, các bộ lọc cơ bản, khoảng giá, thông số kỹ thuật và tên sản phẩm cụ thể nếu có. Prompt được thiết kế với các ràng buộc rõ ràng nhằm tránh suy đoán, chỉ tạo bộ lọc khi có giá trị cụ thể và chuẩn hóa các đơn vị đo lường.

Sau bước trích xuất, node thực hiện hợp nhất với trạng thái hiện tại. Việc hợp nhất không đơn thuần là ghi đè mà được thực hiện theo nguyên tắc cập nhật tối thiểu: chỉ những trường có giá trị mới mới được thay đổi, trong khi các thông tin khác được giữ nguyên. Ở đây có điểm quan trọng đó là cơ chế reset có điều kiện: khi danh mục sản phẩm thay đổi, toàn bộ các bộ lọc liên quan được đưa về trạng thái mặc định nhằm tránh sai lệch ngữ cảnh. Danh sách sản phẩm được đề cập được gộp và loại bỏ trùng lặp để phục vụ bước định danh.

node Resolve

Node Resolve thực hiện ánh xạ các tên sản phẩm tự nhiên sang định danh trong cơ sở dữ liệu. Cách tiếp cận được sử dụng là fuzzy matching dựa trên pg_trgm của PostgreSQL.

Với mỗi tên sản phẩm, hệ thống tính toán độ tương đồng với các tên trong cơ sở dữ liệu và chọn kết quả tốt nhất nếu vượt qua ngưỡng tối thiểu. Các trường hợp tên quá ngắn hoặc mang tính chung chung được loại bỏ trước khi truy vấn nhằm giảm nhiễu.

Kết quả định danh được gắn kèm một mức độ tin cậy. Giá trị này không chỉ dùng để đánh giá độ chính xác mà còn được đưa vào bước suy luận tiếp theo. Cụ thể, các kết quả có độ tin cậy thấp sẽ kích hoạt cơ chế xác nhận lại với người dùng trước khi tiếp tục tư vấn.

node Reason

Node Reason là thành phần điều phối trung tâm của hệ thống, có thể được xem như một “policy layer” nằm giữa mô hình ngôn ngữ và các công cụ truy xuất dữ liệu. Nhiệm vụ của node này là đánh giá trạng thái hiện tại và quyết định hành động tiếp theo.

Quy trình ra quyết định được tổ chức theo thứ tự ưu tiên. Trước hết, node kiểm tra các sản phẩm có độ tin cậy thấp để xác định xem có cần xác nhận lại với người dùng hay không. Tiếp theo, node đánh giá mức độ đầy đủ của thông tin; nếu thiếu các điều kiện quan trọng như danh mục hoặc bộ lọc, agent sẽ đặt câu hỏi bổ sung. Khi đã có đủ thông tin, node lựa chọn công cụ phù hợp để truy vấn dữ liệu. Cuối cùng, khi đã có kết quả từ công cụ, mô hình tổng hợp thông tin và sinh phản hồi.

Để đảm bảo tính ổn định, node Reason được thiết kế với cơ chế retry có kiểm soát. Trong trường hợp mô hình không sinh ra phản hồi hợp lệ hoặc không gọi công cụ khi cần thiết,

hệ thống sẽ bổ sung tín hiệu điều chỉnh và thực hiện lại bước suy luận mà không làm thay đổi trạng thái chính. Cách làm này giúp giảm ảnh hưởng của các lỗi ngẫu nhiên từ mô hình.

Thiết kế cơ chế gọi công cụ

Hệ thống sử dụng ba nhóm công cụ chính, tương ứng với ba loại nhu cầu truy vấn khác nhau: tìm kiếm sản phẩm theo điều kiện, truy vấn sản phẩm cụ thể và tìm kiếm ngữ nghĩa.

Các truy vấn SQL được xây dựng động từ trạng thái hội thoại. Đối với các điều kiện liên quan đến biến thể và thông số kỹ thuật, hệ thống sử dụng mệnh đề EXISTS thay vì JOIN trực tiếp nhằm tránh việc nhân bản bản ghi khi một sản phẩm có nhiều biến thể hoặc nhiều thông số. Điều này giúp giữ cho kết quả truy vấn gọn và chính xác.

Kết quả được giới hạn ở một số lượng nhỏ (ví dụ top 5 sản phẩm) để giảm tải cho bước sinh phản hồi của mô hình, đồng thời đảm bảo tính dễ đọc cho người dùng. Việc giới hạn này cũng đóng vai trò như một cơ chế kiểm soát chi phí và độ trễ của hệ thống.

Cá nhân hóa hội thoại

Hệ thống tích hợp dữ liệu lịch sử mua hàng từ cơ sở dữ liệu đồ thị để cá nhân hóa tư vấn. Khi bắt đầu phiên làm việc, agent truy vấn để xác định các thuộc tính phổ biến như thương hiệu và màu sắc mà người dùng đã lựa chọn trong quá khứ.

Các giá trị này được sử dụng để khởi tạo trạng thái hội thoại, đóng vai trò như các bộ lọc mặc định. Nhờ đó, agent có thể ưu tiên các sản phẩm phù hợp với sở thích của người dùng mà không cần yêu cầu nhập lại thông tin.

5.5.3. Thiết kế API

Lớp API đóng vai trò là cầu nối giữa AI Agent và các thành phần bên ngoài, bao gồm giao diện người dùng và hệ thống ghi nhận giao dịch. API được xây dựng theo kiến trúc RESTful, sử dụng FastAPI nhằm tận dụng khả năng xử lý bất đồng bộ, hiệu năng cao và dễ mở rộng.

Một đặc điểm quan trọng trong thiết kế là sự tách biệt giữa hai mức trạng thái:

- Ở mức HTTP, API hoạt động theo mô hình stateless
- Ở mức ứng dụng, trạng thái hội thoại được duy trì thông qua session

Quản lý phiên hội thoại

Mỗi phiên làm việc được định danh bằng một `session_id` do phía client cung cấp. Trên phía server, các phiên được lưu trữ trong một cấu trúc dictionary trong bộ nhớ, trong đó mỗi phần tử chứa:

- Trạng thái hội thoại (AgentState)
- Thời điểm tương tác cuối cùng

Khi một yêu cầu mới được gửi đến, hệ thống sẽ kiểm tra sự tồn tại của session. Nếu session chưa tồn tại, một trạng thái mới sẽ được khởi tạo; nếu đã tồn tại, trạng thái hiện tại sẽ được tái sử dụng và cập nhật.

Để tránh việc chiếm dụng bộ nhớ không cần thiết, hệ thống áp dụng cơ chế TTL (time-to-live). Các phiên không hoạt động trong một khoảng thời gian xác định (ví dụ 30 phút) sẽ bị xóa tự động khi có yêu cầu mới. Cách làm này đơn giản nhưng hiệu quả trong bối cảnh số lượng người dùng ở mức vừa phải.

Thiết kế endpoint hội thoại

Endpoint trung tâm của hệ thống là `/api/agent/chat`, chịu trách nhiệm tiếp nhận tin nhắn người dùng và trả về phản hồi từ agent.

Quy trình xử lý của endpoint này gồm các bước chính:

- Dọn dẹp các session hết hạn
- Lấy hoặc khởi tạo session tương ứng
- Cập nhật trạng thái với tin nhắn mới
- Thực thi đồ thị agent
- Lưu lại trạng thái và trả về kết quả

Trong quá trình này, trạng thái hội thoại được serialize thành dạng dictionary để truyền vào LangGraph, sau đó được khôi phục lại thành đối tượng `AgentState` sau khi xử lý. Cách làm này giúp tách biệt rõ giữa lớp API và lớp xử lý agent.

Endpoint này đóng vai trò như một “entry point” duy nhất cho toàn bộ luồng hội thoại, đảm bảo mọi tương tác đều đi qua cùng một pipeline xử lý.

Thiết kế endpoint ghi nhận giao dịch

Endpoint `/api/agent/purchase` được sử dụng để ghi nhận hành vi mua hàng của người dùng, phục vụ cho cơ chế cá nhân hóa.

Quy trình xử lý gồm hai bước. Trước hết, hệ thống kiểm tra sự tồn tại của biến thể sản phẩm dựa trên tên sản phẩm, màu sắc và giá. Bước kiểm tra này nhằm đảm bảo dữ liệu đầu vào hợp lệ và tránh ghi nhận các giao dịch không tồn tại.

Sau khi xác nhận hợp lệ, hệ thống thực hiện thao tác ghi vào Neo4j:

- Tạo hoặc tái sử dụng nút Customer
- Tạo hoặc cập nhật quan hệ BOUGHT với thuộc tính số lượng

Việc sử dụng MERGE trong truy vấn Cypher đảm bảo tính idempotent, cho phép cập nhật số lượng mua mà không tạo trùng dữ liệu.

Các endpoint hỗ trợ

Ngoài hai endpoint chính, hệ thống cung cấp một số endpoint hỗ trợ nhằm phục vụ việc kiểm tra và quản lý:

- `/api/agent/state/{session_id}`: trả về toàn bộ trạng thái hội thoại hiện tại, hữu ích cho debug và kiểm tra
- `/api/agent/reset/{session_id}`: xóa session tương ứng, buộc hệ thống tạo lại trạng thái mới
- `/api/agent/health`: kiểm tra trạng thái hoạt động của server và số lượng session hiện tại

6 Hiện thực và triển khai hệ thống

Chương này trình bày quá trình hiện thực và triển khai hệ thống dựa trên các thiết kế đã xây dựng ở chương trước. Các nội dung bao gồm cách hiện thực từng thành phần trong hệ thống, tổ chức pipeline xử lý và phương pháp triển khai trên môi trường thực tế, nhằm đảm bảo hệ thống có thể vận hành ổn định và hiệu quả.

6.1 Backend

6.1.1. Hiện thực Backend

Kiến trúc phân lớp và điều phối luồng dữ liệu

Tất cả các dịch vụ trong hệ thống được xây dựng trên một cấu trúc đồng nhất, áp dụng mô hình CQRS để tách biệt hoàn toàn luồng xử lý ghi dữ liệu và luồng truy vấn dữ liệu. Cấu trúc này giúp tối ưu hóa hiệu năng và tăng khả năng bảo trì hệ thống.

- **Nhóm Commands:** Tập trung vào việc xử lý các yêu cầu thay đổi trạng thái như tạo mới, cập nhật hoặc xóa. Luồng đi từ Controller qua Service Interface đến Service thực thi và được hỗ trợ bởi các Event Handler để xử lý các sự kiện bất đồng bộ liên quan đến Kafka.
- **Nhóm Queries:** Chuyên biệt cho các tác vụ lấy dữ liệu. Luồng này được tối ưu để trả về kết quả nhanh nhất mà không cần đi qua các bước kiểm tra nghiệp vụ phức tạp của luồng ghi.

Tổ chức đối tượng truyền tải dữ liệu DTO

Dữ liệu trao đổi giữa client và server hoặc giữa các dịch vụ với nhau được chuẩn hóa trong thư mục dto. Các lớp này được phân chia chi tiết thành Requests và Responses cho từng loại nghiệp vụ cụ thể như auth, address, hay payment_account. Việc tách bạch DTO cho Command và Query đảm bảo tính chặt chẽ của dữ liệu đầu vào và đầu ra, tránh việc phơi bày các thực thể cơ sở dữ liệu trực tiếp ra bên ngoài.

Tầng truy xuất dữ liệu Repository

Tương ứng với mô hình CQRS, tầng Repository được chia làm hai loại:

- **jpa_repositories:** Sử dụng Spring Data JPA để thực hiện các thao tác lưu trữ và cập nhật vào cơ sở dữ liệu.
- **read_only_repositories:** Được cấu hình tối ưu chỉ để đọc dữ liệu, giúp giảm tải cho hệ thống và tăng tốc độ phản hồi cho các yêu cầu truy vấn.

Các thành phần hỗ trợ hệ thống

- **Mapper và Factory:** Tầng mapper chịu trách nhiệm chuyển đổi dữ liệu giữa Model và DTO một cách tự động. Tầng factory được sử dụng để khởi tạo các đối tượng phức tạp, đảm bảo tính đóng gói của mã nguồn.
- **Email Handler và Util:** Các tác vụ hỗ trợ như gửi email thông báo được tách thành module riêng để không gây nghẽn luồng xử lý chính. Các hàm tiện ích dùng chung được tập trung tại util để tái sử dụng trên toàn dịch vụ.
- **Config:** Chứa toàn bộ các cấu hình hệ thống bao gồm cấu hình bảo mật, cấu hình Kafka và cấu hình kết nối cơ sở dữ liệu.

Tổ chức module common như một thư viện nội bộ

Module common được xây dựng như một thư viện dùng chung cho toàn bộ các microservices. Module này chứa các thành phần cốt lõi như cấu hình JWT Security, Global Exception Handling và các DTO dùng chung cho việc giao tiếp qua Kafka. Module này được đóng gói thành một dependency và được thêm vào file cấu hình Maven của từng dịch vụ, giúp đảm bảo tính đồng nhất về xử lý lỗi và cấu trúc dữ liệu trên toàn hệ thống.

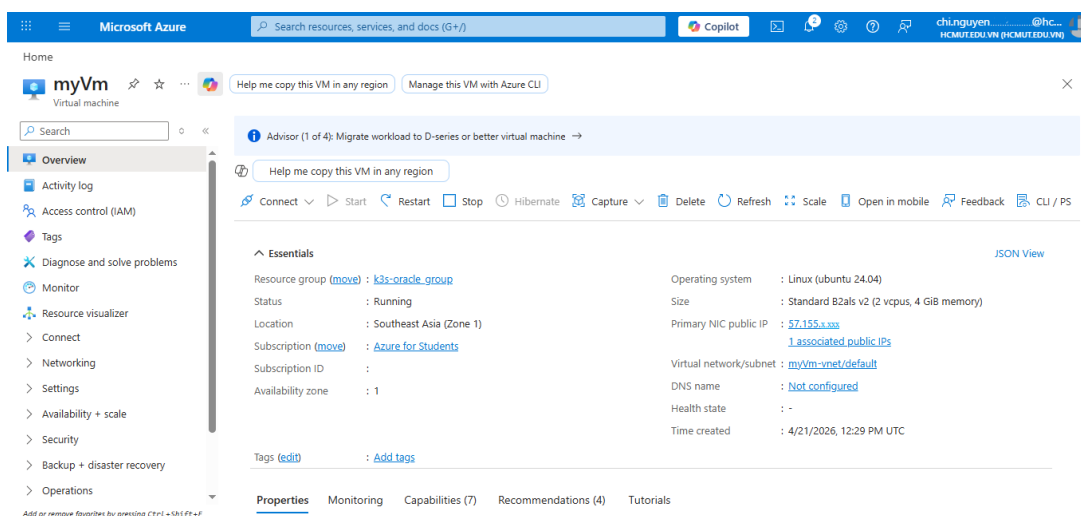
6.1.2. Triển khai Backend

Toàn bộ hệ thống được triển khai trên nền tảng điện toán đám mây Microsoft Azure, sử dụng máy ảo để vận hành các dịch vụ Backend và hạ tầng.

Thông số hạ tầng máy ảo

Hệ thống được vận hành trên một máy ảo với cấu hình cụ thể như sau:

- **Cấu hình:** Standard B2als v2 (2 vCPUs, 4 GiB memory).
- **Hệ điều hành:** Linux (Ubuntu Server).
- **Vị trí:** Southeast Asia (Zone 1).
- **Địa chỉ IP Public:** 57.155.x.xxx.



Hình 6.1: Giao diện quản lý Virtual Machine (myVm) trên Microsoft Azure Portal

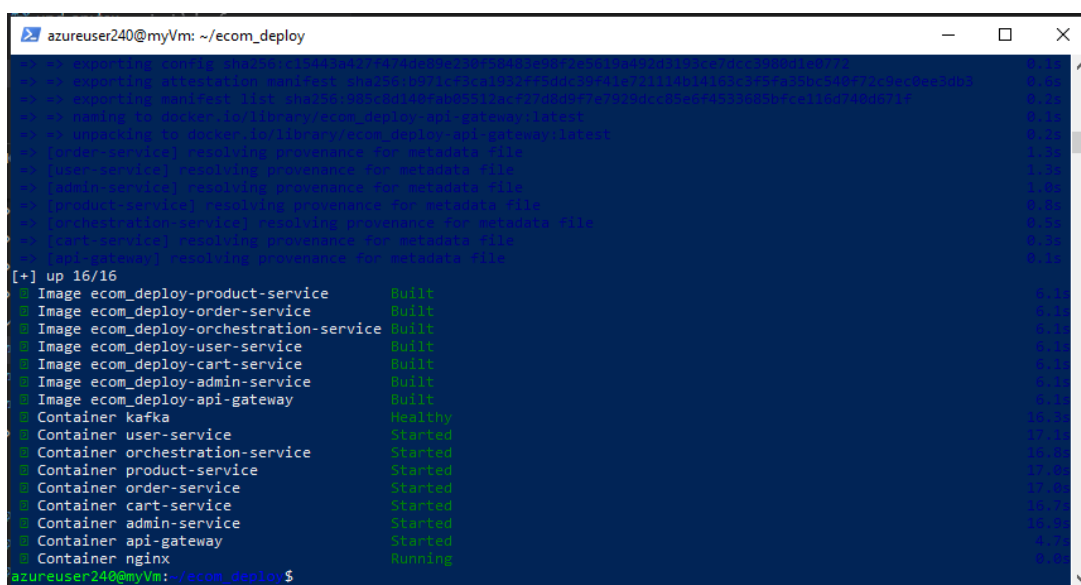
Hình 6.1 hiển thị trạng thái hoạt động của máy ảo được sử dụng để triển khai hệ thống Backend. Máy ảo thuộc dòng Standard B2als v2 với 2 vCPUs và 4 GiB Memory, chạy hệ điều hành Ubuntu 24.04 và được đặt tại khu vực Southeast Asia để đảm bảo độ trễ thấp nhất cho người dùng tại Việt Nam.

Quy trình đóng gói và xây dựng Image

Quá trình triển khai được thực hiện theo quy trình thủ công nhằm kiểm soát chặt chẽ các phiên bản thực thi:

- Các dịch vụ Backend được đóng gói thành file .jar tại máy cục bộ để đảm bảo tính ổn định của mã nguồn sau khi kiểm thử.
- File .jar cùng với Dockerfile được chuyển lên máy ảo Azure thông qua giao thức bảo mật.
- Tại máy ảo, lệnh docker build được thực thi để tạo các Docker Image. Các Image này sử dụng cơ chế Multi-stage build với môi trường JRE nhẹ để tối ưu hóa dung lượng lưu trữ trên ổ đĩa của máy ảo.

Hình 6.2 minh họa quá trình đóng gói thực tế, trong đó các lớp của image được xây dựng dựa trên Dockerfile đã thiết kế, đảm bảo môi trường vận hành được cô lập hoàn toàn.



```
azureuser240@myVm: ~/ecom_deploy
$ docker build -t ecom_deploy-product-service ./product-service
$ docker build -t ecom_deploy-order-service ./order-service
$ docker build -t ecom_deploy-orchestration-service ./orchestration-service
$ docker build -t ecom_deploy-user-service ./user-service
$ docker build -t ecom_deploy-cart-service ./cart-service
$ docker build -t ecom_deploy-admin-service ./admin-service
$ docker build -t ecom_deploy-api-gateway ./api-gateway
$ docker build -t ecom_deploy-nginx ./nginx
$ docker-compose up
[+] up 16/16
Image ecom_deploy-product-service Built 0.1s
Image ecom_deploy-order-service Built 0.1s
Image ecom_deploy-orchestration-service Built 0.1s
Image ecom_deploy-user-service Built 0.1s
Image ecom_deploy-cart-service Built 0.1s
Image ecom_deploy-admin-service Built 0.1s
Image ecom_deploy-api-gateway Built 0.1s
Container kafka Healthy 0.1s
Container user-service Started 0.1s
Container orchestration-service Started 0.1s
Container product-service Started 0.1s
Container order-service Started 0.1s
Container cart-service Started 0.1s
Container admin-service Started 0.1s
Container api-gateway Started 0.1s
Container nginx Running 0.1s
azureuser240@myVm:~/ecom_deploy$
```

Hình 6.2: Quá trình thực thi lệnh docker build để đóng gói dịch vụ trên máy ảo Azure

Điều phối hệ thống với Docker Compose

Hệ thống sử dụng Docker Compose trên máy ảo để quản lý vòng đời và sự kết nối giữa các container:

- **API Gateway Service:** API Gateway được triển khai như một dịch vụ độc lập bên trong mạng lưới microservices. Đây là thành phần duy nhất được cấu hình mở cổng

kết nối thông qua Azure Network Security Group (NSG) để tiếp nhận và điều phối các yêu cầu từ Client vào hệ thống.

- **Quản lý hạ tầng:** Docker Compose điều phối các container PostgreSQL, Kafka và Zookeeper, thiết lập các Docker Volume để đảm bảo dữ liệu không bị mất khi container khởi động lại.
- **Mạng nội bộ (Virtual Network):** Tất cả các service giao tiếp với nhau trong một mạng nội bộ ảo của Docker. Các dịch vụ nghiệp vụ được ẩn hoàn toàn khỏi Internet, chỉ có thể truy cập thông qua luồng điều phối của API Gateway Service, giúp tăng cường bảo mật cho toàn hệ thống.

6.2 AI Agent

6.2.1. Hiện thực AI Agent

Toàn bộ hệ thống AI Agent được phát triển bằng Python. Việc lựa chọn một ngôn ngữ thống nhất giúp giảm chi phí tích hợp giữa các module, đồng thời tận dụng được hệ sinh thái thư viện mạnh cho scraping, xử lý dữ liệu và xây dựng hệ thống AI.

Kiến trúc hiện thực có thể được xem như một pipeline nhiều tầng: dữ liệu được thu thập và chuẩn hóa, sau đó được lưu trữ vào các hệ quản trị phù hợp, và cuối cùng được khai thác thông qua AI Agent và API.

Hiện thực pipeline thu thập dữ liệu

Pipeline thu thập dữ liệu được tổ chức thành ba bước chính: crawl URL, crawl chi tiết sản phẩm và chuẩn hóa dữ liệu. Thay vì sử dụng các request HTTP đơn thuần, hệ thống sử dụng Playwright để mô phỏng trình duyệt, cho phép xử lý các trang có nội dung động.

Ở bước crawl URL, hệ thống duyệt qua các trang danh mục và thực hiện thao tác “xem thêm” (load more) để thu thập toàn bộ liên kết sản phẩm. Việc sử dụng Playwright giúp đảm bảo rằng tất cả sản phẩm đều được tải, kể cả khi nội dung được render bằng JavaScript.

Ở bước crawl chi tiết, mỗi URL sản phẩm được truy cập để trích xuất thông tin bao gồm:

- Thông tin chung: tên, thương hiệu, dòng sản phẩm
- Danh sách biến thể: màu sắc, giá
- Thông số kỹ thuật

Hiện thực tầng lưu trữ dữ liệu

Sau khi chuẩn hóa, dữ liệu được ghi vào ba hệ quản trị theo đúng vai trò thiết kế.

Đối với PostgreSQL, dữ liệu được insert theo thứ tự product → variant → product_spec để đảm bảo ràng buộc khóa ngoại. Các thao tác ghi được thực hiện trong transaction nhằm đảm bảo tính toàn vẹn: nếu một bước thất bại, toàn bộ dữ liệu liên quan sẽ không được lưu.

Đối với Neo4j, dữ liệu được ánh xạ thành các node và quan hệ. Việc ghi dữ liệu sử dụng câu lệnh MERGE để đảm bảo không tạo trùng node, đồng thời cho phép cập nhật khi dữ liệu đã tồn tại. Cách tiếp cận này giúp pipeline có thể chạy lại mà không gây sai lệch dữ liệu.

Đối với ChromaDB, dữ liệu văn bản (mô tả sản phẩm, FAQ, chính sách) được chia thành các đoạn nhỏ và lưu trực tiếp dưới dạng document. Hệ thống không tự tính embedding mà sử dụng cơ chế embedding tích hợp của ChromaDB để đảm bảo tính nhất quán giữa quá trình insert và truy vấn.

Hiện thực AI Agent

AI Agent được hiện thực dựa trên LangGraph, trong đó mỗi node trong thiết kế được ánh xạ trực tiếp thành một hàm xử lý. Trạng thái hội thoại được định nghĩa bằng lớp AgentState, cho phép chuyển đổi linh hoạt sang dạng dictionary khi tương tác với LangGraph.

Đồ thị xử lý được xây dựng thông qua Graph Builder, trong đó:

- Các node Parse, Resolve, Reason và Tools được khai báo rõ ràng
- Các cạnh (edges) được định nghĩa kèm điều kiện chuyển trạng thái
- Vòng lặp giữa Reason và Tools được hiện thực thông qua conditional edge

Luồng xử lý được biên dịch thành một graph hoàn chỉnh trước khi sử dụng. Khi nhận yêu cầu, agent chỉ cần gọi graph với trạng thái đầu vào và nhận lại trạng thái sau khi xử lý.

Các tool được triển khai như các hàm độc lập, mỗi hàm thực hiện truy vấn tới một nguồn dữ liệu cụ thể (PostgreSQL hoặc ChromaDB). Kết quả từ tool được đưa trở lại state để tiếp tục vòng suy luận.

Một điểm quan trọng trong hiện thực là việc kiểm soát lỗi. Các ngoại lệ phát sinh trong quá trình gọi tool không làm dừng hệ thống mà được đưa vào ngữ cảnh để node Reason xử lý. Điều này giúp agent có thể phản hồi linh hoạt ngay cả khi một phần hệ thống gặp lỗi.

Hiện thực lớp API

Lớp API được xây dựng bằng FastAPI, với endpoint trung tâm là /api/agent/chat. Endpoint này tiếp nhận tin nhắn người dùng và trả về phản hồi từ AI Agent.

Quản lý session được thực hiện bằng dictionary trong bộ nhớ, trong đó mỗi session chứa trạng thái hội thoại và timestamp của lần truy cập cuối. Khi có request mới, hệ thống sẽ:

- Dọn dẹp các session hết hạn
- Lấy hoặc khởi tạo session
- Cập nhật trạng thái với tin nhắn mới

- Gọi agent graph để xử lý
- Lưu lại trạng thái và trả về phản hồi

FastAPI lifespan được sử dụng để khởi tạo tài nguyên khi server khởi động, bao gồm:

- Kết nối PostgreSQL
- Kết nối Neo4j
- Khởi tạo ChromaDB
- Load graph của AI Agent

Việc khởi tạo một lần tại startup giúp giảm chi phí cho mỗi request và đảm bảo hiệu năng ổn định.

Ngoài endpoint hội thoại, hệ thống còn cung cấp endpoint `/api/agent/purchase` để ghi nhận hành vi mua hàng vào Neo4j. Endpoint này kiểm tra dữ liệu đầu vào và sử dụng MERGE để đảm bảo không tạo trùng quan hệ.

6.2.2. Triển khai AI Agent

Hệ thống được triển khai trên các nền tảng cloud khác nhau, theo hướng tận dụng các dịch vụ managed để giảm chi phí vận hành và đơn giản hóa quá trình cấu hình.

Cụ thể, lớp API được triển khai trên nền tảng Railway dưới dạng một service duy nhất. Toàn bộ mã nguồn, bao gồm cả AI Agent và dữ liệu ChromaDB, được đóng gói chung trong cùng một repository để đảm bảo tính đồng nhất khi deploy.

Cơ sở dữ liệu PostgreSQL được triển khai trên Supabase, giúp tận dụng các tính năng như backup, connection pooling và giao diện quản lý. API server kết nối đến Supabase thông qua connection string được cấu hình bằng biến môi trường.

Đối với cơ sở dữ liệu đồ thị, hệ thống sử dụng Neo4j AuraDB, cung cấp endpoint truy cập và cơ chế xác thực sẵn có.

ChromaDB được sử dụng dưới dạng lưu trữ cục bộ, dữ liệu vector được lưu trong thư mục và đóng gói cùng mã nguồn khi deploy.

Thiết kế triển khai này có thể được xem là một dạng “hybrid cloud architecture”, trong đó:

- API và AI Agent chạy trên Railway
- PostgreSQL được quản lý bởi Supabase
- Neo4j được quản lý bởi AuraDB
- ChromaDB được lưu trữ cục bộ

7 Kiểm thử và đánh giá hệ thống

Chương này trình bày quá trình kiểm thử hệ thống AI Agent nhằm đánh giá độ chính xác và tính ổn định của các thành phần. Các nội dung tập trung vào thiết kế test case, phương pháp đánh giá và phân tích kết quả, từ đó xác nhận hệ thống đáp ứng các yêu cầu đã đề ra.

7.1 Kiểm thử luồng nghiệp vụ cuối

Kiểm thử luồng nghiệp vụ cuối là giai đoạn quan trọng nhất nhằm xác minh khả năng phối hợp giữa các dịch vụ phân tán và hạ tầng truyền tin. Hệ thống được kiểm thử theo phương pháp hộp đen dựa trên các kịch bản sử dụng. Quy trình này xác minh sự phối hợp giữa các thành phần giao diện và hệ thống xử lý nghiệp vụ phân tán trong một luồng nghiệp vụ hoàn chỉnh.

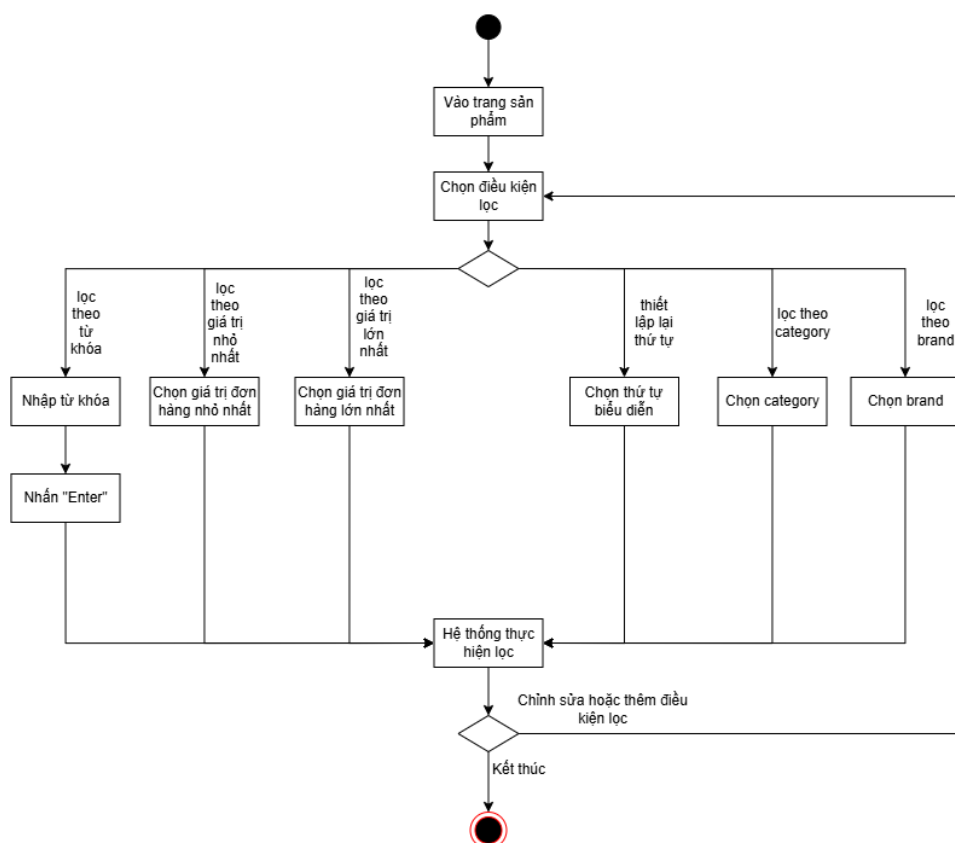
Khác với các bài kiểm thử đơn vị, kiểm thử luồng nghiệp vụ cuối tập trung vào việc mô phỏng hành trình thực tế của khách hàng trên hệ thống. Quá trình này đảm bảo rằng dữ liệu được luân chuyển chính xác qua các ranh giới dịch vụ từ khi người dùng xác thực danh tính, tìm kiếm sản phẩm cho đến khi hoàn tất thanh toán. Việc kiểm chứng tập trung vào sự đồng bộ trạng thái giữa các cơ sở dữ liệu độc lập thông qua cơ chế truyền tải sự kiện, đảm bảo hệ thống không xảy ra sai sót về logic nghiệp vụ khi có sự tương tác giữa nhiều thành phần cùng lúc.

Dưới đây là nội dung chi tiết về các kịch bản kiểm thử được phân tách theo từng tính năng cốt lõi của hệ thống. Mỗi tính năng bao gồm sơ đồ luồng nghiệp vụ và danh sách các ca kiểm thử tương ứng nhằm đảm bảo tính đúng đắn của logic phần mềm. Ở phần này, nhóm chỉ mô tả kịch bản kiểm thử cho một số tính năng chủ chốt.

7.1.1. Tính năng tìm kiếm và lọc sản phẩm

Sơ đồ dưới đây trình bày các bước thực hiện và các nhánh rẽ nghiệp vụ chi tiết khi người dùng tương tác với bộ lọc sản phẩm trên hệ thống.

Function: Tìm kiếm và lọc sản phẩm



Hình 7.1: Sơ đồ luồng nghiệp vụ tính năng tìm kiếm và lọc sản phẩm

Dựa trên các luồng xử lý tại Hình 7.1, danh sách các ca kiểm thử chi tiết cho tính năng tìm kiếm và lọc sản phẩm được tổng hợp tại Bảng 7.1 nhằm xác minh tính chính xác của dữ liệu trả về.

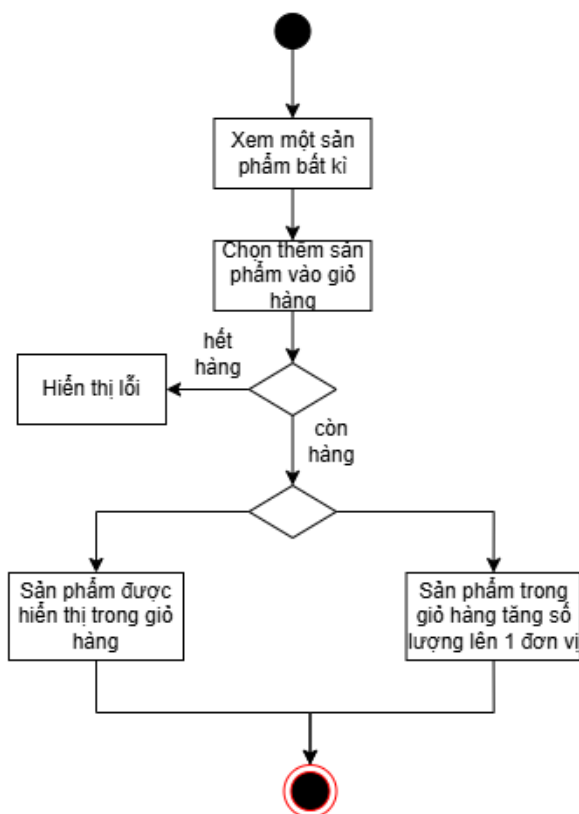
Bảng 7.1: Các ca kiểm thử chi tiết tính năng Tìm kiếm và lọc sản phẩm

ID	Tên kiểm thử	Mô tả kịch bản	Kết quả mong đợi	Trạng thái
TC-005-001	Tìm kiếm từ khóa	Nhập từ khóa vào ô tìm kiếm và nhấn Enter	Danh sách hiển thị sản phẩm có tên hoặc mô tả chứa từ khóa	Passed
TC-005-002	Lọc giá tối thiểu	Chọn mức giá thấp nhất tại bộ lọc	Hệ thống chỉ hiển thị sản phẩm có đơn giá lớn hơn hoặc bằng mức chọn	Passed
TC-005-003	Lọc giá tối đa	Chọn mức giá cao nhất tại bộ lọc	Hệ thống chỉ hiển thị sản phẩm có đơn giá nhỏ hơn hoặc bằng mức chọn	Passed
TC-005-004	Sắp xếp danh sách	Lựa chọn các tiêu chí sắp xếp như giá hoặc ngày tạo	Danh sách sản phẩm được thay đổi thứ tự hiển thị đúng theo yêu cầu	Passed
TC-005-005	Lọc theo danh mục	Chọn một loại sản phẩm cụ thể trên menu	Chỉ các sản phẩm thuộc danh mục đã chọn mới xuất hiện trên màn hình	Passed
TC-005-006	Lọc theo nhãn hàng	Lựa chọn thương hiệu sản phẩm mong muốn	Kết quả trả về đúng với thương hiệu người dùng đã yêu cầu lọc	Passed
TC-005-007	Kết hợp giá và hãng	Thiết lập khoảng giá kèm theo thương hiệu cụ thể	Sản phẩm hiển thị phải thỏa mãn đồng thời cả điều kiện giá và nhãn hàng	Passed
TC-005-008	Kết hợp danh mục và từ khóa	Tìm kiếm từ khóa bên trong một danh mục nhất định	Kết quả trả về thuộc đúng danh mục và có chứa từ khóa tìm kiếm	Passed
TC-005-009	Kết hợp nhiều tiêu chí	Lọc theo danh mục và giá kèm sắp xếp kết quả	Dữ liệu hiển thị đúng danh mục và khoảng giá theo thứ tự sắp xếp chọn lọc	Passed
TC-005-010	Áp dụng tất cả bộ lọc	Thiết lập toàn bộ các tiêu chí lọc hiện có	Hệ thống xử lý chính xác và trả về kết quả thỏa mãn mọi ràng buộc	Passed
TC-005-011	Từ khóa không tồn tại	Tìm kiếm với một chuỗi ký tự không có trong dữ liệu	Hệ thống thông báo không tìm thấy sản phẩm phù hợp với tiêu chí	Passed
TC-005-012	Sai biệt hãng và danh mục	Kết hợp nhãn hàng và danh mục không có sản phẩm	Màn hình hiển thị thông báo không có kết quả phù hợp	Passed
TC-005-013	Giá tối đa thấp hơn tối thiểu	Thiết lập mức giá tối đa nhỏ hơn mức tối thiểu	Hệ thống xử lý logic và không trả về sản phẩm nào hoặc báo lỗi lọc	Passed
TC-005-014	Giá tối thiểu cao hơn tối đa	Điều chỉnh giá tối thiểu vượt quá giới hạn giá tối đa	Hệ thống đảm bảo tính nhất quán và yêu cầu điều chỉnh lại khoảng giá	Passed

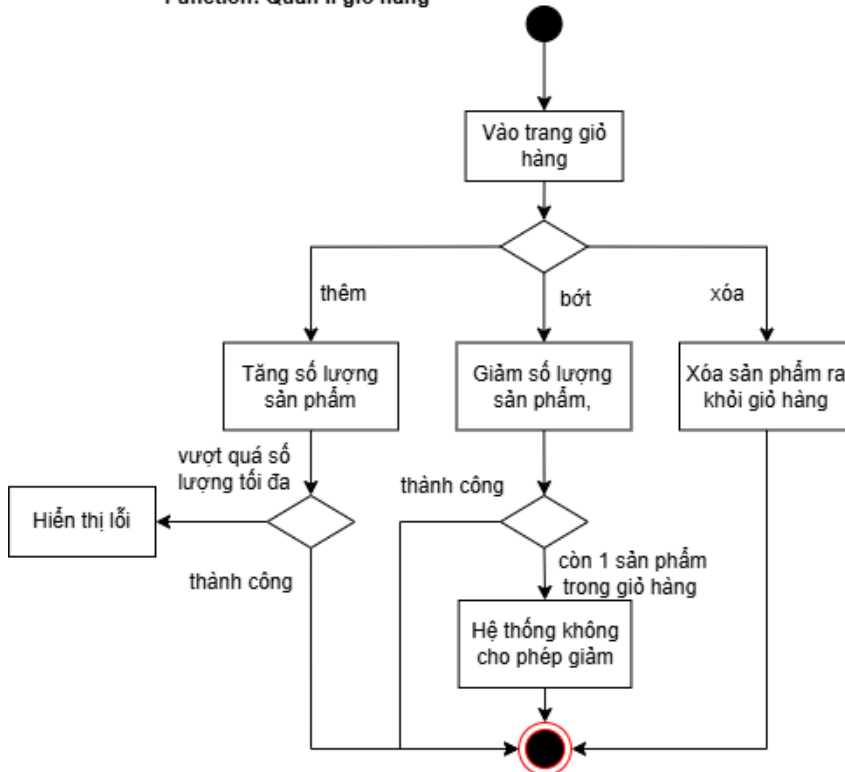
7.1.2. Tính năng thêm sản phẩm vào giỏ hàng và đặt hàng

Đây là luồng nghiệp vụ trọng tâm kết nối trực tiếp giữa các dịch vụ sản phẩm và kho hàng. Quy trình này bao gồm ba giai đoạn chính là thêm sản phẩm vào hệ thống lưu trữ tạm thời, điều chỉnh số lượng hàng hóa và thực hiện khởi tạo đơn hàng chính thức. Các sơ đồ dưới đây mô tả chi tiết các bước thực hiện và các nhánh rẽ logic tương ứng với từng giai đoạn.

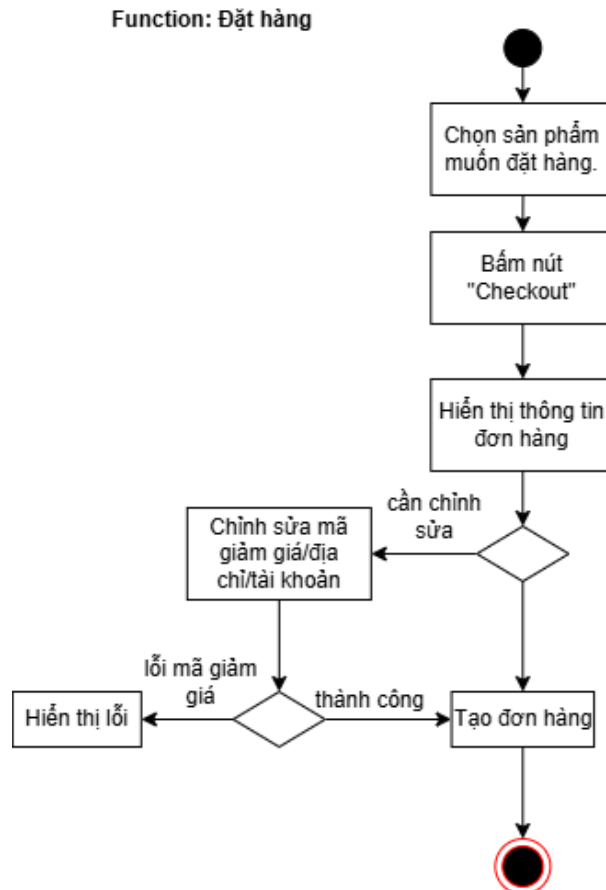
Function: Thêm sản phẩm vào giỏ hàng .



Function: Quản lí giỏ hàng



Hình 7.2: Sơ đồ luồng nghiệp vụ Thêm sản phẩm và Quản lý giỏ hàng



Hình 7.3: Sơ đồ luồng nghiệp vụ Đặt hàng

Dựa trên các quy trình tại Hình 7.2 và Hình 7.3, danh sách các ca kiểm thử dưới đây được thiết lập để xác minh tính chính xác của dữ liệu khi luân chuyển qua các trạng thái khác nhau trong chu trình mua sắm.

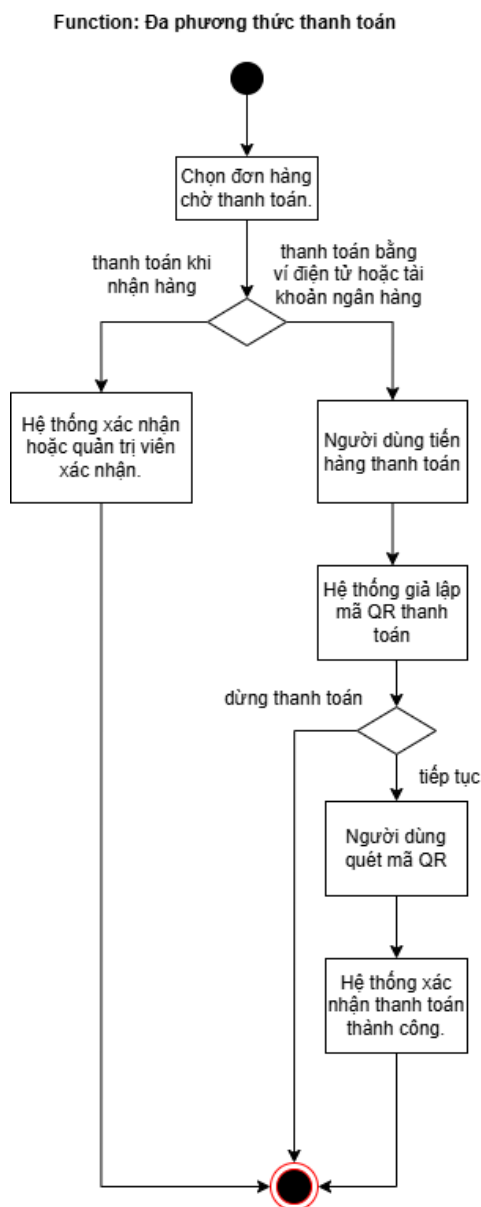
Bảng 7.2 dưới đây là danh mục mô tả các kịch bản kiểm thử chi tiết cho quy trình này. Các ca kiểm thử được thiết kế để xác minh tính chính xác của dữ liệu khi người dùng thực hiện thay đổi số lượng hàng hóa và áp dụng các chính sách khuyến mãi trong giai đoạn thanh toán.

Bảng 7.2: Các ca kiểm thử quy trình Quản lý giỏ hàng và Đặt hàng

ID	Tên kiểm thử	Mô tả kịch bản	Kết quả mong đợi	Trạng thái
Giai đoạn 1: Thêm vào giỏ hàng				
TC-007-001	Sản phẩm hết hàng	Thêm sản phẩm có số lượng tồn kho bằng không.	Hệ thống hiển thị thông báo lỗi và từ chối thao tác.	Passed
TC-007-002	Thêm sản phẩm mới	Thêm sản phẩm còn hàng chưa có trong giỏ.	Bản ghi mới được tạo với số lượng mặc định là một.	Passed
TC-007-003	Sản phẩm hiện hữu	Thêm sản phẩm đã tồn tại sẵn trong giỏ hàng.	Hệ thống tự động tăng số lượng sản phẩm lên một đơn vị.	Passed
Giai đoạn 2: Quản lý giỏ hàng				
TC-007-004	Tăng số lượng thành công	Nhấn nút tăng số lượng khi còn đủ hàng tồn kho.	Số lượng cập nhật và tổng giá trị giỏ hàng tăng chính xác.	Passed
TC-007-005	Tăng số lượng thất bại	Tăng số lượng vượt quá khả năng cung ứng của kho.	Hệ thống ngăn chặn hành động và hiển thị cảnh báo lỗi.	Passed
TC-007-006	Giảm số lượng thành công	Nhấn nút giảm khi sản phẩm đang có từ hai món trở lên.	Số lượng giảm đi một đơn vị và tổng tiền cập nhật đúng.	Passed
TC-007-007	Giảm số lượng thất bại	Giảm số lượng sản phẩm xuống dưới mức tối thiểu là một.	Hệ thống không thực thi lệnh giảm hoặc yêu cầu xóa món hàng.	Passed
TC-007-008	Xóa sản phẩm	Sử dụng chức năng loại bỏ sản phẩm ra khỏi giỏ.	Sản phẩm bị xóa hoàn toàn khỏi danh sách chờ mua.	Passed
Giai đoạn 3: Thanh toán và Đặt hàng				
TC-007-009	Thanh toán tiêu chuẩn	Đặt hàng với thông tin mặc định không thay đổi.	Đơn hàng khởi tạo thành công và lưu trữ vào cơ sở dữ liệu.	Passed
TC-007-010	Mã giảm giá hết hạn	Áp dụng mã Voucher đã quá thời hạn sử dụng.	Hệ thống từ chối áp dụng và yêu cầu kiểm tra lại mã.	Passed
TC-007-011	Mã giảm giá hết lượt	Áp dụng mã Voucher đã đạt giới hạn số lần sử dụng.	Hệ thống báo lỗi mã không còn hiệu lực sử dụng.	Passed
TC-007-012	Mã không tồn tại	Nhập chuỗi ký tự mã giảm giá sai lệch.	Hệ thống thông báo mã không hợp lệ hoặc không tìm thấy.	Passed
TC-007-013	Thay đổi thông tin	Cập nhật địa chỉ hoặc ghi chú trước khi đặt hàng.	Đơn hàng ghi nhận chính xác các thông tin mới nhất.	Passed

7.1.3. Tính năng thanh toán đa phương thức

Tính năng thanh toán đa phương thức cho phép người dùng linh hoạt lựa chọn giữa các hình thức giao dịch truyền thống và trực tuyến giả lập. Quy trình này xác minh khả năng xử lý của hệ thống khi tiếp nhận các phương thức thanh toán khác nhau và đảm bảo trạng thái đơn hàng được cập nhật chính xác sau khi giao dịch hoàn tất.



Hình 7.4: Sơ đồ luồng nghiệp vụ tính năng Thanh toán đa phương thức

Dưới đây là mô tả các kịch bản kiểm thử nhằm đánh giá tính đúng đắn của luồng thanh toán giả lập cũng như khả năng xử lý các tình huống hủy bỏ giao dịch từ phía người dùng.

Bảng 7.3: Các ca kiểm thử tính năng Thanh toán đa phương thức

ID	Tên kiểm thử	Mô tả kịch bản	Kết quả mong đợi	Trạng thái
TC-008-001	Phương thức COD	Lựa chọn hình thức thanh toán khi nhận hàng và xác nhận đơn.	Hệ thống ghi nhận phương thức COD và chuyển đơn hàng sang trạng thái chờ xác nhận.	Passed
TC-008-002	Hủy thanh toán trực tuyến	Lựa chọn thanh toán mã QR nhưng thực hiện hủy bỏ giao dịch.	Hệ thống dừng quy trình thanh toán an toàn và giữ nguyên trạng thái chờ của đơn hàng.	Passed
TC-008-003	Thanh toán trực tuyến thành công	Thực hiện quét mã QR giả lập và xác nhận hoàn tất giao dịch trực tuyến.	Hệ thống xác nhận thanh toán thành công và cập nhật trạng thái đơn hàng đã thanh toán.	Passed

7.1.4. Tài liệu quản lý kiểm thử chi tiết

Toàn bộ các bước thực hiện cụ thể và các trường hợp kiểm thử mở rộng cho toàn bộ hệ thống được lưu trữ tại tệp tin dưới đây:

Truy cập tài liệu chi tiết tại: [Google Sheet](#)

7.2 Kiểm thử yêu cầu phi chức năng

Việc kiểm thử các yêu cầu phi chức năng đóng vai trò then chốt trong việc đánh giá chất lượng vận hành của hệ thống, vượt ra ngoài các tính năng nghiệp vụ thông thường. Đối với một hệ thống dựa trên kiến trúc Microservices tích hợp AI Agent, các thử nghiệm tập trung vào việc xác minh tính ổn định, độ trễ của phản hồi AI, khả năng chịu tải và tính nhất quán dữ liệu giữa các dịch vụ phân tán. Các kịch bản kiểm thử dưới đây được thực hiện bằng phương pháp Kiểm thử hộp đen và kiểm thử tích hợp, sử dụng các công cụ hỗ trợ như k6, JUnit và Postman.

7.2.1. Kiểm thử tính khả dụng và trải nghiệm người dùng

Chi tiết các kịch bản và kết quả đánh giá về trải nghiệm người dùng cũng như tính dễ tiếp cận của hệ thống được tổng hợp cụ thể trong Bảng 7.4.

7.2.2. Kiểm thử hiệu năng hệ thống và AI Agent

Bảng 7.5 tổng hợp kết quả đánh giá khả năng vận hành của hệ thống dưới tải trọng giả lập và độ chính xác trong việc xử lý ngôn ngữ tự nhiên của trợ lý ảo AI.

7.2.3. Kiểm thử bảo mật và toàn vẹn dữ liệu phân tán

Bảng 7.6 trình bày kết quả thực nghiệm về các cơ chế phòng vệ tầng dữ liệu và xác thực quyền truy cập trên kiến trúc hệ thống.

7.3 Unit testing cho các thành phần logic backend

Unit testing là bước đầu tiên và quan trọng nhất trong chiến lược kiểm thử đa tầng, tập trung vào việc xác minh tính đúng đắn của các "tế bào" nghiệp vụ nhỏ nhất trong hệ

Bảng 7.4: Kết quả kiểm thử Usability và Learnability

ID	Tên kiểm thử	Các bước thực hiện	Dữ liệu đầu vào	Kết quả mong đợi	Trạng thái
NFR-TC-001	Tính nhất quán UI	Chuyển đổi qua lại giữa Home, Product, Cart.	Thao tác điều hướng.	Layout, màu sắc, font chữ đồng bộ 100%.	Passed
NFR-TC-002	Tốc độ điều hướng	Click vào chi tiết sản phẩm từ trang chủ.	ID sản phẩm bất kỳ.	Trang chi tiết hiển thị trong < 1.5s.	Passed
NFR-TC-003	Thông báo lỗi UI	Nhập sai định dạng email khi đăng ký tài khoản.	"abc@xyz" (thiếu .com).	Hiển thị cảnh báo đỏ ngay tại trường nhập liệu.	Passed
NFR-TC-004	Hiển thị phản hồi hệ thống	Thực hiện thêm sản phẩm vào giỏ hàng hoặc gửi đơn hàng.	Click nút "Thêm vào giỏ" hoặc "Thanh toán".	Hệ thống hiển thị thông báo xác nhận thành công ngay lập tức, không để người dùng phải chờ đợi hoặc phân vân.	Passed
NFR-TC-005	Tính dễ học	Nhờ người mới thực hiện luồng đặt hàng.	Không có hướng dẫn.	Người dùng hoàn thành đơn hàng trong thời gian ít hơn 5 phút.	Passed

Bảng 7.5: Kết quả kiểm thử hiệu năng và độ nhạy AI

ID	Tên kiểm thử	Các bước thực hiện	Dữ liệu đầu vào	Kết quả mong đợi	Trạng thái
P-01	Tốc độ AI	Nhập câu hỏi tư vấn cấu hình máy.	"Máy nào chơi game tốt?"	Phản hồi hiển thị trong < 3 giây.	Passed
P-02	Ngữ cảnh AI	Hỏi câu tiếp theo về sản phẩm vừa hỏi.	"Nó giá bao nhiêu?"	AI hiểu "Nó" là sản phẩm ở câu trước.	Passed
P-03	Chịu tải	Giả lập nhiều người dùng cùng lúc.	50 Virtual Users (k6).	Hệ thống không crash, CPU < 80%.	Passed
P-04	Tính độc lập	Tắt Service AI, thử truy cập Web.	Request trang chủ.	Web vẫn chạy, chỉ phần AI báo bảo trì.	Passed
P-05	Độ chính xác AI	Hỏi về giá hoặc cấu hình của một sản phẩm cụ thể.	"iPhone 15 bản 128GB giá bao nhiêu?"	AI trả về đúng giá tiền và dung lượng lưu trong Database.	Passed

Bảng 7.6: Kết quả kiểm thử bảo mật và đồng bộ dữ liệu

ID	Tên kiểm thử	Các bước thực hiện	Dữ liệu đầu vào	Kết quả mong đợi	Trạng thái
S-01	Hash mật khẩu	Đăng ký và check Database.	Pass: "hash.password".	Lưu dạng mã hóa BCrypt, không lộ text.	Passed
S-02	Xác thực JWT	Gọi API bảo mật không mang theo Token.	Request không Header.	Trả về mã lỗi 401 Unauthorized.	Passed
S-03	Nhất quán kho	Đặt hàng và check số lượng sản phẩm trong product service.	Đặt 01 iPhone 15.	Kho hàng giảm đúng 01 đơn vị.	Passed
S-04	Message Broker	Gửi message khi service nhận đang tắt.	Event: "order_created".	Service bật lại nhận và xử lý đủ tin.	Passed
S-05	Chặn SQL Injection	Nhập code SQL vào ô tìm kiếm.	"" OR 1=1 --	Hệ thống lọc bỏ hoặc báo không tìm thấy.	Passed

thống Backend. Tại đây, các phương thức xử lý logic, các quy tắc tính toán và các ràng buộc dữ liệu được kiểm tra một cách độc lập, tách biệt hoàn toàn với cơ sở dữ liệu, hệ thống tin nhắn hay các dịch vụ ngoại vi khác.

7.3.1. Chiến lược và mô hình kiểm thử

Nhờ kiến trúc phân lớp rõ ràng và khả năng tiêm phụ thuộc của Spring Boot, việc thực hiện kiểm thử đơn vị cho các dịch vụ nghiệp vụ trở nên thuận tiện và hiệu quả. Nhóm sử dụng khung làm việc JUnit 5 kết hợp cùng thư viện Mockito để cô lập hoàn toàn lớp logic cần kiểm thử khỏi các thành phần ngoại vi như Database hay các dịch vụ khác.

Trong hệ thống, tầng service đóng vai trò là "bộ não" điều khiển toàn bộ luồng nghiệp vụ. Do đó, kiểm thử đơn vị tại đây được thực hiện theo chiến lược White-box Testing, tập trung vào cấu trúc mã nguồn và tính chính xác của các thuật toán xử lý.

Để đảm bảo các bài thử nghiệm có tính cấu trúc và dễ bảo trì, nhóm áp dụng chặt chẽ mô hình Arrange – Act – Assert (AAA):

- Arrange: Đây là giai đoạn chuẩn bị ngữ cảnh cho bài test. Nhóm sử dụng annotation `@Mock` để tạo ra các đối tượng giả lập cho repository, kafka producer hoặc các feign client. Các dữ liệu mẫu được khởi tạo và định nghĩa hành vi thông qua `Mockito.when(...).thenReturn(...)`.
- Act: Gọi phương thức nghiệp vụ cụ thể trong service cần kiểm tra với các tham số đã chuẩn bị. Đây là bước kích hoạt logic lõi của hệ thống.
- Assert: Sử dụng các hàm Assertions như `assertEquals`, `assertNotNull`, `assertThrows` để đối chiếu kết quả thực tế với kỳ vọng. Đồng thời, nhóm sử dụng `Mockito.verify()` để xác nhận các tương tác với các thành phần phụ thuộc như việc lưu dữ liệu vào DB hay gửi tin nhắn qua Kafka, có diễn ra đúng như thiết kế hay không.

7.3.2. Công cụ thực hiện chuyên sâu

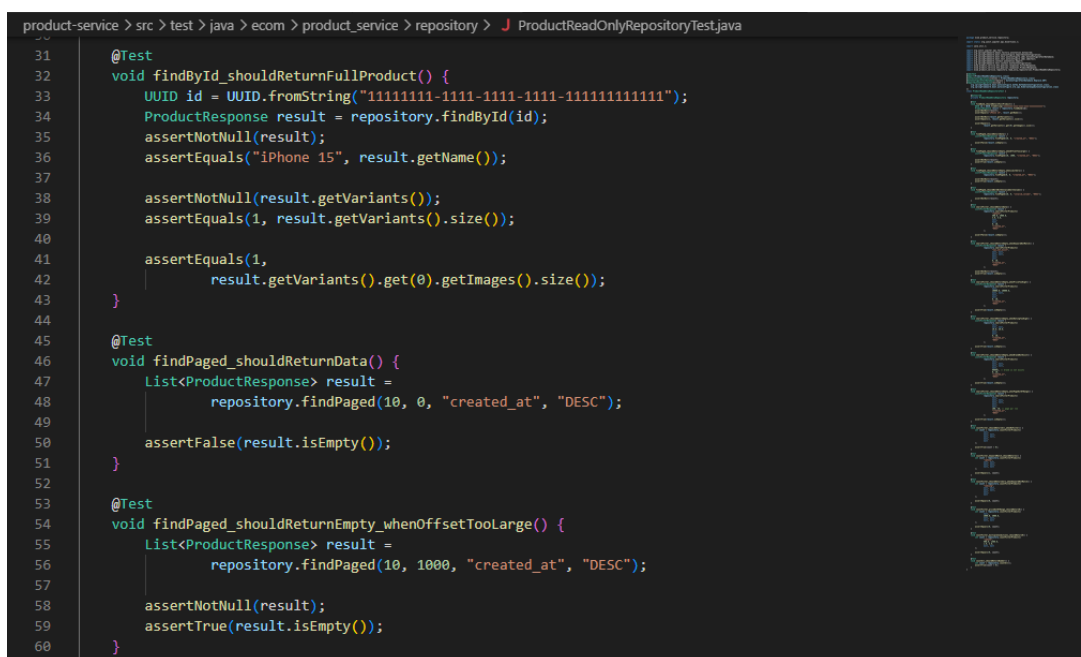
Nhóm sử dụng bộ công cụ tiêu chuẩn công nghiệp trong hệ sinh thái Java Spring Boot:

- JUnit 5 (Jupiter): Cung cấp các Annotation điều khiển luồng test như `@BeforeEach`, `@AfterEach` và các câu lệnh khẳng định mạnh mẽ.
- Thư viện then chốt để thực hiện kỹ thuật Isolation Testing. Nhờ Mockito, nhóm có thể kiểm tra logic của một Service mà không cần khởi chạy toàn bộ Application Context, giúp tốc độ thực thi test chỉ tính bằng mili giây.
- Được tích hợp để tăng khả năng đọc hiểu của mã nguồn kiểm thử thông qua các câu lệnh dạng chuỗi, giúp việc so sánh các đối tượng phức tạp như danh sách sản phẩm trở nên trực quan hơn.

7.3.3. Phân loại và các kịch bản kiểm thử tiêu biểu

Các kịch bản kiểm thử được chia thành ba nhóm chính để bao phủ toàn bộ các khả năng có thể xảy ra trong thực tế:

- Nhóm kịch bản thành công: Xác nhận hệ thống hoạt động đúng khi mọi điều kiện đầu vào đều lý tưởng.
- Nhóm kịch bản biên và điều kiện logic: Kiểm tra các tình huống nhạy cảm hoặc dữ liệu nằm ở ngưỡng giới hạn.
- Nhóm kịch bản xử lý ngoại lệ: Xác minh khả năng "tự vệ" của hệ thống trước dữ liệu sai lệch hoặc vi phạm quy tắc nghiệp vụ.



```

product-service > src > test > java > ecom > product_service > repository > J ProductReadOnlyRepositoryTest.java
31  @Test
32  void findById_shouldReturnFullProduct() {
33      UUID id = UUID.fromString("11111111-1111-1111-1111-111111111111");
34      ProductResponse result = repository.findById(id);
35      assertNotNull(result);
36      assertEquals("iPhone 15", result.getName());
37
38      assertNotNull(result.getVariants());
39      assertEquals(1, result.getVariants().size());
40
41      assertEquals(1,
42          result.getVariants().get(0).getImages().size());
43  }
44
45  @Test
46  void findPaged_shouldReturnData() {
47      List<ProductResponse> result =
48          repository.findPaged(10, 0, "created_at", "DESC");
49
50      assertFalse(result.isEmpty());
51  }
52
53  @Test
54  void findPaged_shouldReturnEmpty_whenOffsetTooLarge() {
55      List<ProductResponse> result =
56          repository.findPaged(10, 1000, "created_at", "DESC");
57
58      assertNotNull(result);
59      assertTrue(result.isEmpty());
60  }

```

Hình 7.5: Các kịch bản kiểm thử thuộc nhóm thành công trong *ProductReadOnlyRepositoryTest*

Hình 7.5 minh họa một số kịch bản kiểm thử tiêu biểu thuộc nhóm kịch bản thành công. Cụ thể, các phương thức như `findById_shouldReturnFullProduct` và `findPaged_shouldReturnData` xác nhận rằng hệ thống có thể truy vấn và trả về dữ liệu chính xác khi các tham số đầu vào hợp lệ và dữ liệu tồn tại trong cơ sở dữ liệu. Trong đó, kịch bản `findById` kiểm tra việc trả về đầy đủ thông tin của một sản phẩm, bao gồm các biến thể và hình ảnh liên quan, thể hiện tính toàn vẹn của dữ liệu. Đồng thời, kịch bản `findPaged` đảm bảo chức năng phân trang hoạt động đúng, trả về danh sách sản phẩm không rỗng khi tham số phân trang hợp lệ. Những kiểm thử này đóng vai trò quan trọng trong việc xác nhận các luồng chức năng chính của hệ thống hoạt động đúng như kỳ vọng trong điều kiện lý tưởng.

Tiếp theo, các Hình 7.6 và Hình 7.7 đại diện cho nhóm kịch bản biên và điều kiện logic, tập trung vào việc xác minh các ràng buộc nghiệp vụ phức tạp của hệ thống.

```

cart-service > src > test > java > ecom > cart_service > service > J CountPriceServiceTest.java
117
118 @Test
119 void shouldNotApplyDiscount_WhenEligibleTotalIsBelowMinOrderValue() {
120     when(cartItemReadOnlyRepository.validateCartItemsOwnership(userId, cartItemIds)).thenReturn(true);
121
122     CartItemDetail item1 = mock(CartItemDetail.class);
123     when(item1.getProductId()).thenReturn(productId1);
124     when(item1.getVariantId()).thenReturn(variantId1);
125     when(item1.getPrice()).thenReturn(new BigDecimal("40"));
126     when(item1.getCurrentQuantity()).thenReturn(1);
127
128     when(cartItemReadOnlyRepository.getCartItemDetailsByIds(cartItemIds)).thenReturn(List.of(item1));
129
130     DiscountCopy discountCopy = mock(DiscountCopy.class);
131     when(discountCopy.getMinOrderValue()).thenReturn(new BigDecimal("50"));
132
133     ProductVariantId applicableProduct = mock(ProductVariantId.class);
134     when(applicableProduct.getProductId()).thenReturn(productId1);
135     when(applicableProduct.getVariantId()).thenReturn(variantId1);
136
137     doReturn(Set.of(applicableProduct)).when(discountCopy).getApplicableProductIds();
138
139     when(discountReadOnlyRepository.findById(userDiscountId)).thenReturn(discountCopy);
140
141     CountPriceResponse response = countPriceService.countTotalPriceByCartItemIds(cartItemIds, userDiscountId, userId);
142
143     assertEquals(BigDecimal.ZERO, response.getTotalDiscount());
144     assertEquals(new BigDecimal("40"), response.getTotalPrice());
145 }

```

Hình 7.6: Kiểm tra điều kiện ngưỡng giá trị đơn hàng tối thiểu trong CountPriceServiceTest

Hình 7.6 minh họa kịch bản kiểm thử một trong những quy tắc nghiệp vụ quan trọng nhất của hệ thống giảm giá: điều kiện về giá trị đơn hàng tối thiểu. Trong trường hợp này, dữ liệu kiểm thử được thiết lập với tổng giá trị sản phẩm hợp lệ chỉ đạt 40 đơn vị, trong khi mã giảm giá yêu cầu ngưỡng tối thiểu là 50 đơn vị. Kết quả kiểm chứng qua các câu lệnh assertEquals xác nhận rằng mức giảm giá thực tế trả về bằng 0 và tổng giá trị đơn hàng được giữ nguyên. Điều này khẳng định hệ thống có khả năng kiểm soát chính xác các điều kiện ràng buộc, đảm bảo các chương trình khuyến mãi chỉ được kích hoạt khi người dùng thỏa mãn đầy đủ các tiêu chí kinh doanh đã đề ra.

Bên cạnh đó, Hình 7.7 trình bày kịch bản biên giả lập tình huống mức giảm giá tính toán được (100 đơn vị) vượt quá tổng giá trị của các sản phẩm có thể áp dụng giảm giá (50 đơn vị). Thông qua việc giả lập hành vi của CaculateDiscountValueUtil, kết quả cho thấy hệ thống đã thực hiện "chốt chặn" mức giảm giá tối đa bằng đúng giá trị sản phẩm, dẫn đến giá sau giảm không bị âm. Việc kiểm tra này không chỉ đảm bảo tính ổn định của thuật toán tính toán mà còn ngăn chặn các rủi ro về sai lệch dữ liệu tiền tệ trong các tình huống cấu hình giảm giá cực đoan.

```

cart-service > src > test > java > ecom > cart_service > service > J CountPriceServiceTest.java
196 @Test
197 void shouldCapDiscountAtEligibleTotal_WhenDiscountExceedsEligibleTotal() {
198     when(cartItemReadOnlyRepository.validateCartItemsOwnership(userId, cartItemIds)).thenReturn(true);
199
200     CartItemDetail item1 = mock(CartItemDetail.class);
201     when(item1.getProductId()).thenReturn(productId1);
202     when(item1.getVariantId()).thenReturn(variantId1);
203     when(item1.getPrice()).thenReturn(new BigDecimal("50"));
204     when(item1.getCurrentQuantity()).thenReturn(1);
205
206     when(cartItemReadOnlyRepository.getCartItemDetailsByIds(cartItemIds)).thenReturn(List.of(item1));
207
208     DiscountCopy discountCopy = mock(DiscountCopy.class);
209     when(discountCopy.getMinOrderValue()).thenReturn(new BigDecimal("20"));
210
211     ProductVariantId applicableProduct = mock(ProductVariantId.class);
212     when(applicableProduct.getProductId()).thenReturn(productId1);
213     when(applicableProduct.getVariantId()).thenReturn(variantId1);
214
215     doReturn(Set.of(applicableProduct)).when(discountCopy).getApplicableProductIds();
216
217     when(discountReadOnlyRepository.findById(userDiscountId)).thenReturn(discountCopy);
218
219     mockedDiscountUtil.when(() -> CaculateDiscountValueUtil.calculateDiscountValue(eq(new BigDecimal("50")), eq(discountCopy)))
220         .thenReturn(new BigDecimal("100"));
221
222     CountPriceResponse response = countPriceService.countTotalPriceByCartItemIds(cartItemIds, userDiscountId, userId);
223
224     assertEquals(new BigDecimal("50"), response.getTotalDiscount());
225     assertEquals(new BigDecimal("0"), response.getTotalPrice());
226 }
227

```

Hình 7.7: Kịch bản kiểm thử giới hạn mức giảm giá tối đa

Cuối cùng, để hoàn thiện chiến lược kiểm thử đơn vị, nhóm thực hiện xây dựng các kịch bản thuộc nhóm xử lý ngoại lệ nhằm xác minh khả năng phản ứng của hệ thống trước các tình huống dữ liệu sai lệch hoặc vi phạm quy trình nghiệp vụ.

```

order-service > src > test > java > ecom > order_service > service > J UpdateOrderStatusServiceTest.java
70 @Test
71 void updateOrderStatus_ThrowsException_WhenOrderNotFound() {
72     when(orderJpaRepository.findById(orderId)).thenReturn(Optional.empty());
73
74     assertThrows(ResourceNotFoundException.class, () ->
75         updateOrderStatusService.updateOrderStatus(orderId, DeliveryStatus.ON_SHIPPING, adminId)
76     );
77     verify(orderJpaRepository, never()).save(any());
78 }
79
80 @Test
81 void updateOrderStatus_ThrowsException_WhenCancelDeliveredOrder() {
82     sampleOrder.setDeliveryStatus(DeliveryStatus.DELIVERED);
83     when(orderJpaRepository.findById(orderId)).thenReturn(Optional.of(sampleOrder));
84
85     IllegalStateException exception = assertThrows(IllegalStateException.class, () ->
86         updateOrderStatusService.updateOrderStatus(orderId, DeliveryStatus.CANCELED, adminId)
87     );
88     assertEquals("Cannot cancel an order that has already been delivered", exception.getMessage());
89 }
90
91 @Test
92 void updateOrderStatus_ThrowsException_WhenUpdatingAlreadyCanceledOrder() {
93     sampleOrder.setDeliveryStatus(DeliveryStatus.CANCELED);
94     when(orderJpaRepository.findById(orderId)).thenReturn(Optional.of(sampleOrder));
95
96     IllegalStateException exception = assertThrows(IllegalStateException.class, () ->
97         updateOrderStatusService.updateOrderStatus(orderId, DeliveryStatus.ON_SHIPPING, adminId)
98     );
99     assertEquals("Cannot update status of a canceled order", exception.getMessage());
100 }
101

```

Hình 7.8: Các kịch bản kiểm thử xử lý ngoại lệ trong UpdateOrderStatusServiceTest

Hình 7.8 minh họa các bài kiểm thử ngoại lệ cho dịch vụ cập nhật trạng thái đơn hàng. Các kịch bản này tập trung vào việc bảo vệ tính toàn vẹn của luồng nghiệp vụ thông qua việc kiểm soát chặt chẽ các trạng thái đơn hàng.

Cụ thể, kịch bản `updateOrderStatus_ThrowsException` xác nhận hệ thống ném ra đúng các ngoại lệ `ResourceNotFoundException` khi không tìm thấy đơn hàng, hoặc `IllegalStateException` khi vi phạm quy tắc chuyển đổi trạng thái. Các "chốt chặn" logic này đảm bảo những đơn hàng đã giao (DELIVERED) không thể bị hủy và đơn hàng đã hủy (CANCELED) không thể tiếp tục vận chuyển trái phép. Thông qua việc kiểm chứng các thông báo lỗi cụ thể bằng `assertThrows`, nhóm khẳng định hệ thống luôn vận hành ổn định và ngăn chặn hiệu quả các thao tác sai sót từ phía quản trị viên.

7.3.4. Tổng hợp và thống kê kết quả kiểm thử đơn vị

Bảng 7.7 dưới đây là bảng thống kê tóm tắt kết quả Unit Testing cho các module cốt lõi:

STT	Dịch vụ	Tính năng	Pass	Fail	Tổng
1	User service	Quên mật khẩu	5	0	5
		Đăng nhập	9	0	9
		Tổng cộng User service	14	0	14
2	Product service	Truy vấn và lọc sản phẩm	17	0	17
		Quản lý tồn kho	11	0	11
		Quản lý mã giảm giá	5	0	5
		Tổng cộng Product service	33	0	33
3	Cart service	Thay đổi số lượng sản phẩm	14	0	14
		Tính toán giá và giảm giá	6	0	6
		Quản lý mục trong giỏ hàng	6	0	6
		Tổng cộng Cart service	26	0	26
4	Order service	Truy vấn báo cáo và phí vận chuyển	10	0	10
		Quản lý trạng thái và hủy đơn	11	0	11
		Thanh toán và Đánh giá	9	0	9
		Tổng cộng Order service	30	0	30
5	Orchestration service	Điều phối giao dịch phân tán (Saga)	11	0	11
		Xử lý xem trước và xác nhận thanh toán	10	0	10
		Tổng cộng Orchestration service	21	0	21
6	Admin service	Khởi tạo dữ liệu báo cáo	4	0	4
		Kết xuất và xử lý báo cáo (Report)	3	0	3
		Tổng cộng Admin service	7	0	7
TỔNG CỘNG HỆ THỐNG			131	0	131

Bảng 7.7: Thống kê chi tiết kết quả Unit Testing toàn bộ hệ thống Backend

7.4 Kiểm thử API với SwaggerUI

Sau khi hoàn tất kiểm thử đơn vị, nhóm tiến hành kiểm thử tầng API để xác minh khả năng giao tiếp giữa các thành phần và tính đúng đắn của dữ liệu trả về thực tế. Trong dự án này, nhóm sử dụng Swagger UI (OpenAPI 3.0) làm công cụ chính để thực hiện kiểm thử công trực tiếp trên từng Microservice.

7.4.1. Thiết lập môi trường kiểm thử

Nhóm cấu hình thư viện `springdoc-openapi` cho tất cả các dịch vụ, cho phép truy cập giao diện kiểm thử tại đường dẫn: `http://localhost:[port]/swagger-ui/index.html` hoặc địa chỉ IP `57.155.x.xxx` của Azure khi triển khai thực tế.

Các bước thực hiện kiểm thử tiêu biểu bao gồm:

- **Xác thực:** Đối với các Endpoint yêu cầu bảo mật, nhóm thực hiện đăng nhập để

lấy mã JWT Access Token, sau đó sử dụng tính năng "Authorize" trên Swagger để gắn token vào header của mọi yêu cầu tiếp theo.

- Thực thi: Truyền các tham số đầu vào như path variable hay query parameter hoặc cấu trúc JSON của request body trực tiếp trên giao diện và nhấn "Execute".
- Kiểm chứng: Nhóm đánh giá kết quả dựa trên các tiêu chí:
 - Mã trạng thái HTTP phù hợp.
 - Cấu trúc dữ liệu JSON trả về đúng tên trường và định dạng dữ liệu.
 - Sự thay đổi trạng thái của hệ thống (ví dụ: dữ liệu được lưu vào Database hoặc tin nhắn được gửi đi thành công).

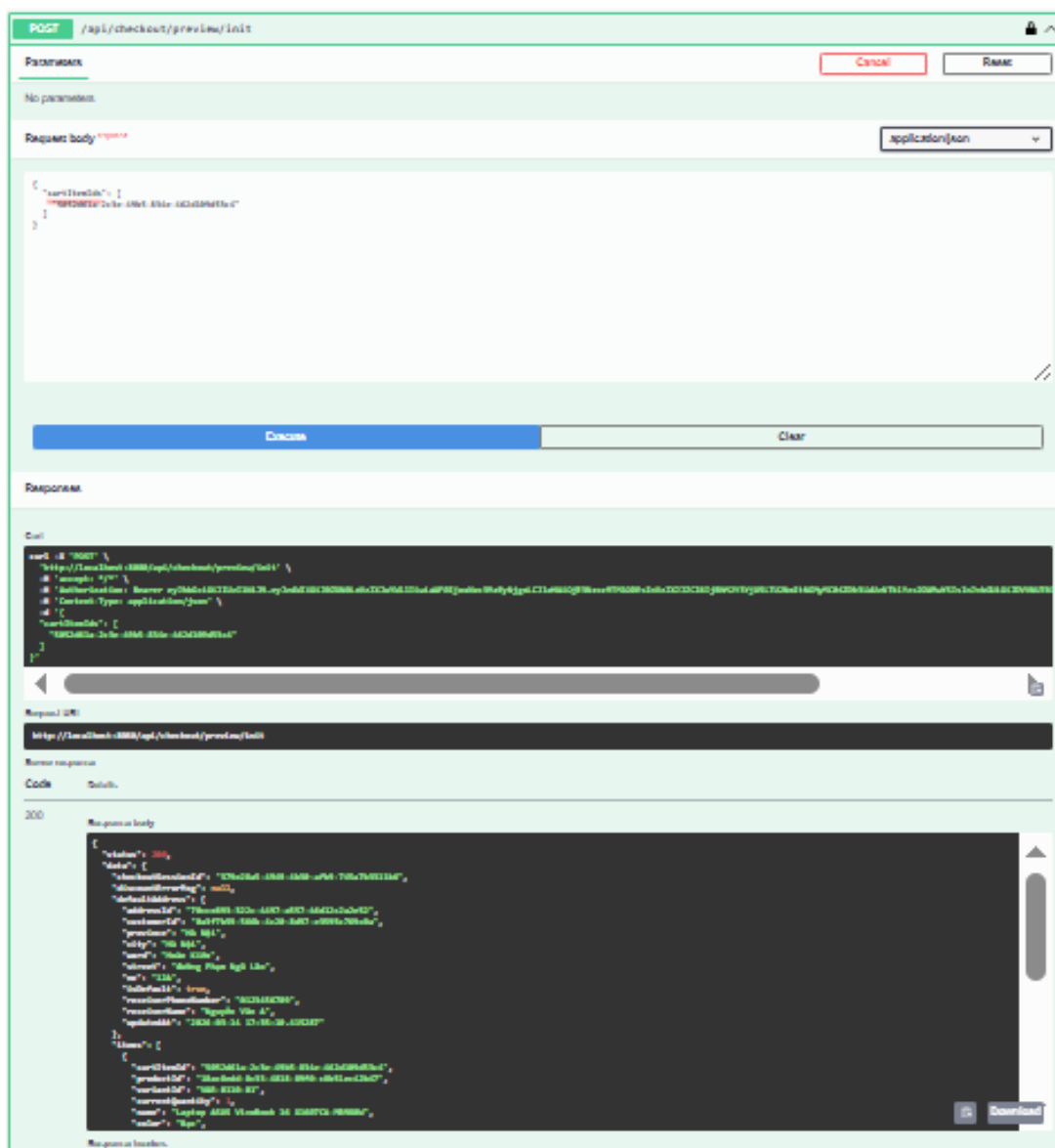
7.4.2. Phạm vi và kịch bản tiêu biểu

Để khẳng định tính sẵn sàng của hệ thống, nhóm tập trung thực hiện các kịch bản kiểm thử luồng nghiệp vụ chuẩn trực tiếp trên giao diện Swagger UI. Đây là giai đoạn quan trọng nhằm kiểm chứng sự phối hợp đồng bộ giữa các Microservices độc lập thông qua cơ chế gọi API trực tiếp và truyền tin bất đồng bộ.

A. Kiểm thử tích hợp các luồng chức năng cốt lõi

Nhóm thực hiện các chuỗi thao tác liên tục để đảm bảo luồng dữ liệu không bị ngắt quãng giữa các tầng xử lý:

- Luồng xác thực và định danh: Thực hiện đăng nhập để nhận JWT Token. Sau đó, sử dụng token này để kiểm tra quyền truy cập vào các tài nguyên bảo mật, đảm bảo tính năng phân quyền hoạt động đúng như thiết kế.
- Luồng truy vấn và lựa chọn sản phẩm: Kiểm tra các Endpoint lấy danh sách cây cảnh với các bộ lọc phức tạp. Xác nhận dữ liệu JSON trả về khớp hoàn toàn với cấu trúc DTO, đảm bảo đầy đủ thông tin về biến thể và hình ảnh sản phẩm.
- Luồng đặt hàng hoàn chỉnh: Đây là kịch bản phức tạp nhất, đi từ việc thêm sản phẩm vào giỏ hàng, tính toán giá trị đến khi khởi tạo quy trình thanh toán thông qua orchestration service.



Hình 7.9: Thực thi thành công API lấy thông tin đặt hàng trên Swagger UI

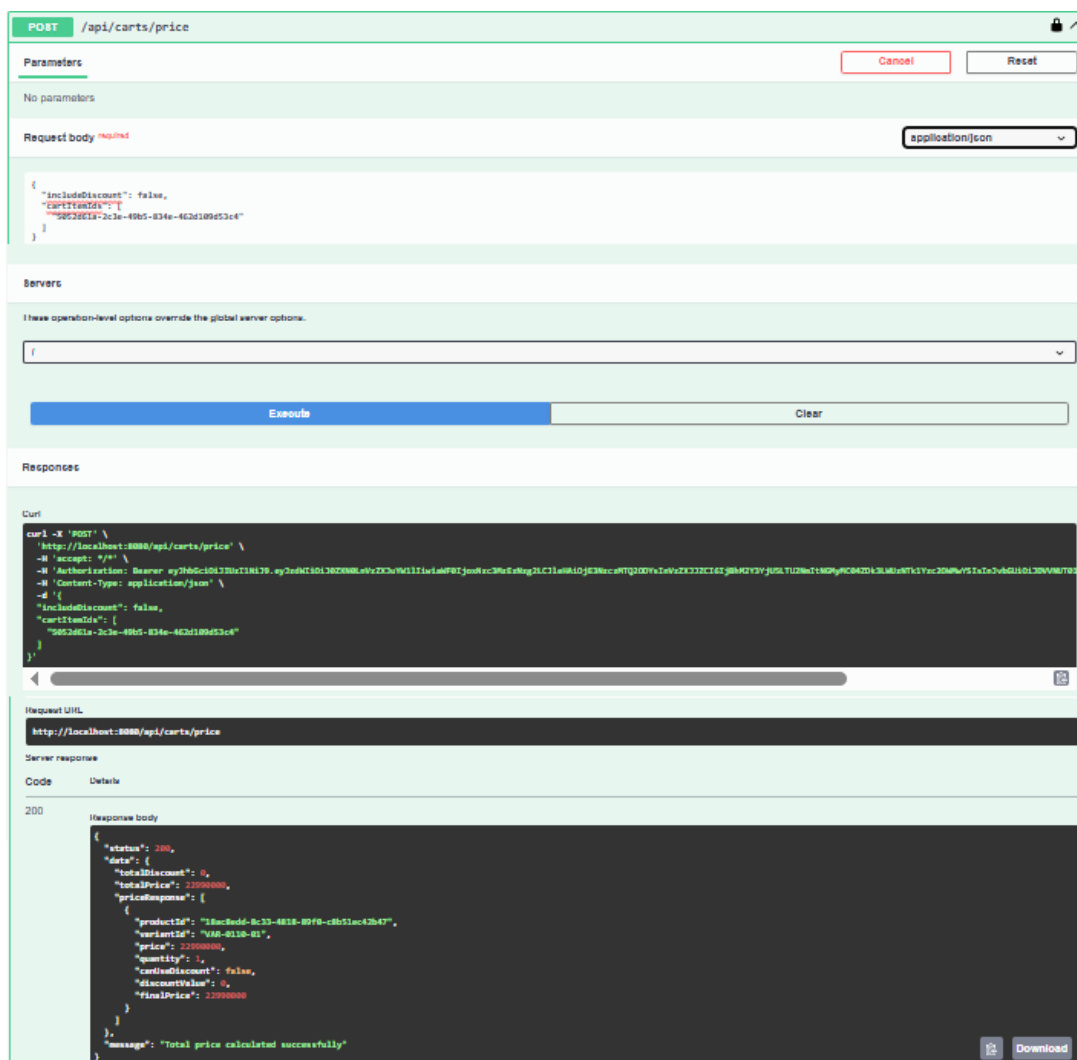
Giao diện Swagger hiển thị ở hình 7.9 kết quả khởi tạo quy trình thanh toán thành công với mã phản hồi 200. Hệ thống đã tiếp nhận danh sách các mã mục giỏ hàng và trả về thông tin chi tiết về địa chỉ mặc định cùng danh sách sản phẩm chờ xác nhận.

B. Xác minh phản hồi và tính nhất quán dữ liệu

Thông qua việc quan sát kết quả trực tiếp, nhóm tập trung đánh giá hai yếu tố then chốt là tính chính xác của dữ liệu và cơ chế báo lỗi hệ thống:

- Nhóm thực hiện kiểm chứng các thông tin định lượng quan trọng như tổng giá trị giỏ hàng sau khi áp dụng mã giảm giá tại cart, hoặc trạng thái tồn kho thực tế tại product. Các dữ liệu này phải phản hồi chính xác tuyệt đối với mã trạng thái 200 OK, đảm bảo sự đồng nhất giữa logic xử lý tại Backend và dữ liệu hiển thị cho người dùng.

- Thông báo trạng thái hệ thống: Backend được thiết lập cơ chế xử lý ngoại lệ tập trung. Trong các tình huống phát sinh lỗi logic hoặc sai lệch dữ liệu ngoài kịch bản chuẩn, hệ thống được cấu hình để phản hồi mã 500 Internal Server Error kèm theo thông báo mô tả nguyên nhân cụ thể, giúp đội ngũ phát triển dễ dàng theo dõi và khắc phục sự cố.



Hình 7.10: Minh họa phản hồi kết quả tính toán giá trị giỏ hàng trên Cart Service

Hình 7.10 minh họa kết quả kiểm thử API tính toán giá trị giỏ hàng thực tế sau khi áp dụng các quy tắc giảm giá. Dữ liệu phản hồi với mã trạng thái 200 OK đã xác nhận tính chính xác của tổng số tiền cần thanh toán và các mức chiết khấu đi kèm, đảm bảo logic nghiệp vụ tại cart service vận hành hoàn toàn khớp với các kịch bản kiểm thử đơn vị đã thiết lập trước đó.

Toàn bộ các API được kiểm thử thủ công đều phản hồi đúng theo đặc tả kỹ thuật ban đầu. Việc sử dụng Swagger UI không chỉ giúp nhóm phát hiện sớm các lỗi về định dạng dữ liệu mà còn đóng vai trò là tài liệu hướng dẫn trực quan cho việc tích hợp với giao diện sau này.

7.5 Kiểm thử AI Agent

Hệ thống AI Agent được kiểm thử theo phương pháp phân tầng, bao gồm kiểm thử đơn vị (unit testing) và kiểm thử tích hợp (integration testing). Mỗi cấp độ tập trung vào một thành phần cụ thể nhằm đảm bảo rằng từng phần hoạt động chính xác trước khi kiểm tra toàn bộ luồng hệ thống.

Chiến lược kiểm thử được xây dựng dựa trên hai nguyên tắc chính. Thứ nhất, mỗi thành phần quan trọng trong pipeline (parse, resolve, semantic search, API) đều được kiểm thử độc lập để xác định lỗi ở mức cục bộ. Thứ hai, các test case được thiết kế nhằm bao phủ cả trường hợp điển hình và trường hợp biên, đảm bảo hệ thống hoạt động ổn định ngay cả khi đầu vào không hoàn hảo.

Tổng quan các cấp độ kiểm thử được trình bày trong Bảng 5.1.

Cấp độ	Thành phần kiểm thử	Số test case	Kết quả
Unit	Node Parse	20	20/20 (100%)
Unit	Node Resolve	14	14/14 (100%)
Unit	Semantic Search	10	10/10 (100%)
Integration	REST API	11	10/11 (91%)

Bảng 7.8: Tổng quan các cấp độ kiểm thử

7.5.1. Kiểm thử node Parse

Mục tiêu và phương pháp

Node Parse đóng vai trò là bước đầu tiên trong pipeline, chịu trách nhiệm trích xuất ý định và các điều kiện lọc từ câu hỏi tiếng Việt tự nhiên. Sai sót ở bước này sẽ lan truyền và ảnh hưởng trực tiếp đến toàn bộ quá trình tư vấn.

Việc kiểm thử được thực hiện bằng cách cung cấp các câu truy vấn đầu vào và so sánh kết quả trích xuất với giá trị kỳ vọng theo từng trường dữ liệu (danh mục, thương hiệu, giá, thông số kỹ thuật và tên sản phẩm). Các trường số được so sánh trực tiếp, trong khi các trường chuỗi được so sánh không phân biệt hoa thường. Đối với thông số kỹ thuật, từng điều kiện lọc được kiểm tra cả giá trị và toán tử so sánh.

Thiết kế test case

Nhóm	Nội dung kiểm thử	Kết quả
Nhận diện danh mục	laptop, mobile, tivi, tablet, máy in	PASS
Nhận diện thương hiệu	Acer, Samsung, MSI	PASS
Lọc theo giá	giá tối đa, tối thiểu, khoảng giá	PASS
Thông số kỹ thuật	RAM, pin, màn hình, response time	PASS
Kết hợp nhiều điều kiện	giá + RAM + kích thước màn hình	PASS
Nhận diện sản phẩm	một hoặc nhiều sản phẩm trong câu	PASS
Trường hợp đặc biệt	câu hỏi chung, mô tả định tính	PASS

Bảng 7.9: Thiết kế test case node Parse

Các trường hợp đặc biệt đóng vai trò quan trọng. Ví dụ, với truy vấn mang tính tổng quát (“laptop tốt nhất hiện nay”), hệ thống không được suy đoán các trường không xuất hiện trong input. Tương tự, với mô tả định tính (“pin trâu”), hệ thống không được tự động chuyển thành giá trị số cụ thể.

Kết quả

Toàn bộ 20 test case đạt PASS (100%). Kết quả cho thấy node Parse có khả năng trích xuất chính xác thông tin từ nhiều dạng câu hỏi khác nhau, bao gồm cả các trường hợp biên và input không đầy đủ.

7.5.2. Kiểm thử node Resolve

Mục tiêu và phương pháp

Node Resolve có nhiệm vụ ánh xạ tên sản phẩm do người dùng nhập sang sản phẩm tương ứng trong cơ sở dữ liệu. Bài toán này bao gồm cả trường hợp tên đầy đủ, tên viết tắt và các biến thể không chính xác hoàn toàn.

Việc kiểm thử được thực hiện bằng cách cung cấp tên sản phẩm đầu vào và đánh giá kết quả trả về theo hai hướng:

- Trường hợp hợp lệ: hệ thống phải tìm đúng sản phẩm
- Trường hợp không hợp lệ: hệ thống phải từ chối (trả về rỗng)

Thiết kế test case

Nhóm	Mô tả	Kết quả
Khớp chính xác	Tên trùng hoàn toàn	PASS
Khớp một phần	Thiếu hoặc dư từ	PASS
Khớp điểm thấp	Tên đúng nhưng không đầy đủ	PASS
Không khớp	Tên quá ngắn hoặc không tồn tại	PASS

Bảng 7.10: Thiết kế test case node Resolve

Một điểm đáng chú ý là các trường hợp khớp điểm thấp vẫn được nhận diện đúng nhờ sử dụng ngưỡng similarity phù hợp.

Kết quả

Toàn bộ 14 test case đạt PASS (100%). Kết quả xác nhận node Resolve hoạt động ổn định trong cả hai tình huống: nhận diện đúng và từ chối đúng.

7.5.3. Kiểm thử tìm kiếm ngữ nghĩa

Mục tiêu và phương pháp

Thành phần semantic search chịu trách nhiệm truy xuất thông tin dựa trên ý nghĩa của câu truy vấn, thay vì khớp từ khóa trực tiếp.

Việc đánh giá được thực hiện bằng các chỉ số:

- Hit@1
- Hit@3

Kết quả

Chỉ số	Products	Policies	Tổng
Hit@1	100%	60%	80%
Hit@3	100%	100%	100%

Bảng 7.11: Kết quả semantic search

Phân tích

Kết quả cho thấy semantic search hoạt động hiệu quả, đặc biệt trên tập dữ liệu sản phẩm. Đối với policies, Hit@1 thấp hơn do nội dung tương tự nhau, nhưng Hit@3 đạt 100%.

7.5.4. Kiểm thử tích hợp API

Mục tiêu và phương pháp

Kiểm thử tích hợp nhằm xác minh toàn bộ hệ thống hoạt động đúng từ đầu đến cuối.

Kết quả

Nhóm	Mô tả	Kết quả
Endpoint cơ bản	health check, chat	PASS
Session	duy trì trạng thái nhiều lượt	PASS
Error handling	thiếu dữ liệu, sai session	PASS
Purchase flow	ghi nhận hành vi mua	PASS
Multi-turn chat	hội thoại nhiều bước	FAIL (timeout)

Bảng 7.12: Kết quả kiểm thử API

Phân tích

10/11 test case đạt PASS (91%). Trường hợp thất bại do timeout, không phải lỗi logic.

7.6 Tổng kết kiểm thử

Thành phần	Test case	Passed	Tỉ lệ
Node Parse	20	20	100%
Node Resolve	14	14	100%
Semantic Search	10	10	100%
API Integration	11	10	91%
Tổng	55	54	98%

Bảng 7.13: Tổng kết kết quả kiểm thử

Kết quả kiểm thử cho thấy hệ thống đạt độ chính xác cao trên các thành phần cốt lõi.

7.6.1. Hạn chế và hướng cải thiện

Mặc dù kết quả kiểm thử tích cực, vẫn tồn tại một số hạn chế như chưa đánh giá trên dữ liệu lớn và chưa so sánh với các phương pháp baseline.

Trong tương lai, hệ thống có thể được cải thiện bằng cách mở rộng tập dữ liệu kiểm thử và bổ sung các chỉ số đánh giá.

8 Kết luận và hướng phát triển

Trong chương này, nhóm tổng kết những kết quả đã đạt được trong quá trình nghiên cứu và triển khai hệ thống. Bên cạnh đó, chương cũng đề xuất một số hướng phát triển trong tương lai.

Tài liệu tham khảo

- [1] I. Smalley and P. Powell, “What is monolithic architecture?” <https://www.ibm.com/think/topics/monolithic-architecture>, [Online]. Truy cập vào: 01-04-2026.
- [2] Microsoft, “Microservices architecture style,” <https://learn.microsoft.com/en-us/azure/architecture/guide/architecture-styles/microservices>, 2025, [Online]. Truy cập vào: 09-11-2025.
- [3] IBM, “What is event-driven architecture?” <https://www.ibm.com/think/topics/event-driven-architecture>, [Online]. Truy cập vào: 14-3-2026.
- [4] C. Stryker and E. Kavlakoglu, “What is artificial intelligence (ai)?” <https://www.ibm.com/think/topics/artificial-intelligence>, [Online]. Truy cập vào: 09-12-2025.
- [5] Google, “What is an ai agent?” <https://cloud.google.com/discover/what-are-ai-agents>, [Online]. Truy cập vào: 09-11-2025.
- [6] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, “React: Synergizing reasoning and acting in language models,” 2023. [Online]. Available: <https://arxiv.org/abs/2210.03629>
- [7] Amazon, “What is rag (retrieval-augmented generation)?” <https://aws.amazon.com/what-is/retrieval-augmented-generation/>, [Online]. Truy cập vào: 09-11-2025.
- [8] IBM, “What is a knowledge graph?” <https://www.ibm.com/think/topics/knowledge-graph>, [Online]. Truy cập vào: 09-11-2025.
- [9] R. D. Caballar and C. Stryker, “What is a recommendation engine?” <https://www.ibm.com/think/topics/recommendation-engine>, [Online]. Truy cập vào: 09-11-2025.
- [10] VMware, “Spring boot: Build anything with spring boot,” <https://spring.io/projects/spring-boot>, 2024, [Online]. Truy cập vào: 24-04-2026.
- [11] Apache Software Foundation, “Apache kafka: A distributed streaming platform,” <https://kafka.apache.org/>, 2024, [Online]. Truy cập vào: 24-04-2026.